

Outline

1.1 Introduction

Tasks

1.2 Task Control in FreeRTOS

Task states

FreeRTOS API

1.3 Shared Data Access

1.4 Semaphores

Theory and applications

1.5 Message Queues

Theory and applications

1.6 Synchronisation

Theory and applications

1.7 Scheduling Theory

Priority scheduling

Round-robin scheduling

RMA, extended RMA, & deadlines

1.8 & 1.9 Scheduling Problems

Deadlock, starvation & livelock

Priority inversion

PART 1

PART 2

2.1 Interrupts & Exceptions

Hardware

Background/foreground systems

Interrupts vs tasks

2.2 RTOS Task Lists

Linked List implementation

Bit-mapped task list implementation

2.3 Context Switching

AVR example

Source code

2.4 Timing Functions

Clock Tick

Task Delay implementation

2.5 RTOS Objects

Semaphores

Message Queues

2.6 Writing an RTOS

Coursework

◆ Use **LPC2138** microprocessor & Keil development tools

- ❖ Hardware is available and easy to use
- ❖ Excellent simulation of all hardware + peripherals from any PC
- ❖ Development system is free to download.

◆ **FreeRTOS**

- ❖ GNU Public License (GPL) RTOS free to use, good for 32 bit systems
- ❖ API includes semaphores, message queues, etc, all with timeouts.
- ❖ Allows priority & round-robin scheduling, dynamic priorities
- ❖ Still under development – small feature-set

Language Prerequisites

- ◆ All RTOS we consider are written in C.
 - ❖ *This course assumes you are familiar with C syntax*
- ◆ Pseudocode will be written using C constructs
- ◆ RTOS source code makes use of C **macros** to implement simple operations
 - ❖ Macros use textual expansion to provide similar feature to functions but without time overhead of function call
- ◆ RTOS make use of C **conditional compilation** to customise code
- ◆ C preprocessor references:
 - ❖ Introduction in C web reference on previous slide
 - ❖ Comprehensive reference

http://en.wikipedia.org/wiki/C_preprocessor

```
#include "sysdefs.h"
#define NUMTASKS 10
#define MAXPRIO (NUMTASKS-1)
```

```
#define portEXIT_SWITCHING_ISR( SwitchRequired ) \
if( SwitchRequired ) \
{ \
    vTaskSwitchContext(); \
} \
/*this closing brace must match with { in ENTER macro*\
portRESTORE_CONTEXT();
```

```
#if( configUSE_16_BIT_TICKS == 1 )
    typedef unsigned portSHORT portTickType;
    #define portMAX_DELAY ( portTickType ) 0xffff
#else
    typedef unsigned portLONG portTickType;
    #define portMAX_DELAY ( portTickType ) 0xffffffff
#endif
```

Language Prerequisites (2)

- ◆ RTOS implementation is based on objects (tasks, semaphores, etc) which are defined by a **C structure** which implements an object *control block*. Control blocks are invariably accessed via **pointers**.
 - ❖ So a newly created task will be referenced via a pointer to its task control block
 - In FreeRTOS these pointers have type void * (pointer to anything). No type checking!
 - Void * pointer is given type name **xTaskHandle**
 - ❖ RTOS internals are full of pointers between structures
- ◆ **C uses consistent but very confusing syntax for describing pointers & pointer types. Revise/learn this on a "need-to-know" basis.**
 - ❖ You may need to read about this at some time during the coursework when you get confused. Depending on your background try one of:
 - ❖ [2 page C reference card](#) (summary of syntax will not help understanding)
 - ❖ [C FAQ](#) (Ch 4 on pointers). If you can answer these questions you don't need to read further!
 - ❖ C Pocket reference (on textbook slide). Good concise reference not for the beginner.
 - ❖ [Great intro to pointers & local memory](#), mostly language-independent but with examples in C from Nick Parlante Stanford. 31 pages, but comprehensive. Explains pointer diagrams etc. Good for those new to pointers, but fast.
 - ❖ [Long \(53 pages\) tutorial](#) (Jensen) on pointers, arrays & memory alloc in C, as above but longer.

Jargon used in these lectures

- ◆ **RTOS** – Real-Time Operating System
 - ❖ **Kernel** – RTOS code that implements the API & controls task switching
- ◆ **API** – Application Programming Interface (to an RTOS)
- ◆ **RTOS port** – version of RTOS for specific CPU/compiler
- ◆ **RMA** – Rate Monotonic Analysis
- ◆ **EDF** – Earliest Deadline First (scheduling)
- ◆ **PIP** – Priority Inheritance Protocol
- ◆ **CPP** – Ceiling Priority Protocol
- ◆ **PCP** – Priority Ceiling protocol
- ◆ **TCB** – Task Control Block (SCB – semaphore control block etc)

Lecture 1: Introduction

"I love deadlines. I like the whooshing sound they make as they fly by."

Douglas Adams

◆ What is a Real-Time Operating System?

- ❖ Controls CPU and resource allocation to application software
- ❖ Enables programs to be written easily which interact with time-critical hardware
- ❖ Allows "hard deadlines" to be guaranteed.
- ❖ Typically used for small embedded systems
- ❖ May be used for very complex systems – e.g. spacecraft

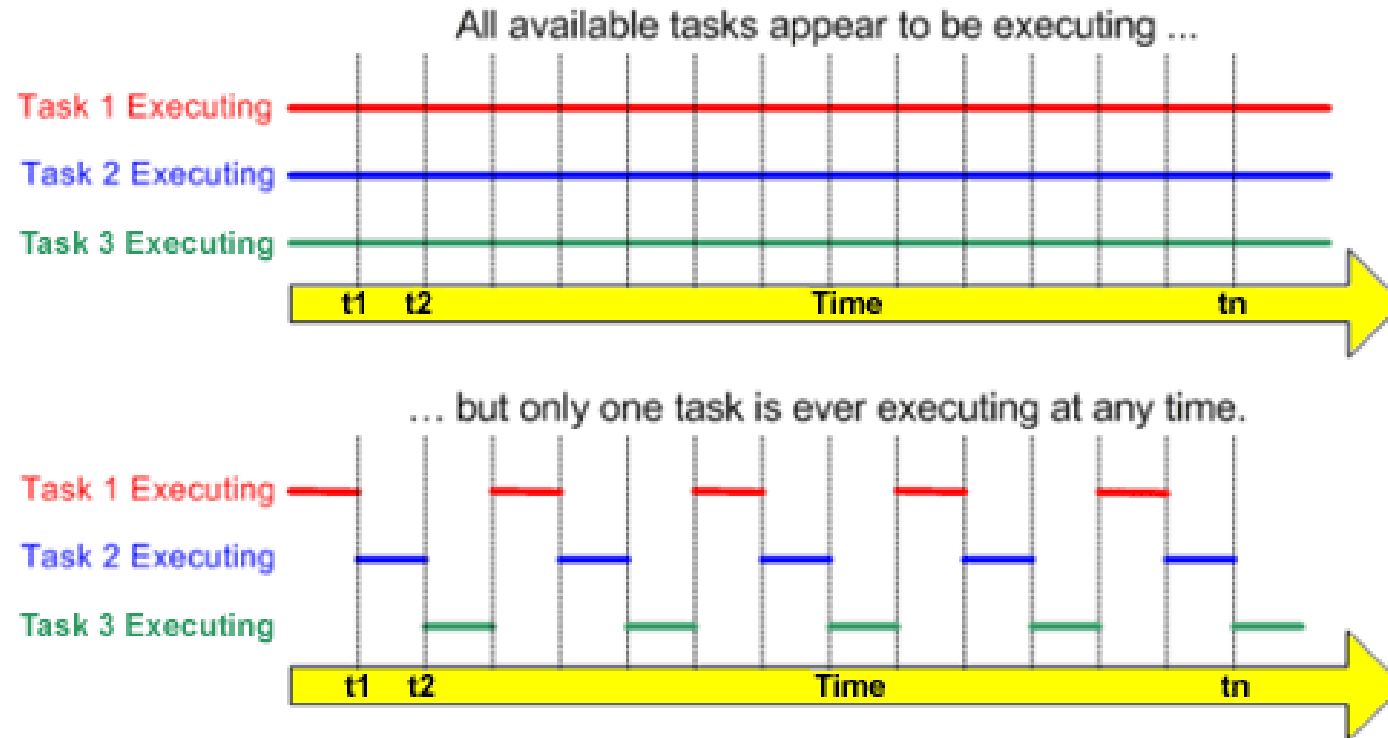
◆ Basic framework

- ❖ Work to be done is split into **tasks**
- ❖ Each task has a state: **ready** or **blocked**
- ❖ RTOS controls **which** task runs **when**
 - Called **scheduling**
- ❖ Tasks have **priorities** chosen so that hard deadlines are met

◆ Real-Time → add capabilities to meet hard deadlines

- ❖ Windows can't do this

Multitasking vs true concurrency



- ◆ Normally a single CPU runs multiple programs (called tasks).
- ◆ Multitasking - Moving rapidly between different tasks - gives the illusion that all are executing concurrently & independently.

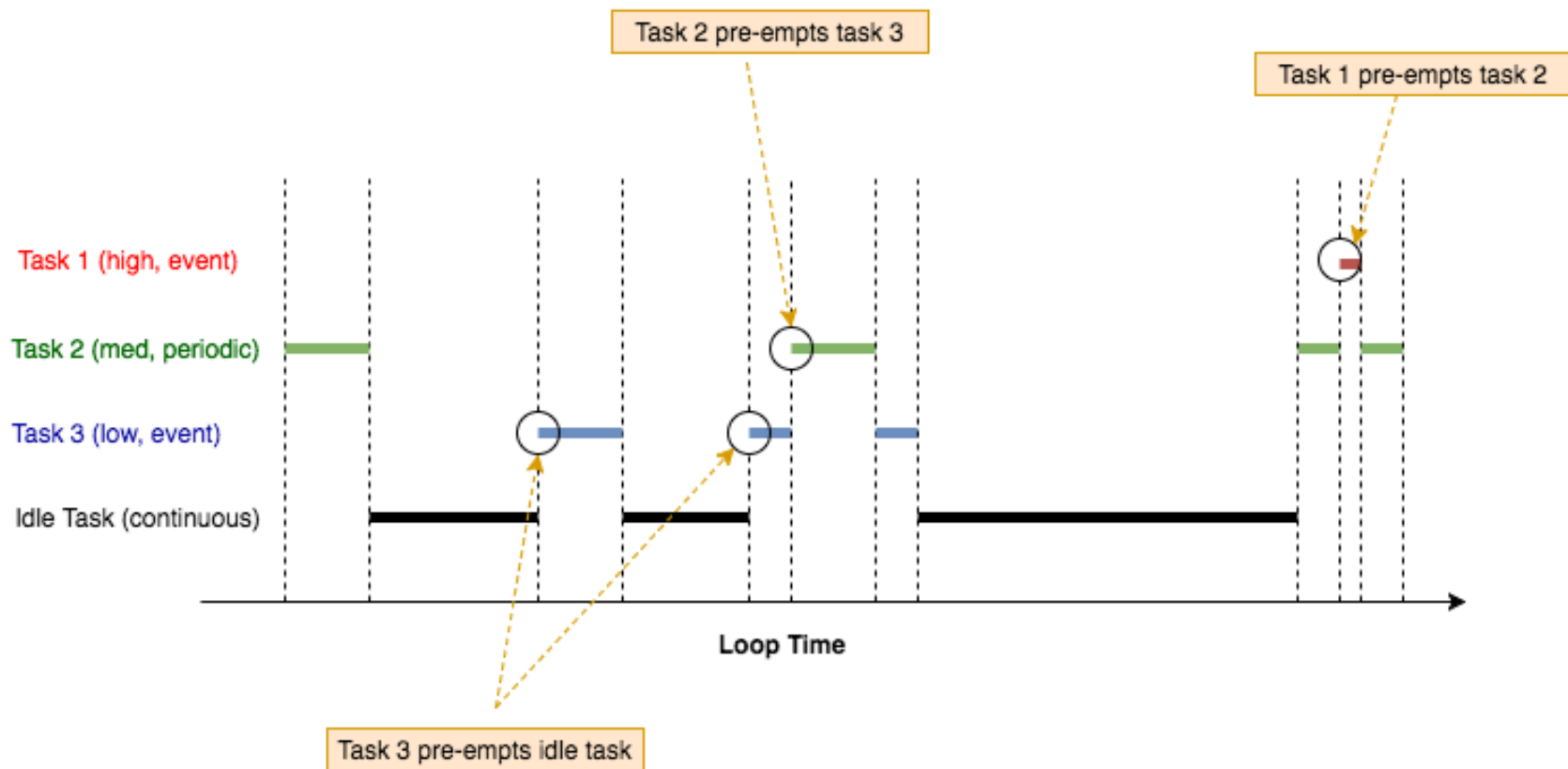
Task States

- ◆ Tasks in a multi-tasking system can be suspended by the RTOS as on previous slide, even though they are **ready** to run.
- ◆ Tasks may suspend themselves, e.g. by **sleeping** till a specific future time or **blocked** till a given future event
 - ❖ A blocked or sleeping task cannot be scheduled to run by the RTOS until it has been **woken up** (unblocked)
- ◆ Every task has a priority. The RTOS will schedule the highest priority task that is ready (i.e. not sleeping or blocked).
- ◆ The application programmer must assign a priority to each task

State	Sub-state	description
READY	Running	Only one task can be running at one time
	Ready-to-run	All others are waiting to be scheduled
BLOCKED	Sleeping	Task is waiting for fixed time – will be woken by system clock-tick
	Event Waiting	Task is waiting for event to happen – will be woken by event

Multi-tasking issues for RTOS

- ◆ **How are tasks scheduled (when does execution need to switch)?**
 - ❖ Lecture 1.2 - **task control** - shows how tasks are created and can themselves provoke task switches by going to "**sleep**".
 - ❖ Lectures 1.3 & 1.4 describe the most basic mechanisms - **semaphores** and **message queues** – that mediate **event-based task switching**.
- ◆ **How is task switching implemented?**
 - ❖ This is the key question for any operating system. In Part 2 we will look in detail about how this is done in two RTOS - FreeRTOS and MicroC/OS-II
 - Key to understanding implementation is the role of interrupts in RTOS – Lecture 2.1
- ◆ **How can deadlines be met?**
 - ❖ There is theory on different scheduling methods that guarantee real-time response – so called "hard deadlines"
 - ❖ 1.7 looks at the theory, 1.8 discusses situations where performance is bad, how to recognise them and how to cure them.



Trong FreeRTOS, mỗi một task sẽ được:

- Gán một priority
- Mỗi task chỉ có duy nhất một trạng thái vào bất kì thời điểm
- Chỉ duy nhất một task ở Running state vào bất kì thời điểm

Scheduler sẽ luôn chọn task đang ở Ready state và có priority cao nhất để đưa vào Running state.

A Simple real-time system

- ◆ The LPC2138 boards you will use for practical work have keys, and LCD, and both digital-to-analog output and analog-to-digital input.
- ◆ One real-time task is to ensure that key-strokes are processed and the LCD updated within 50ms. Anything slower than 50ms is a noticeable delay and not acceptable. Any time between 0 and 50ms is OK.
- ◆ A Task is implemented by a function with an infinite loop.
 - ❖ In these lectures we will use C language with pseudocode description of operations to illustrate code
 - ❖ Variables and functions will use Hungarian notation, where the type is indicated by prefix characters in name.
 - This is difficult at first, but easy to get used to and makes understanding code easier.

Task pseudocode

**v=>void (no
return value)**

**pv=>pointer to void
In C pointer to nothing means
pointer which can point to anything**

`void vKeyHandlerTask( void *pvParameters )`

`{`

/ Key handling is a continuous process and as such
the task is implemented using an infinite loop (as most
tasks are). */*

`for( ;; )`

`{`

[Suspend waiting for a key press]

[Process the key press]

`}`

`}`

**Square brackets
indicate
pseudocode**

Another task

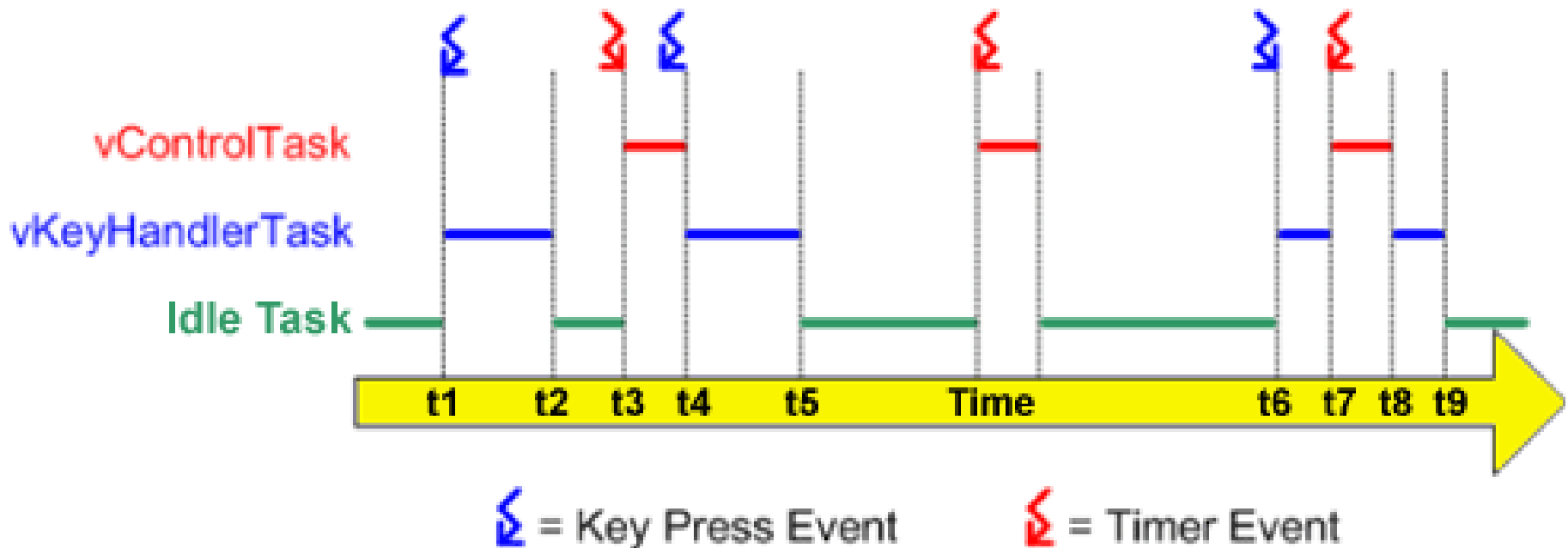
- ◆ To make this system more complex, suppose there is another task which performs A to D conversions, sampling the input voltage at regularly intervals and taking appropriate action

```
void vControlTask( void *pvParameters )  
{  
    for( ;; )  
    {  
        [Suspend waiting for 2ms since the start of the previous cycle ]  
        [Sample the input ]  
        [Filter the sampled input ]  
        [Perform control algorithm ]  
        [Output result ]  
    }  
}
```

Require sampling accuracy of 500us – deadline for this task is therefore 500us after timer event which initiates sampling

Execution Trace

- ◆ The way in which these two tasks behave can be analysed by looking at an execution trace of the system
 - ❖ The idle task is added by the RTOS to ensure that something is executing even when the application tasks are both suspended
 - ❖ The control task must be higher priority since has shorter deadline
 - 500us vs 50ms



Execution trace details...

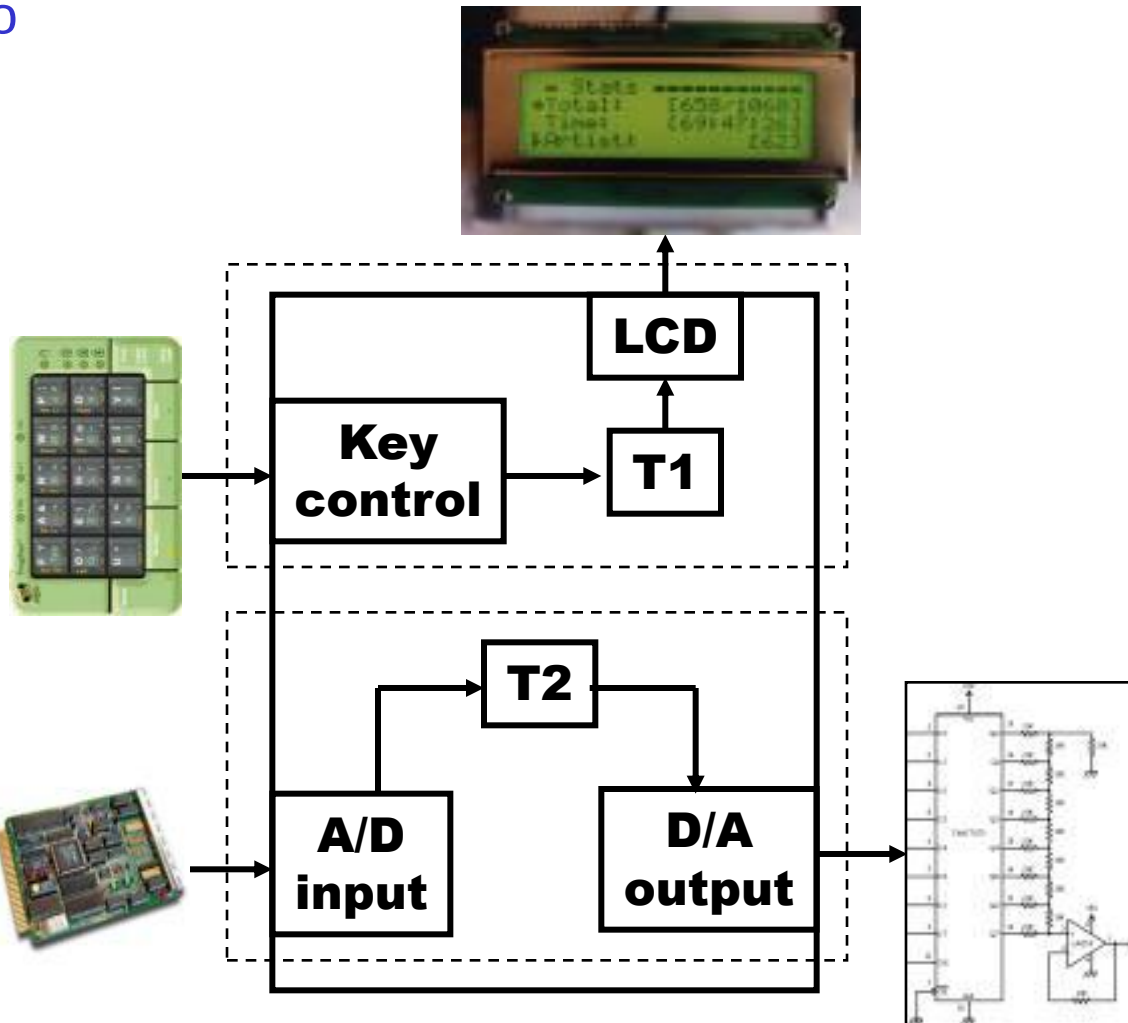
- a) At the start neither of our two tasks are able to run - **vControlTask** is waiting for the correct time to start a new control cycle and **vKeyHandlerTask** is waiting for a key to be pressed. Processing time is given to the idle task.
- b) At time **t1**, a key press occurs. **vKeyHandlerTask** is now able to execute—it has a higher priority than the idle task so is given processing time.
- c) At time **t2** **vKeyHandlerTask** has completed processing the key and updating the LCD. It cannot continue until another key has been pressed so suspends itself and the idle task is again resumed.
- d) At time **t3** a timer event indicates that it is time to perform the next control cycle. **vControlTask** can now execute and as the highest priority task is scheduled processing time immediately.
- e) Between time **t3** and **t4**, while **vControlTask** is still executing, a key press occurs. **vKeyHandlerTask** is now able to execute, but as it has a lower priority than **vControlTask** it is not scheduled any processing time.

... Cont'd

- f) At **t4** **vControlTask** completes processing the control cycle and cannot restart until the next timer event—it suspends itself. **vKeyHandlerTask** is now the task with the highest priority that is able to run so is scheduled processing time in order to process the previous key press.
- g) At **t5** the key press has been processed, and **vKeyHandlerTask** suspends itself to wait for the next key event. Again neither of our tasks are able to execute and the idle task is scheduled processing time.
- h) Between **t5** and **t6** a timer event is processed, but no further key presses occur.
- i) The next key press occurs at time **t6**, but before **vKeyHandlerTask** has completed processing the key a timer event occurs. Now both tasks are able to execute. As **vControlTask** has the higher priority **vKeyHandlerTask** is suspended before it has completed processing the key, and **vControlTask** is scheduled processing time.
- j) At **t8** **vControlTask** completes processing the control cycle and suspends itself to wait for the next. **vKeyHandlerTask** is again the highest priority task that is able to run so is scheduled processing time so the key press processing can be completed.

Simple Real-Time Application Design – “outside-in” approach

- ◆ Decompose computation into multiple tasks.
- ◆ Move from peripherals inwards, specifying a separate task for each independent I/O device and adding extra tasks **as necessary** to perform associated computation
- ◆ Additional tasks may not be needed, e.g. previous example uses just two tasks (dotted lines)
- ◆ Good design uses minimum number of tasks necessary to represent concurrent operations



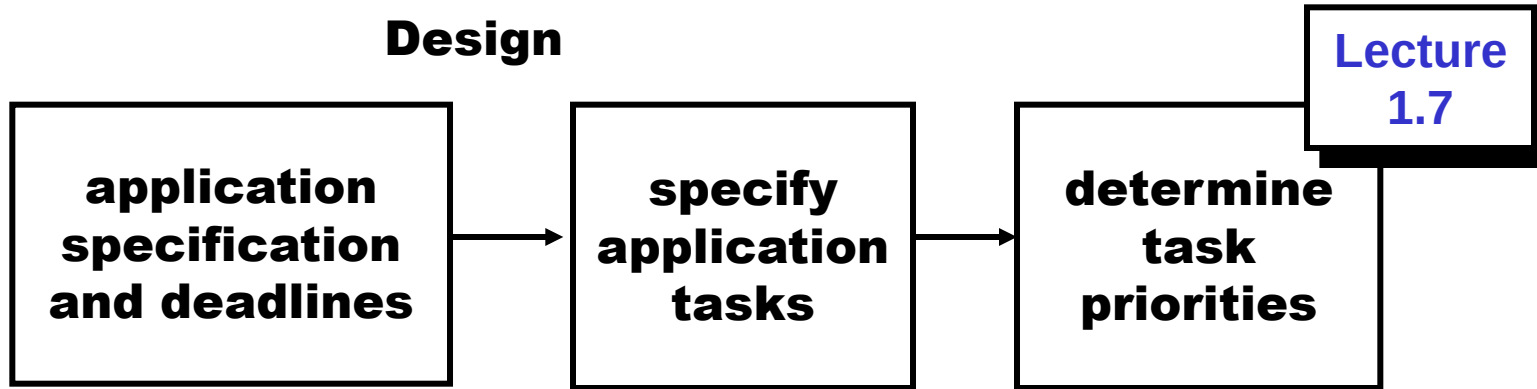
Real-Time Task Structure

- ◆ In a normal OS tasks often run continuously without blocking till endpoint.
- ◆ In an RTOS typically tasks will run in response to an event, and then block until the next event happens
 - ❖ Real-time tasks spend most time blocked
 - ❖ Most real-time tasks run forever
- ◆ All tasks must block (except the lowest priority task) in order to allow lower priority tasks to run.
 - ❖ Total CPU utilisation must be $< 100\%$
 - ❖ Lowest priority **Idle Task** may run continuously "soaking up" any spare CPU
 - Avoids special case of no task running

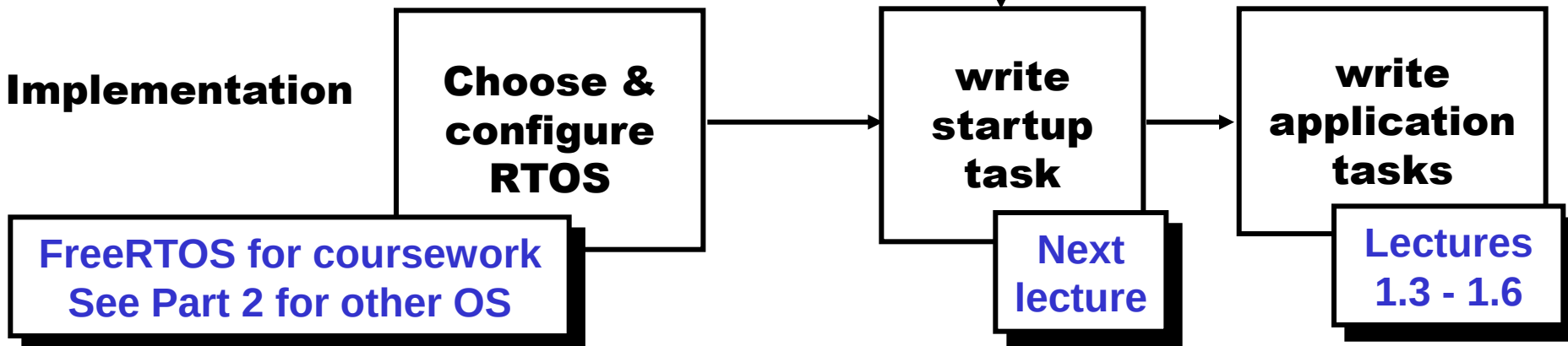
```
Task()
{
    [ initialise ]
    for (;;) {
        [ wait for event ]
        [ process event ]
        [ signal other tasks ]
    }
}
```

Real-Time System Design: Summary

Design



Implementation



Summary

- ◆ Real-time computation can be implemented via multitasking where tasks share CPU execution time with a relatively fine grain
- ◆ Tasks have STATE:
 - ❖ READY: Running, Ready (to run)
 - ❖ BLOCKED: Sleeping, Event-waiting
- ◆ In general more than one task may be READY – which task is actually run (scheduling) will be considered later
 - ❖ In RTOS most tasks spend most of time BLOCKED
 - ❖ For simple problems scheduling may be ignored
 - In many cases all possible schedules will work OK
- ◆ Real-time application design splits the computation into multiple tasks based on control of I/O devices
- ◆ In typical embedded system only one application runs
 - ❖ RTOS may be optimised for this application by eliminating unnecessary functions or reserved memory

Lecture 2: Task implementation in FreeRTOS

Now the earth was formless and empty. Darkness was on the surface of the deep.

Genesis 1:2

◆ How to use FreeRTOS

❖ Startup & task creation

➤ Startup task

➤ Stacks

◆ Timing functions

❖ Delaying the current task

◆ Controlling the scheduler

❖ Changing task priorities

❖ Suspending and restarting the multi-tasking

FreeRTOS

- ◆ FreeRTOS is a simple Gnu Public License (GPL) real-time OS written in C which is highly portable and supports:
 - ❖ Prioritised scheduling
 - ❖ Round-robin scheduling for equal timeslicing tasks with same priority
 - ❖ Dynamic creation & deletion of tasks
 - ❖ Semaphores, message queues, etc.
 - ❖ Dynamic task priority changing
 - Enables many more complex scheduling operations
- ◆ FreeRTOS **Kernel** implements FreeRTOS **API** (*application programming interface*)
- ◆ **Advantages**
 - ❖ Has been ported to many different microprocessors and compilers
 - ❖ Very simple and regular **Kernel** code based on lists and queues.
 - ❖ Very flexible and scalable – suitable for large systems or small
- ◆ **Disadvantages**
 - ❖ Not as efficient as some other RTOS (e.g. MicroC/OS-II)
 - ❖ Not very well documented
 - ❖ Minimal set of features implemented so far

FreeRTOS tasks

- ◆ In FreeRTOS a **task** is implemented using
 - ❖ A C function with an infinite loop
 - This can call other functions as necessary to implement the task
 - The function can have parameters passed when the task is created, so that multiple tasks can share the same code.
 - ❖ A (private) stack on which to store local variables
 - Created automatically when the task is created
 - ❖ A task control block (TCB) which contains all the task-specific information
 - Pointer to this is returned as a **task handle**
 - Can be used by subsequent task-control functions
- ◆ All FreeRTOS tasks are created dynamically.
- ◆ The FreeRTOS startup is initiated by a single startup function.
- ◆ Startup sequence:
 - ❖ Initialise hardware
 - ❖ Create at least one task
 - ❖ Start the RTOS scheduler
 - At this point control moves from the startup function to the RTOS scheduler which will run the created task(s).
 - ❖ Additional tasks can be created from running tasks
 - Often the number of tasks is known in advance and all are created from the startup function.

Task Creation

Constants in C use

#define

Note constant naming convention:

moduleName

Task handle optionally returned in a variable. Note & makes pointer to allow call-by-reference

Return value is either pdPASS or an error code (see projdefs.h)

```
#include "freertos.h" /*FreeRTOS type definitions etc */
```

```
/* constants */
```

```
#define mainLCD_STACK_SIZE 100 /* determines stack size */
```

```
→ #define mainLCD_TASK_PRIORITY 4 /*determines priority */
```

```
int main(void)  
{
```

```
    xTaskHandle task1; /* variable to store task handle */
```

```
    portBASE_TYPE x; /* variable to store return value from task */
```

```
    x = xTaskCreate(  
        vLcdDriveTask, /*task function*/  
        "LCD", /* name for debugging */  
        mainLCD_STACK_SIZE,  
        NULL, /*pointer to parameters, not used in this task so NULL*/  
        mainLCD_TASK_PRIORITY,  
        &task1 /* replace by NULL if task handle not required */  
    );
```

```
→ if (x!=pdPASS) hwLcdFatalError("Task creation failed");
```

```
#include "freertos.h" /*FreeRTOS type definitions etc */
```

Task Function definition

```
void vLcdDriveTask(void * pvParameters);
```

Function prototype needed if function definition is after its use

```
int main(void)
```

```
{
```

```
    [create the task]
```

```
}
```

```
void vLcdDriveTask(void *pvParameters)
```

```
{
```

```
    [define local variables]
```

Be careful to check storage of local variables – if big need to increase stack size

```
    pvParameters=pvParameters; /*stops compiler warning*/
```

```
    [initialise variables used in this task]
```

```
    for (;;) { /*loop forever*/
```

```
        if [characters waiting to be displayed] {
```

```
            [write characters to the LCD]
```

```
        }
```

```
        vTaskDelay(10); /* wait 10 clock ticks */
```

```
    }
```

```
}
```

Note typical task structure: is asleep most of time, uses CPU only when doing something.

DON'T put wait-for-data loops in tasks

RTOS Startup

```
#include "freertos.h" /*FreeRTOS type definitions etc */
#include "hwdefs.h" /*hardware-specific definitions */
```

```
#define mainKBD_TASK_PRIORITY      ( tskIDLE_PRIORITY + 2 )
#define mainLCD_TASK_PRIORITY      ( tskIDLE_PRIORITY + 4 )
#define mainADC_TASK_PRIORITY      ( tskIDLE_PRIORITY + 3 )
```

Good practice to define task priorities relative to **idle task** priority (which is 0)

```
int main( void )
{
```

```
    prvSetupHardware();
```

Setup app-specific hardware

```
    vlcdStartLcdTask( mainLCD_TASK_PRIORITY);
    vkbdStartKbdTask( mainKBD_TASK_PRIORITY);
    vadcStartADCTask( mainADC_TASK_PRIORITY);
```

Task creation details have been put in extra functions, including error check. Each task has its own C file, indicated by the prefix)

```
/* Now all the tasks have been started - start the scheduler.*/
```

```
vTaskStartScheduler();
```

After scheduler starts it will normally run forever – so a return is an error

```
/* Should never reach here! */
```

```
hwLcdFatalError("Kernel terminated!"); /*good practice to check this*/
return 0;
```

```
}
```

Before we start writing code, let's configure FreeRTOS. To do so, we need to edit FreeRTOSConfig.h file contents. There are lots of them, but most important are the following:

```
#define configUSE_PREEMPTION 1
#define configCPU_CLOCK_HZ ( ( unsignedlong ) 72000000 )
#define configTICK_RATE_HZ ( ( portTickType ) 1000 )
#define configMAX_PRIORITIES ( ( unsigned portBASE_TYPE ) 5 )
#define configMINIMAL_STACK_SIZE ( ( unsignedshort ) 120 )
#define configTOTAL_HEAP_SIZE ( ( size_t ) ( 18 * 1024 ) )
#define configMAX_TASK_NAME_LEN ( 16 )
#define configUSE_TRACE_FACILITY 1
#define configIDLE_SHOULD_YIELD 1
#define configUSE_MUTEXES 1
#define configUSE_COUNTING_SEMAPHORES 1
#define INCLUDE_vTaskPrioritySet 1
#define INCLUDE_vTaskDelayUntil 1
#define INCLUDE_vTaskDelay 1
```

Just a quick overview of these. We will use preemption, so we set it to 1, then we select the CPU clock rate, which is 72MHz, also we configure the tick timer, which means that the scheduler will run every 1ms.

Then we select a minimum stack size for a task and set a total heap size.

Our code is going to use task priorities, so we set vTaskPrioritySet to 1. Also, we are going to use vTaskDelay utilities that help with task timing. So we select them too.

There are a lot more settings you'll find in the Config file. Many of them are self-explanatory, but checking their meaning before using them as setting one or another may significantly increase in ram or CPU usage.

```

//STM32F103ZET6 FreeRTOS Test
#include "stm32f10x.h"
//#include "stm32f10x_it.h"
#include "mytasks.h"

//task priorities
#define mainLED_TASK_PRIORITY      ( tskIDLE_PRIORITY )
#define mainButton_TASK_PRIORITY  ( tskIDLE_PRIORITY )
#define mainButtonLEDs_TASK_PRIORITY  ( tskIDLE_PRIORITY + 1 )
#define mainLCD_TASK_PRIORITY     ( tskIDLE_PRIORITY )
#define mainUSART_TASK_PRIORITY   ( tskIDLE_PRIORITY )
#define mainLCD_TASK_STACK_SIZE configMINIMAL_STACK_SIZE+50
#define mainUSART_TASK_STACK_SIZE configMINIMAL_STACK_SIZE+50

int main(void)
{
    //init hardware
    LEDsInit();
    ButtonsInit();
    LCD_Init();
    Usart1Init();

    xTaskCreate( vLEDFlashTask, ( signed char * ) "LED", configMINIMAL_STACK_SIZE, NULL, mainLED_TASK_PRIORITY, NULL );
    xTaskCreate( vButtonCheckTask, ( signed char * ) "Button", configMINIMAL_STACK_SIZE, NULL, mainButton_TASK_PRIORITY, NULL );
    xTaskCreate( vButtonLEDsTask, ( signed char * ) "ButtonLED", configMINIMAL_STACK_SIZE, NULL, mainButtonLEDs_TASK_PRIORITY, NULL );

    xTaskCreate( vLCDTask, ( signed char * ) "LCD", mainLCD_TASK_STACK_SIZE, NULL, mainLCD_TASK_PRIORITY, NULL );
    xTaskCreate( vUSARTTask, ( signed char * ) "USART", mainUSART_TASK_STACK_SIZE, NULL, mainUSART_TASK_PRIORITY, NULL );

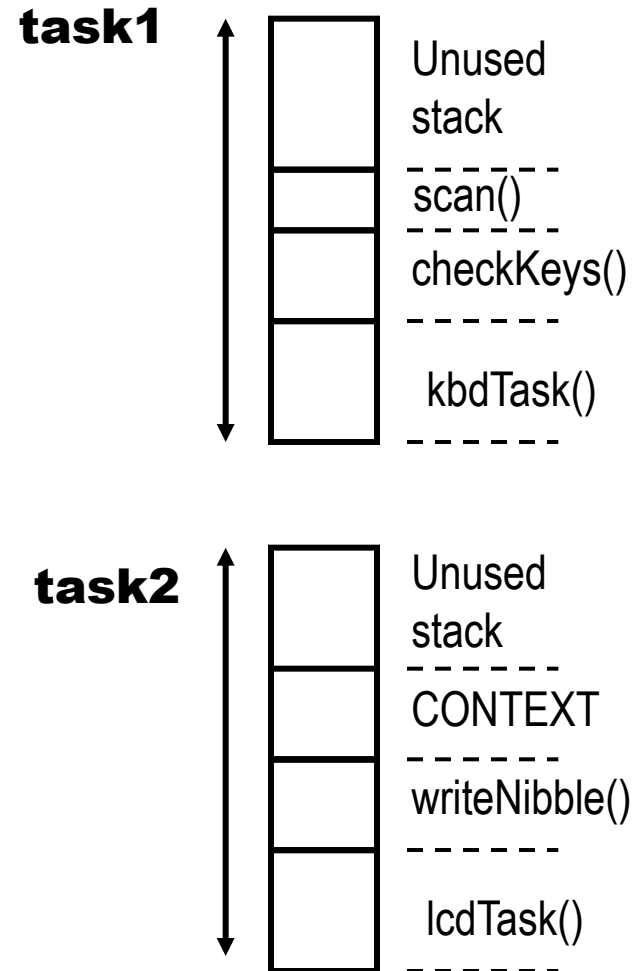
    //start scheduler
    vTaskStartScheduler();                //After this, the highest priority task will be implemented

    //you should never get here
    while(1)
    {
    }
}

```


Stack use in RTOS

- ◆ A running computer program requires stack
 - ❖ Store local variables, return addresses
 - ❖ Size depends on program, can be large
 - ❖ For non-recursive programs can compute size. Normally just make an over-estimate.
- ◆ Every task is an independently running program and **needs its own stack**.
 - ❖ In practice, it is too difficult to dynamically change space allocated to stacks.
 - ❖ Fix stack-size on task creation
- ◆ When a task is not running its **context** (all the CPU registers including PC & status register) must be saved on its own stack.
 - ❖ Must add CPU context size to required stack
- ◆ Stack overflow is common cause of RTOS bugs



Stack sizing for ARM7/Keil port

- ◆ FreeRTOS stack sizes are measured in number of stack items
 - ❖ ARM7 is 32 bit so each item is 4 bytes
- ◆ Minimal stack by default is 100 items
- ◆ Always allow 17 items on stack for task context (17 registers)
 - ❖ Leaves 83 items
- ◆ Stack use is determined by number of **nested function calls**
 - ❖ Typically between 2 and 8 items per function call
 - ❖ Difficult to calculate stack size precisely
- ◆ Consequences of stack overflow are disastrous!
 - ❖ Determine minimum size by trial & error, then add safety margin
 - ❖ Use **instrumentation** to check high water mark of stack actually used.
 - FreeRTOS provides diagnostic functions – see coursework

The optimizing of the stack size for the FreeRTOS task

As you can see, we have created 5 tasks. Each of them has a priority level and stack size. The hardest part is defining proper stack size – if it's too small, it may crash your program; if it's too large, then we are wasting limited resources of the microcontroller. To detect stack overflow, you can use a particular function called

```
void vApplicationStackOverflowHook( xTaskHandle *pxTask, signed portCHAR *pcTaskName )
```

In this function, you can set up any indicators such as LED flash whenever stack overflow occurs. This way, you can leverage the stack size for the task and start over.

FreeRTOS API – Task Control in Detail

◆ Task create/delete

- ❖ xTaskCreate
- ❖ xTaskDelete

◆ Task sleeping

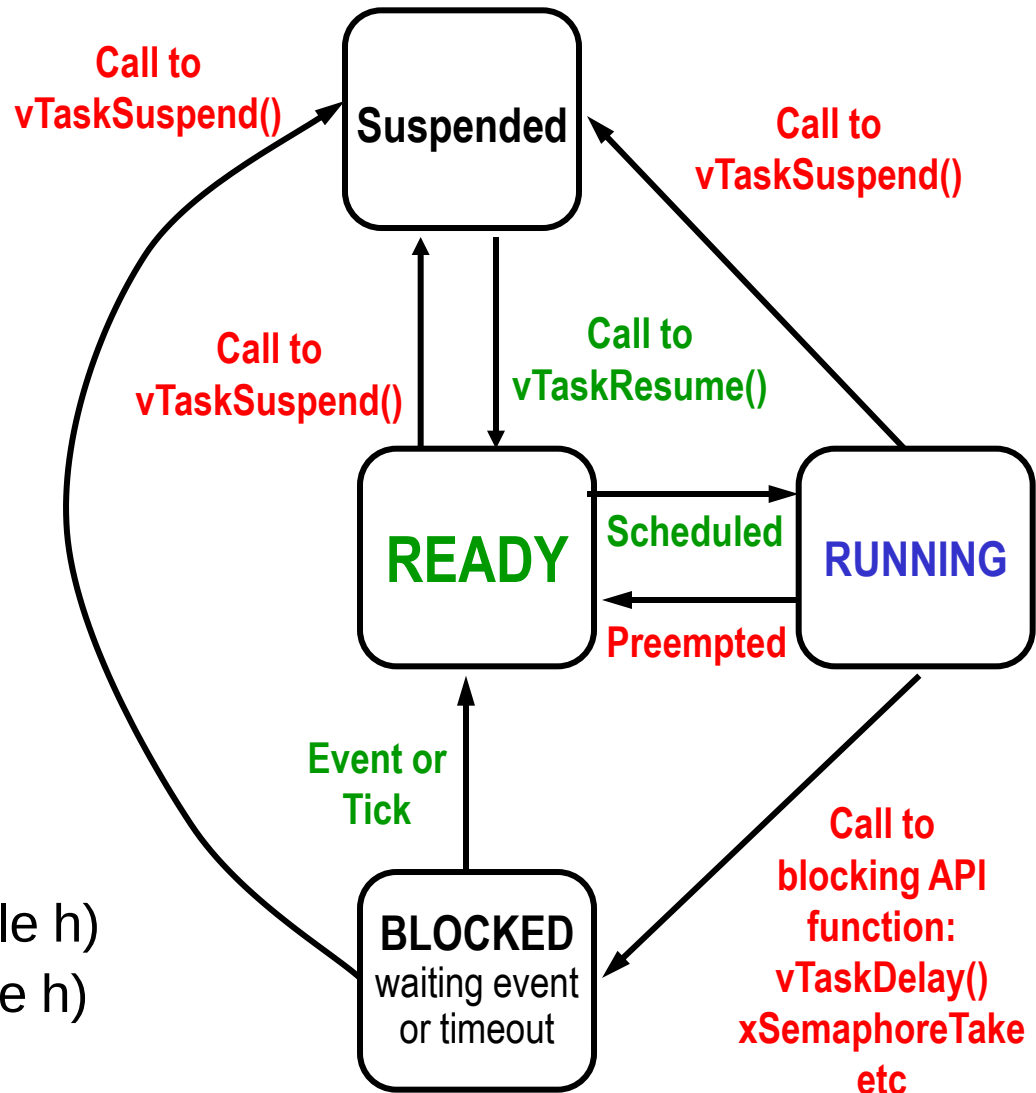
- ❖ vTaskDelay()
- ❖ vTaskDelayUntil()

◆ Kernel control

- ❖ vTaskAllSuspend()
- ❖ vTaskAllResume()
- ❖ portENTER_CRITICAL()
- ❖ portEXIT_CRITICAL()

◆ Task suspension

- ❖ xTaskSuspend(xTaskHandle h)
- ❖ xTaskResume(xTaskHandle h)



Creation

- ◆ Task creation uses RAM
 - ❖ Task Control Block
 - ❖ Task Stack
 - ❖ New task will be executed immediately on creation if high enough priority
- ◆ Task Deletion frees RAM
 - ❖ Programmer must ensure that all resources associated with task – semaphores etc, are also recovered
 - ❖ Programmer must ensure no tasks are left blocked on objects deleted
- ◆ Simple RTOS applications use static tasks, created at startup and persisting forever
- ◆ Complex applications may spawn and delete tasks dynamically

Writing a FreeRTOS task routine

If you are familiar with the RTOS concept, you know that the program written for FreeRTOS is organized as a set of independent tasks.

Each task normally is not directly related to other tasks and run within its context. Practically speaking, the task is a function with its own stack and runs a separate small program.

When multiple tasks are created, a scheduler switches between tasks according to assigned priorities. **The task itself is a function with an endless loop, which never returns from it:**

```
void vATaskFunction( void *pvParameters )
{
    for( ;; )
    {
        -- Task application code here. --
    }
}
```

Task delay - vTaskDelay

- ◆ Task **sleeping** is the most basic method for a task to temporarily give way to lower-priority tasks
 - ❖ Time is measured as number of system **clock ticks**, where the clock tick is a fixed frequency signal derived from a timer.
 - ❖ Task becomes BLOCKED and stays in that state for a given number of system clock ticks, after which it is made READY.
- ◆ vTaskDelay(interval)
 - ❖ Delays until *interval* ticks have passed
 - ❖ Delay time t is $(\text{interval}-1)*\text{TickPeriod} < t < \text{interval}*\text{TickPeriod}$

```
Task()
{
    for (;;) {
        [ do something ]
        vTaskDelay(2); /* wait 2 clock ticks */
    }
}
```

Task delay in detail

Difference Bet. `vTaskDelay` & `vTaskDelayUntil`

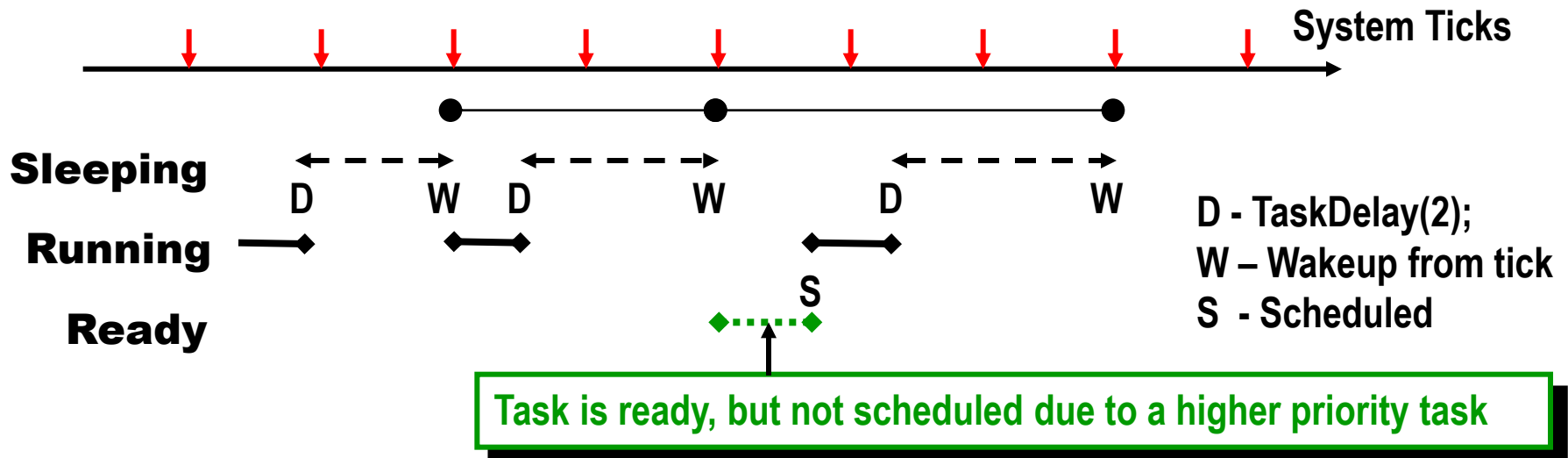
In `vTaskDelay` you say how long after calling `vTaskDelay` you want to be woken.

In `vTaskDelayUntil` you say the time at which you want to be woken.

The parameter in `vTaskDelay` is the delay period in the number of ticks from now.

The parameter in `vTaskDelayUntil` is the absolute time in ticks at which you want to be woken calculated as an increment from the time you were last woken.

- ◆ Repeated calls to `vTaskDelay(n)` should produce delays of $n \times \text{TickPeriod}$ if running time of task is small
 - ❖ Generally this can't be guaranteed due to preemption by other tasks
 - ❖ If exact timing is required, solutions are:
 - Increase task priority
 - Use the more complex `vTaskDelayUntil()`



Task Delay – xTaskDelayUntil()

- ◆ Purpose – delay execution until a specific time to ensure accurate periodic execution.

```
/* Perform an action every 2 ticks. */  
void vTaskFunction( void * pvParameters )  
{  
    portTickType xLastWakeTime;  
    const portTickType time = 2;  
  
    /* Initialise the xLastWakeTime variable with the current time. */  
    xLastWakeTime = xTaskGetTickCount();  
  
    for( ;; ) {  
        /* Wait for the next cycle. */  
        vTaskDelayUntil( &xLastWakeTime, time );  
  
        /* Perform action here. */  
    }  
}
```

xLastWakeTime is automatically updated by each call.

Let's write a simple LED flasher task. This is a basic routine that flashes LED every 1s. This is how the tasks look like:

```
void vLEDFlashTask( void *pvParameters )
{
    portTickType xLastWakeTime;
    const portTickType xFrequency = 1000;
    xLastWakeTime=xTaskGetTickCount();
    for( ;; )
    {
        LEDToggle(5);
        vTaskDelayUntil(&xLastWakeTime,xFrequency);
    }
}
```

To set the timing, we are using the vTaskDelayUntil function. FreeRTOS counts ticks every time scheduler is called (every 1ms by default). By setting the frequency value to 1000, we are getting a 1s delay.

Read an Analog Input from a device and subsequently drive a motor.

Also want to send an SMS using a GSM Module every 20 seconds.

analogMotorTask()

```
{  
  // This occurs only once  
  // Initialization of Analog and Motor here  
  initAnalogPin();  
  initMotor();  
  // This occurs once every second  
  while(1)  
  {  
    // Read Analog Input  
    analogRead();  
    // Signal the Motor  
    signalMotor();  
    // Block analogMotorTask function for 1 second  
    vTaskDelay(1000);  
  }  
}
```

sendSMS()

```
{  
  // This occurs only once  
  // Initialization of GSM here  
  initGSM();  
  // This will occur every 20 seconds  
  while(1)  
  {  
    // Send SMS  
    sendingSMS();  
    // Block sendSMS function for 20 seconds  
    vTaskDelay(20000);  
  }  
}
```

Use of vTaskDelete()

```
/* Scheduler include files. */
#include "FreeRTOSConfig.h";#include "FreeRTOS.h";#include "task.h";#include "croutine.h";#include "uart.h"    // Explore Embedded UART library

xTaskHandle TaskHandle_1;    xTaskHandle TaskHandle_2;    xTaskHandle TaskHandle_3;

/* Local Tasks declaration */
static void MyTask1(void* pvParameters);static void MyTask2(void* pvParameters)
;static void MyTask3(void* pvParameters);static void MyIdleTask(void* pvParameters);

#define LED_IdleTask 0x01u
#define LED_Task1    0x02u
#define LED_Task2    0x04u
#define LED_Task3    0x08u
#define LED_Task4    0x10u

#define LED_PORT LPC_GPIO2->FIOPIN
int main(void)
{

    SystemInit();                /* Initialize the controller */
    UART_Init(38400);            /* Initialize the Uart module */
    LPC_GPIO2->FIODIR = 0xffffffffu;

    /* Create the three tasks with priorities 1,2,3. Only tasks will be created.
    * Tasks will be executed once the scheduler is started.
    * An idle task is also created, which will be run when there are no tasks in RUN state */
    xTaskCreate( MyTask1, ( signed char * )"Task1", configMINIMAL_STACK_SIZE, NULL, 1, &TaskHandle_1 );
    xTaskCreate( MyTask2, ( signed char * )"Task2", configMINIMAL_STACK_SIZE, NULL, 2, &TaskHandle_2 );
    xTaskCreate( MyTask3, ( signed char * )"Task3", configMINIMAL_STACK_SIZE, NULL, 3, &TaskHandle_3 );
    xTaskCreate( MyIdleTask, ( signed char * )"IdleTask", configMINIMAL_STACK_SIZE, NULL, tskIDLE_PRIORITY, NULL );

    UART_Printf("\n\rIn the main");

    vTaskStartScheduler();        /* Start the scheduler */

    while(1);

}
```

```
static void MyTask1(void* pvParameters)
```

```
{
    LED_PORT = LED_Task1;    /* Led to indicate the execution of Task1*/
    UART_Printf("\n\rTask1, waiting for some time");
    vTaskDelay(200);

    LED_PORT = LED_Task1;    /* Led to indicate the execution of Task1*/
    UART_Printf("\n\rBack in Task1");

    vTaskDelete(TaskHandle_1);
}
```

```
static void MyTask2(void* pvParameters)
```

```
{
    LED_PORT = LED_Task2;    /* Led to indicate the execution of Task2*/
    UART_Printf("\n\rTask2, waiting for some time");
    vTaskDelay(200);

    LED_PORT = LED_Task2;    /* Led to indicate the execution of Task2*/
    UART_Printf("\n\rBack in Task2");

    vTaskDelete(TaskHandle_2);
}
```

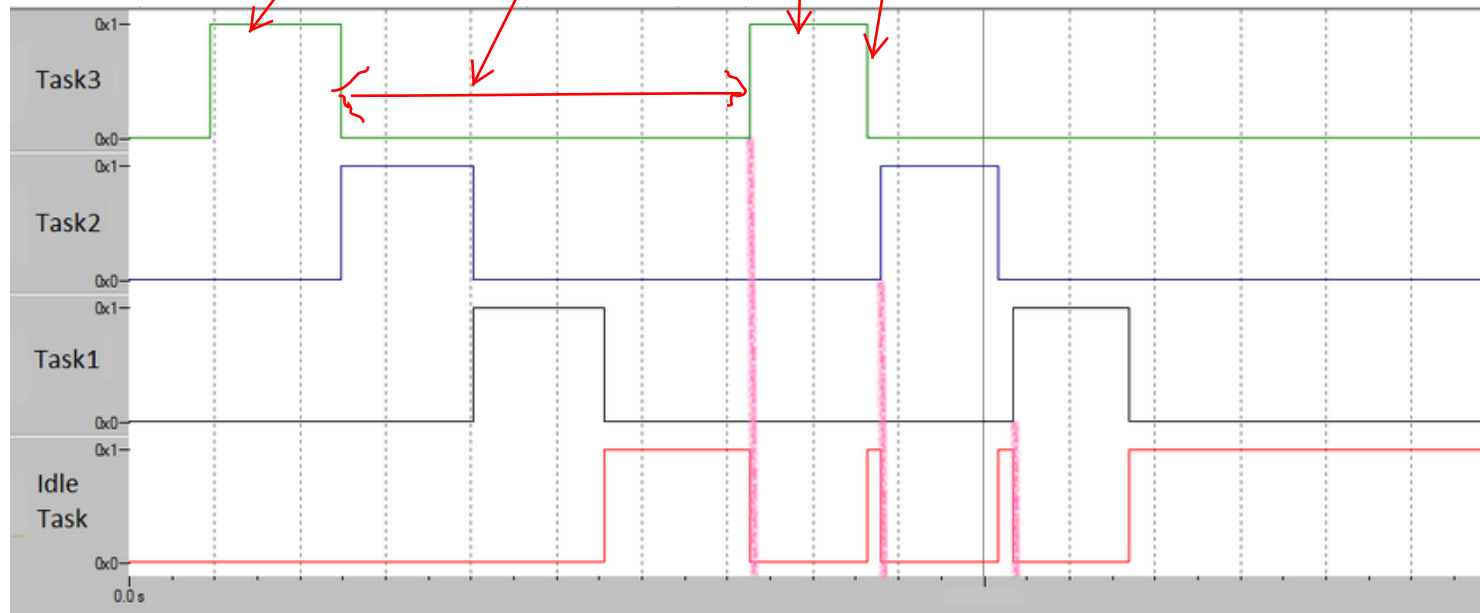
```
static void MyTask3(void* pvParameters)
```

```
{
    LED_PORT = LED_Task3;    /* Led to indicate the execution of Task3*/
    UART_Printf("\n\rTask3, waiting for some time");
    vTaskDelay(200);

    LED_PORT = LED_Task3;    /* Led to indicate the execution of Task3*/
    UART_Printf("\n\rBack in Task3");
    vTaskDelete(TaskHandle_3);
}
```

```
static void MyIdleTask(void* pvParameters)
```

```
{
    while(1)
    {
        LED_PORT = LED_IdleTask; /* Led to indicate the execution of Idle Task*/
        UART_Printf("\n\rIn idle state");
    }
}
```



Task switching depending on the priorities

```
xTaskHandle TaskHandle_1; xTaskHandle TaskHandle_2; xTaskHandle TaskHandle_3;  
xTaskHandle TaskHandle_4; xTaskHandle TaskHandle_5;
```

```
/* Local Tasks declaration */
```

```
static void MyTask1(void* pvParameters); static void MyTask2(void* pvParameters); static void MyTask3(void* pvParameters);  
static void MyTask4(void* pvParameters); static void MyTask5(void* pvParameters); static void MyIdleTask(void* pvParameters);
```

```
#define LED_IdleTask 0x01u
```

```
#define LED_Task1 0x02u
```

```
#define LED_Task2 0x04u
```

```
#define LED_Task3 0x08u
```

```
#define LED_Task4 0x10u
```

```
#define LED_Task5 0x20u
```

```
#define LED_PORT LPC_GPIO2->FIOPIN
```

```
int main(void)
```

```
{  
    SystemInit();           /* Initialize the controller */  
    UART_Init(38400);       /* Initialize the Uart module */  
    LPC_GPIO2->FIODIR = 0xffffffffu;
```

```
    /* Create the 2 tasks with priorities 1 and 3.*/
```

```
    xTaskCreate( MyTask1, ( signed char * )"Task1", configMINIMAL_STACK_SIZE, NULL, 1, &TaskHandle_1);
```

```
    xTaskCreate( MyTask3, ( signed char * )"Task3", configMINIMAL_STACK_SIZE, NULL, 3, &TaskHandle_3 );
```

```
    xTaskCreate( MyIdleTask, ( signed char * )"IdleTask", configMINIMAL_STACK_SIZE, NULL, tskIDLE_PRIORITY, NULL );
```

```
    UART_Printf("\n\rIn main function, invoking scheduler");
```

```
    vTaskStartScheduler();           /* Start the scheduler */
```

```
    while(1);
```

```
}
```

```

static void MyTask1(void* pvParameters)
{
    LED_PORT = LED_Task1;    /* Led to indicate the execution of Task1*/
    UART_Printf("\n\rIn Task1");
    vTaskDelete(TaskHandle_1);
}

static void MyTask2(void* pvParameters)
{
    LED_PORT = LED_Task2;    /* Led to indicate the execution of Task2*/
    UART_Printf("\n\rIn Task2 ");
    vTaskDelete(TaskHandle_2);
}

static void MyTask3(void* pvParameters)
{
    LED_PORT = LED_Task3;    /* Led to indicate the execution of Task3*/
    UART_Printf("\n\rTask3, creating new tasks 2");

    /* Create two new tasks 2, 4 */
    xTaskCreate( MyTask2, ( signed char * )"Task2", configMINIMAL_STACK_SIZE, NULL, 2, &TaskHandle_2);
    UART_Printf("\n\rTask3, creating new tasks 4");

    xTaskCreate( MyTask4, ( signed char * )"Task4", configMINIMAL_STACK_SIZE, NULL, 4, &TaskHandle_4);

    LED_PORT = LED_Task3;    /* Led to indicate the execution of Task3*/
    UART_Printf("\n\rBack in Task3, Creating Task5");

    xTaskCreate( MyTask5, ( signed char * )"Task5", configMINIMAL_STACK_SIZE, NULL, 5, &TaskHandle_5);

    LED_PORT = LED_Task3;    /* Led to indicate the execution of Task3*/
    UART_Printf("\n\rBack in Task3, Exiting task3");

    vTaskDelete(TaskHandle_3);
}

```

```
static void MyTask4(void* pvParameters)
```

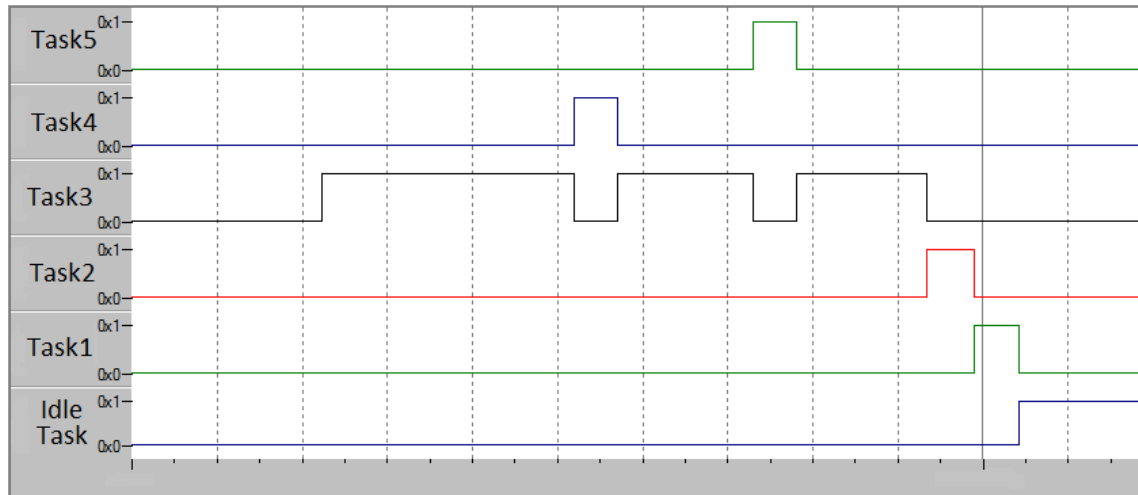
```
{  
    LED_PORT = LED_Task4;    /* Led to indicate the execution of Task4*/  
    UART_Printf("\n\rIn Task4");  
    vTaskDelete(TaskHandle_4);  
}
```

```
static void MyTask5(void* pvParameters)
```

```
{  
    LED_PORT = LED_Task5;    /* Led to indicate the execution of Task4*/  
    UART_Printf("\n\rIn Task5");  
    vTaskDelete(TaskHandle_5);  
}
```

```
static void MyIdleTask(void* pvParameters)
```

```
{  
    while(1)  
    {  
        LED_PORT = LED_IdleTask;    /* Led to indicate the execution of Idle Task*/  
        UART_Printf("\n\rIn idle state");  
    }  
}
```



Task switching depending on the priorities and the effect of task delay

```
xTaskHandle TaskHandle_1; xTaskHandle TaskHandle_2; xTaskHandle TaskHandle_3;  
xTaskHandle TaskHandle_4; xTaskHandle TaskHandle_5;
```

```
/* Local Tasks declaration */
```

```
static void MyTask1(void* pvParameters); static void MyTask2(void* pvParameters); static void MyTask3(void* pvParameters);  
static void MyTask4(void* pvParameters); static void MyTask5(void* pvParameters); static void MyIdleTask(void* pvParameters);
```

```
#define LED_IdleTask 0x01u  
#define LED_Task1 0x02u  
#define LED_Task2 0x04u  
#define LED_Task3 0x08u  
#define LED_Task4 0x10u  
#define LED_Task5 0x20u
```

```
#define LED_PORT LPC_GPIO2->FIOPIN
```

```
int main(void)
```

```
{  
    SystemInit();          /* Initialize the controller */  
    UART_Init(38400);      /* Initialize the Uart module */  
    LPC_GPIO2->FIODIR = 0xffffffffu;
```

```
    /* Create the 2 tasks with priorities 1 and 3.*/
```

```
    xTaskCreate( MyTask1, ( signed char * )"Task1", configMINIMAL_STACK_SIZE, NULL, 1, &TaskHandle_1);  
    xTaskCreate( MyTask3, ( signed char * )"Task3", configMINIMAL_STACK_SIZE, NULL, 3, &TaskHandle_3 );
```

```
    xTaskCreate( MyIdleTask, ( signed char * )"IdleTask", configMINIMAL_STACK_SIZE, NULL, tsIDLE_PRIORITY, NULL );
```

```
    UART_Printf("\n\rIn main function, invoking scheduler");
```

```
    vTaskStartScheduler();          /* Start the scheduler */
```

```
    while(1);
```

```
}
```

```

static void MyTask1(void* pvParameters)
{
    LED_PORT = LED_Task1;    /* Led to indicate the execution of Task1*/
    UART_Printf("\n\rIn Task1");
    vTaskDelete(TaskHandle_1);
}

static void MyTask2(void* pvParameters)
{
    LED_PORT = LED_Task2;    /* Led to indicate the execution of Task2*/
    UART_Printf("\n\rIn Task2, waiting for some time");
    vTaskDelay(200);
    LED_PORT = LED_Task2;    /* Led to indicate the execution of Task2*/
    UART_Printf("\n\rBack in Task2");
    vTaskDelete(TaskHandle_2);
}

static void MyTask3(void* pvParameters)           //Run first
{
    LED_PORT = LED_Task3;    /* Led to indicate the execution of Task3*/
    UART_Printf("\n\rTask3, creating new tasks 2 and 4");
    /* Create two new tasks 2, 4 */
    xTaskCreate( MyTask2, ( signed char * )"Task2", configMINIMAL_STACK_SIZE, NULL, 2, &TaskHandle_2);
    xTaskCreate( MyTask4, ( signed char * )"Task4", configMINIMAL_STACK_SIZE, NULL, 4, &TaskHandle_4); //Run 2nd

    LED_PORT = LED_Task3;    /* Led to indicate the execution of Task3*/
    UART_Printf("\n\rBack in Task3, Creating Task5");

    xTaskCreate( MyTask5, ( signed char * )"Task5", configMINIMAL_STACK_SIZE, NULL, 5, &TaskHandle_5);

    LED_PORT = LED_Task3;    /* Led to indicate the execution of Task3*/
    UART_Printf("\n\rBack in Task3, Exiting task3");

    vTaskDelete(TaskHandle_3);
}

```

```

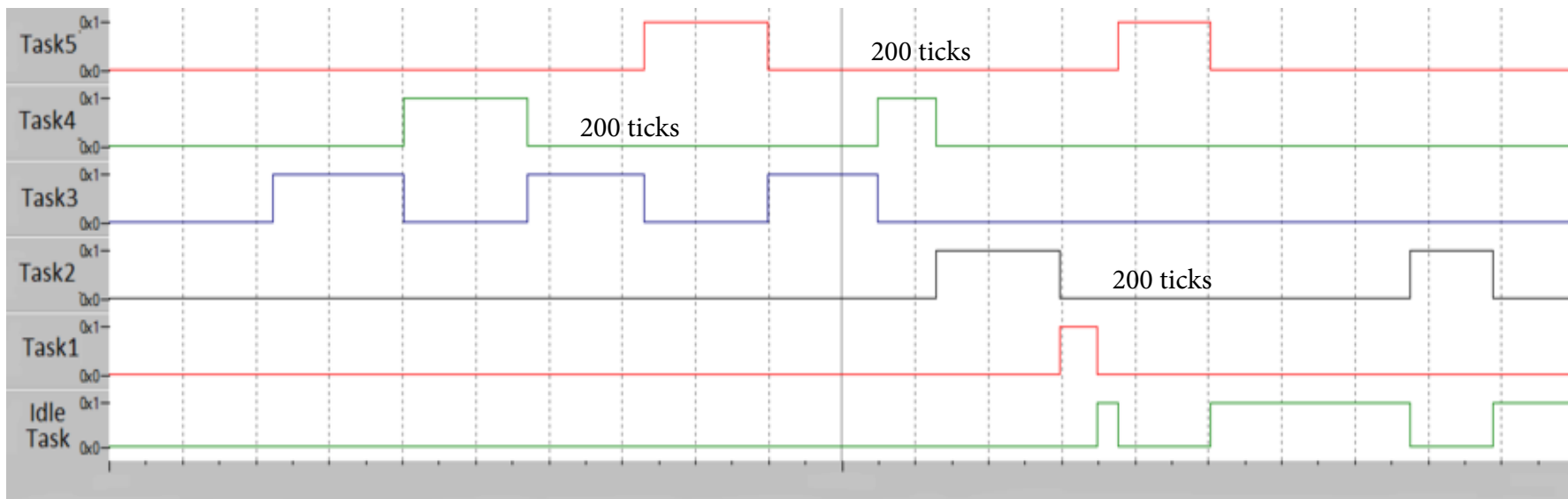
static void MyTask4(void* pvParameters)
{
    LED_PORT = LED_Task4;    /* Led to indicate the execution of Task4*/
    UART_Printf("\n\rIn Task4, waiting for some time");
    vTaskDelay(200);
    LED_PORT = LED_Task4;    /* Led to indicate the execution of Task4*/
    UART_Printf("\n\rBack in Task4");
    vTaskDelete(TaskHandle_4);
}

static void MyTask5(void* pvParameters)
{
    LED_PORT = LED_Task5;    /* Led to indicate the execution of Task4*/
    UART_Printf("\n\rIn Task5, waiting for some time");
    vTaskDelay(200);
    LED_PORT = LED_Task5;    /* Led to indicate the execution of Task4*/
    UART_Printf("\n\rBack in Task5");

    vTaskDelete(TaskHandle_5);
}

static void MyIdleTask(void* pvParameters)
{
    while(1)
    {
        LED_PORT = LED_IdleTask;    /* Led to indicate the execution of Idle Task*/
        UART_Printf("\n\rIn idle state");
    }
}

```



Kernel Control

- ◆ FreeRTOS has two ways for a task to ensure temporarily private access to data or hardware resources.
 - ❖ A section of code requiring this is called a **critical section**
- ◆ **1. Switch off all interrupts (interrupt-lock)**
 - ❖ Advantages
 - Nothing can preempt the current task
 - No ISR will run
 - Fast to implement
 - ❖ Disadvantages
 - Long times will impact ALL interrupt times
 - Therefore only useful for short critical sections
- ◆ **2. Disable scheduling (preemption-lock)**
 - ❖ Advantages
 - Interrupts will still be processed
 - Tasks not allowed to preempt current task
 - ❖ Disadvantages
 - Slower to implement
 - Can't be used to protect data from interrupt access

Critical Sections

A diagram showing a code block for a critical section. It starts with `portENTER_CRITICAL();`, followed by a dashed box containing the text *[Private access to data]*, and ends with `portEXIT_CRITICAL();`. A large curly brace on the left groups the entire block.

```
portENTER_CRITICAL();  
[Private access to data]  
portEXIT_CRITICAL();
```

A diagram showing a code block for a critical section. It starts with `vTaskSuspendAllTasks();`, followed by a dashed box containing the text *[private access to data shared with other task(s)]*, and ends with `vTaskResumeAllTasks();`. A large curly brace on the left groups the entire block.

```
vTaskSuspendAllTasks();  
[private access to data shared with other task(s)]  
vTaskResumeAllTasks();
```

Nesting critical sections

- ◆ The use of functions to enter & exit critical sections raises one subtle but important issue. What happens when critical sections are nested?
- ◆ The desired behaviour is that the nested **portEXIT_CRITICAL()** or **TaskResumeAll()** commands do NOT exit the critical section. This must happen only on the outermost exit.
- ◆ This allows functions define critical sections to call other functions which define critical sections internally.
 - ❖ For example RTOS functions may be called which themselves define critical sections.
 - ❖ Note that blocking RTOS functions, like `vTaskDelay()`, must never be called inside critical sections.
- ◆ FreeRTOS critical section commands are all nestable in this way – but check RTOS documentation before assuming this for other RTOS

```
f1()
{
    portENTER_CRITICAL();
    f2();
    /* want this still to be critical */
    portEXIT_CRITICAL();
    /* no longer critical after
       outermost exit */
}

f2()
{
    portENTER_CRITICAL();
    /* nested critical section */
    portEXIT_CRITICAL();
}
```

Task priorities & scheduling

- ◆ The scheduler uses **task priorities** to determine which task to run whenever multi-tasking is enabled (ie not in a critical section)
 - ❖ The scheduler will ensure that the currently running task is always the highest priority READY task.
 - ❖ Making ready a high priority task will therefore cause **preemption** of the current task.
 - ❖ If there are multiple READY tasks of equal highest priority the scheduler will time-slice between them
 - See slide 1.123
- ◆ Priorities can be inspected and changed dynamically with API functions as below.
 - ❖ A task can change its own priority without storing its task handle, by using NULL as first parameter.

```
{
xTaskHandle *xh1;      /* variable to store task handle of newly created task*/
int uxCurrPriority = uxTaskPriorityGet(NULL); /*store priority of this task*/
                        /*create new task, handle xh1, priority 1 more than this task*/
xCreateTask(vT1Code, "T1", 100, NULL, uxCurrPriority+1, &xh1 );
xTaskDelay(2);          /*wait between 1 & 2 clock ticks*/
vTaskPrioritySet( xh1, uxCurrPriority-1 ); /*decrease priority of new task*/
}
```

Determining task priorities

- ◆ In real-time system assume normal state for all tasks is blocked!
 - ❖ Allows other tasks to execute
 - ❖ A task that runs continuously will prevent lower-priority tasks from executing
- ◆ When a task executes it will delay execution of all lower-priority tasks for as long as it is running.
 - ❖ Determine task priority according to:
 - a) For how long (max) they execute before blocking. Long execution time => low priority
 - b) How much delay can they tolerate between being ready-to-run and running. Long delay => lower priority
- ◆ Often task priority does not matter very much. When it does we will learn how to quantify delays later.

Lecture 2: Summary

- ◆ **Task create/delete**
 - ❖ xTaskCreate()
 - ❖ xTaskDelete()
- ◆ **Task sleeping**
 - ❖ vTaskDelay()
 - ❖ vTaskDelayUntil()
- ◆ **Kernel control**
 - ❖ vTaskSuspendAll()
 - ❖ xTaskResumeAll()
 - ❖ portENTER_CRITICAL()
 - ❖ portEXIT_CRITICAL()
 - ❖ vTaskPrioritySet()
 - ❖ uxTaskPriorityGet()
- ◆ **Task suspension**
 - ❖ vTaskSuspend()
 - ❖ vTaskResume()
- ◆ **Kernel** is all the RTOS code which implements the **API**
- ◆ A single startup function creates tasks and then calls the FreeRTOS **scheduler** to initiate **multi-tasking**.
- ◆ Tasks have states: **RUNNING, READY, BLOCKED, SUSPENDED**. (**SUSPENDED** is special case of **BLOCKED** requiring explicit **Resume()** to restart)
- ◆ The FreeRTOS scheduler uses task **priorities**, with equal priority scheduling time-sliced between tasks. Tasks can be **preempted** when higher priority tasks become **READY**.
- ◆ API functions exist to create, delete, delay tasks. Tasks are referenced via **task handles**.
- ◆ **Critical sections** can be made by temporarily stopping interrupts, or disabling scheduling.
- ◆ The API contains delay functions which make tasks sleep (block) for given number of **clock ticks**, or till a specified clock tick.
- ◆ Tasks can dynamically control priorities of themselves, or any other task where they know the **task handle**.

Review Questions 2

- ◆ A serial port driver task runs code of the form:

```
for (;;) { /*loop forever*/
    while ( [ port hardware is not ready for next character ] ) ; /*busy-wait loop till
                                                    port is free*/
    [ write next character to port ]
}
```

- 2.1 Rewrite this so that the busy-wait loop is removed and replaced by a task delay function.
- 2.2 Explain why the original code would be defective in a multitasking system if the task is given a high priority relative to other system tasks (assume that a continuous stream of output characters is available).
- 2.3 Contrast the performance (speed of character output) of the original code and the new code, assuming no other tasks in the system? What is the maximum speed the new code can attain in characters per ClockTickPeriod?
- 2.3 Write a complete FreeRTOS system with a task which will update a 24 hour clock display every second, assuming that the display can be written to by a function **vWriteToDisplay(int hours, int minutes, int seconds)**; which you are given. vWriteToDisplay runs without blocking and takes 100us to complete. Discuss what priority should your task have relative to other system tasks. What percentage of total CPU time will it use?

Lecture 3: The Shared Data Problem

Collecting data is only the first step toward wisdom, but sharing data is the first step toward community.

Henry Louis Gates Jr.

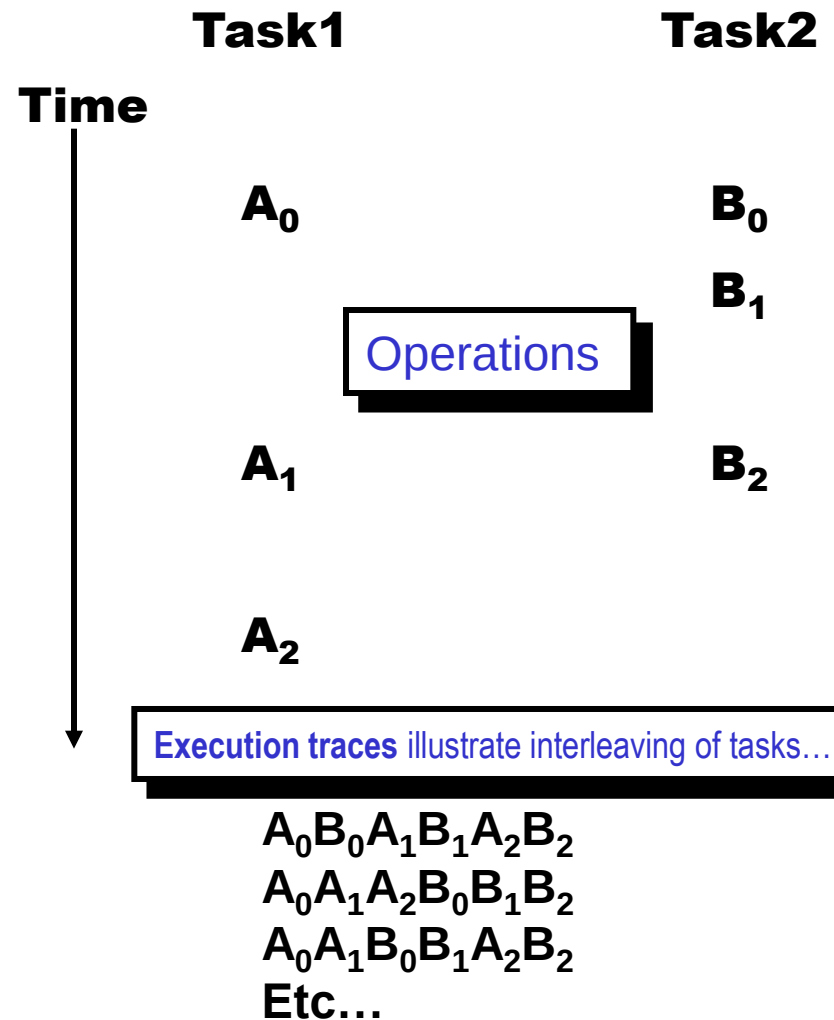
- ◆ This lecture considers why **critical sections** of code are required to ensure correctness when accessing shared data
- ◆ Shared data problems are at the heart of why concurrent systems are more difficult to write than sequential programs
- ◆ This will be in part revision for ISE students who have covered concurrent systems
 - ❖ Material in this lecture is not specific to real-time systems.

How to think about concurrent programming

- ◆ **Lecture 1 introduced scheduling in real-time operating systems.**
 - ❖ This is an important topic which relates to the **timing** of real-time systems
 - ❖ We will deal with this in detail later
- ◆ **In this lecture we will on purpose make minimal assumptions about the way that tasks are scheduled**
 - ❖ Separate **functionality** & timing
 - ❖ Ensure functionality is correct for any scheduling strategy
 - ❖ Later we will consider how scheduling can be adjusted to ensure time deadlines are always met.
- ◆ **This, in practice, is the only feasible way to write robust concurrent code.**
 - ❖ **RESIST TEMPTATION TO WRITE CODE WHERE SCHEDULING CAN DETERMINE CORRECTNESS**

Task execution interleaving

- ◆ In general tasks may be preempted at any time and another task run.
- ◆ We will not assume anything about the scheduler
 - ❖ Must ensure correctness for all possible **execution traces**
- ◆ Assume two distinct tasks may be arbitrarily **interleaved**.
 - ❖ Technically we impose only a **progression requirement** that all tasks that are READY will **eventually** make progress
- ◆ Interleaving can create problems. For example even though B_0 & B_1 happen immediately after each other in Task 2 they may be separated in execution time by code from Task 1.
- ◆ Where data is shared between Task 1 & Task 2 this can cause errors
 - ❖ Multiple execution traces $\rightarrow$ more chance of bugs



Error Counting

- ◆ Problem

- ❖ Implement function **RecordError()** which can be called from multiple tasks and counts the total number of times it is called.

- ◆ Single-threaded solution

```
static int prvErrorCount = 0; /* initialise to 0 */
vRecordError()
{ /* this function assumes overflow will never happen */
    int x;
    x = prvErrorCount;
    prvErrorCount = x + 1;
    return x;
}
```

- ◆ ErrorCount variable is **shared data**

RecordError() is not safe when multi-tasking

- ◆ Note here that the two local **x** variables are separate
 - ❖ Each task has a separate stack.
- ◆ However the relative timing shown here means that two calls result in ErrorCount only increasing by 1
 - ❖ Error!

```
vRecordError()
```

```
{
```

```
    int x;
```

```
    x = ErrorCount;
```

```
    [ Task 2 preempts  
      task 1 ]
```

```
    ErrorCount := x + 1;
```

```
}
```

Task 1

```
vRecordError()
```

```
{
```

```
    int x;
```

```
    x = ErrorCount;
```

```
    ErrorCount := x + 1;
```

```
}
```

```
    [ Task 2 blocks ]
```

Task 2

Curse of the infrequent error

Occasionally you hear someone say:

"Well my program works most of the time!".

This always sends shivers down my spine. Infrequent bugs which cannot be reproduced are the most difficult to fix.

- ◆ The "shared data problem" is a classic example of an infrequent bug. It may be very unlikely that a task will be preempted at precisely the wrong time and so cause this bug. It may indeed never happen in a prototype, only to emerge when a CPU of a slightly different speed is used.
 - ❖ If it CAN happen, it is an error.
 - ❖ Because it will be difficult to debug when the program is running the only realistic solution is to be very, very careful to ensure bugs of this kind are designed out from the start.
- ◆ This is the main reason why concurrent systems programming requires much more care than conventional programming

Atomic operations

- ◆ The solution to this problem is to recognise that the read and write operations on `prvErrorCount` must be atomic (executed without interruption) for correct operation. This can be enforced in an RTOS by making the code a **critical section**
- ◆ Kernel will not allow task switch during critical section

```
static int prvErrorCount = 0; /* initialise to 0 */
```

**Safe
called
from
multiple
tasks**

```
void vRecordError(void)
{ /* this function assumes overflow will never happen */
    int x;
    portENTER_CRITICAL;
    x = prvErrorCount;
    prvErrorCount := x + 1;
    portEXIT_CRITICAL;
}
```

**Critical section creates
atomic read-and-
increment operation**

Assignment statements are not atomic

- ◆ The code from the previous slide can be rewritten to make it look as though the ErrorCount update is a single operation.
- ◆ Unfortunately this does not help.
 - ❖ On a Load/Store architecture the memory read and write must be separate instructions and therefore can be interrupted in the middle.
 - ❖ Even on an architecture which has a single instruction which increments memory there is no guarantee this will be used by the compiler.
- ◆ Make sure all such operations are inside critical sections for safety.

```
static int prvErrorCount = 0;
```

```
void vRecordError(void)
{
    portENTER_CRITICAL;
    prvErrorCount=prvErrorCount+1;
    portEXIT_CRITICAL;
}
```

Critical section is needed because assignment operation may not be atomic (depends on compiler)

Re-entrant Functions

- ◆ Functions which are safe to call from multiple tasks are called **re-entrant**.
- ◆ `vRecordError()` with a critical section protecting the memory read/write is re-entrant
- ◆ To be re-entrant a function must:
 - ❖ Store intermediate results on its stack, not in static or global variables
 - ❖ Ensure all atomic operations on static or global variables or hardware are protected by critical sections
 - ❖ Only call other functions which are re-entrant
- ◆ NB – a function can always be made re-entrant by enclosing its entire body in a critical section
 - ❖ If the function is potentially long this solution is undesirable.
 - ❖ If function can BLOCK this is an error

```
static int prvErrorCount = 0;
```

```
static int x;
```

```
vRecordError()  
{ /* this function assumes  
   overflow will never happen */  
   portENTER_CRITICAL;  
   x = prvErrorCount;  
   prvErrorCount := x + 1;  
   portEXIT_CRITICAL;  
}
```

Critical section protects both x & prvErrorCount so this is re-entrant.

NB why does x static here mean less RAM use than x as a local (previous slides)?

Is **printf** re-entrant?

- ◆ You cannot assume the implementation will be re-entrant – **printf** may use static storage for intermediate results which will be shared between different concurrent function calls
- ◆ Even if this is OK, the results of printing out from two different tasks may be that the individual characters printed are interleaved.
 - ❖ This is usually not the desired behaviour
- ◆ In general do not assume that standard library functions are re-entrant unless this is stated.
 - ❖ You can always create re-entrant wrappers to non-re-entrant functions:

```
unsigned int urand_safe(void)
{
    int x; /* why must x be local here? */
    portENTER_CRITICAL;
    x = rand(); /*library random number routine*/
    portEXIT_CRITICAL;
    return x;
}
```

Inconsistent Data Structures

```
MonitorTask()
```

```
{  
  for (;;) {  
    [ read temperatures ]  
    Temp1 = [ first tank temperature ]  
    Temp2 = [ second tank temperature ]  
    vTaskDelay(100);  
  }  
}
```

```
ControlTask()
```

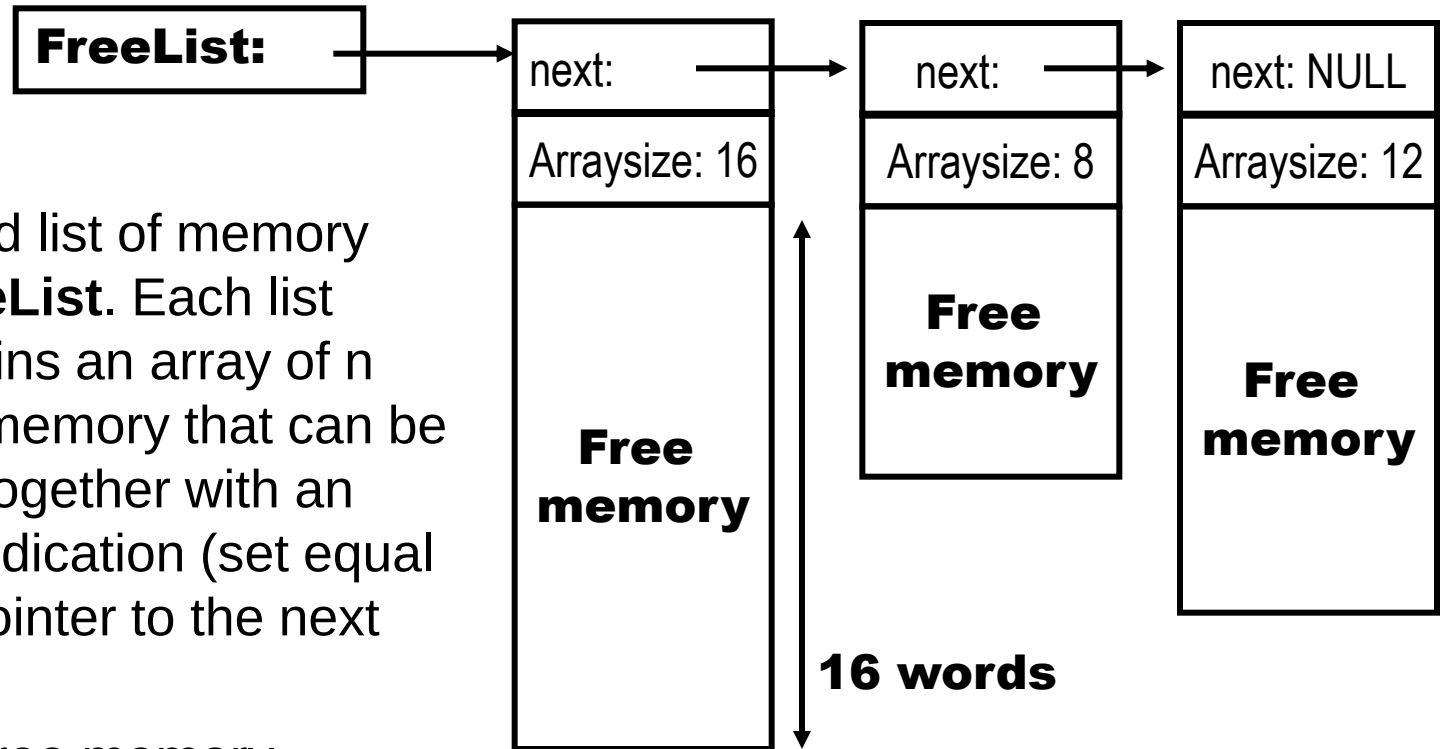
```
{  
  for (;;) {  
    If (Temp1 != Temp2) [ sound a buzzer ]  
    [ do other stuff ]  
  }  
}
```

- ◆ A common cause of the **shared data problem** is when a data structure is updated **inconsistently**. In the code below **ControlTask()** expects **Temp1** & **Temp2** to be temperatures read in the same iteration of **MonitorTask()**.
 - ❖ This will not in general be the case
- ◆ Here **Temp1**, **Temp2** form a data structure which must remain in a consistent state when it is read or written.
 - ❖ The two reads in **ControlTask()** must be atomic
 - ❖ The two writes in **MonitorTask()** must be atomic

Case study: Best Fit Memory Allocation

- ◆ Sophisticated RTOS require a memory management module which will allocate memory dynamically to tasks as they require it, and accept back memory which is no longer required (free).
 - ❖ This module will control all the system free RAM (heap)
 - ❖ NB – static & global variables are allocated by the compiler & not from heap
 - ❖ Task stack memory is allocated from the heap when tasks are created.
- ◆ Memory management is not easy if arbitrary size blocks of memory can be requested and freed, because the "pool" of available memory becomes increasingly fragmented as time progresses.
- ◆ **Best fit allocation** is a simple method which has quite good performance as long as blocks are not continually allocated and freed.
- ◆ Basic idea
 - ❖ keep a list of all the currently free blocks of memory
 - ❖ **MallocWord(int n)** returns an allocated block of size n words from this list on a best-fit basis
 - ❖ **Free()** Adds free block of memory back into the list.

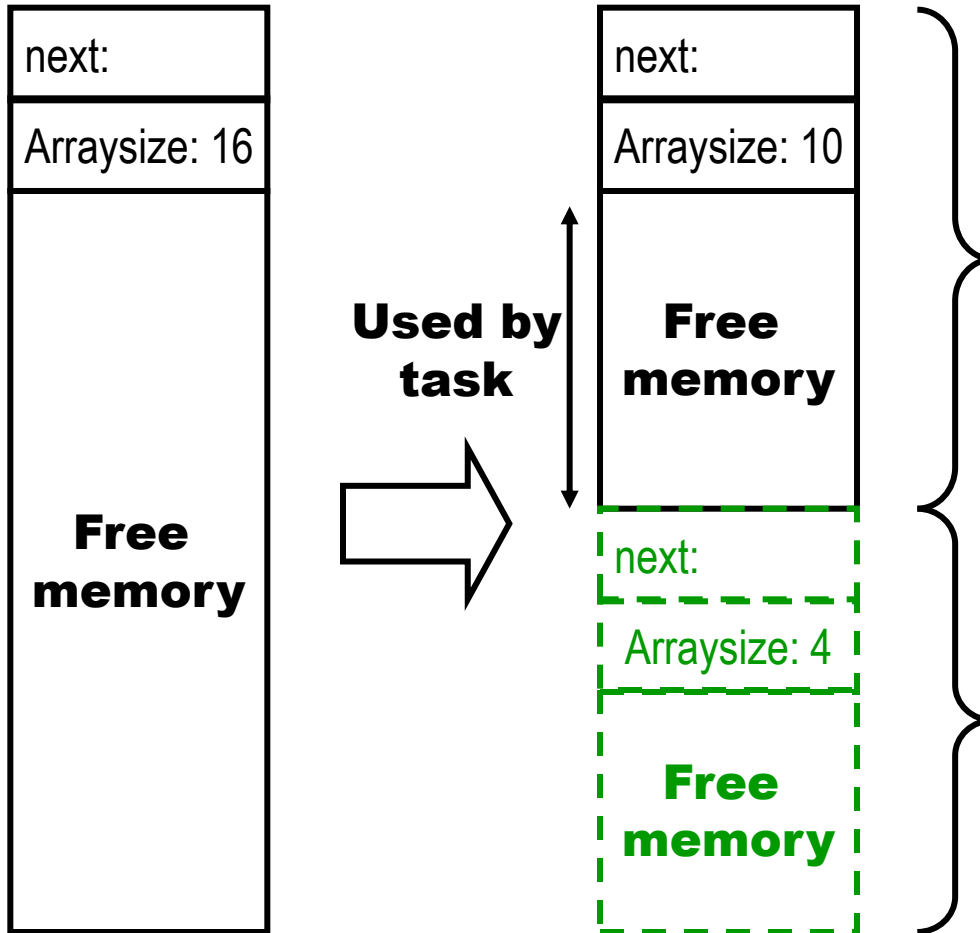
Memory Allocation (2)



- ◆ Use a linked list of memory blocks **FreeList**. Each list node contains an array of n bytes (the memory that can be allocated) together with an arraysize indication (set equal to n) and pointer to the next list node.
- ◆ Initially all free memory (assumed contiguous) forms a single node.
- ◆ Optimisation: store list sorted by arraysize, smallest first

Memory allocation (3)

best fit block:



Old node has size field changed and its free memory is given to requesting task.

Note that the Freenode header is not touched by the application and will be used when the block is freed

New node is created which contains spare space and is added to FreeList

If *best fit* block in FreeList is too big

Sequential code to allocate memory

```
Typedef struct {
    Fnode *p;           /*pointer to next node*/
    int arraysize;      /*size in words of memory block*/
    int mem[0];         /*start of array of free words in the node*/
} Fnode;

Fnode *FreeList=NULL;

void *MallocWord( int n)
{
    Fnode *q = NULL; /* return Null if no mem left */
    Fnode *p = [ smallest node in freeList with
                  arraysize >= n, if one such exists ]
    [ remove p from Freelist ]
    If ( [ space left for another block in p mem array ] ) {
        q = [ new Freenode structure of correct size ]
        [ adjust arraysize field of p ]
        [ insert q into Freelist ]
    }
    return p->mem;
}
```

Data Sharing Issues

- ◆ The shared data structure is the **FreeList** linked list. Operations performed on this are:
 - ❖ Working out the "best-fit" node – *read*
 - ❖ Deleting a chosen node – *read/write*
 - ❖ Inserting a new node – *read/write*
- ◆ Each of these operations must clearly be atomic.
- ◆ During allocation, in order to delete the chosen node efficiently, two pointers must be kept to the node & the previous node.
 - ❖ These will become out-of-date if any list-changing operation happens between choosing the node and deleting it.
 - ❖ Also, the best-fit node must not be chosen by any other task once it has been determined and before it is deleted.
 - ❖ Therefore finding best-fit node & deleting the best-fit node must be atomic, however inserting a new node (if the node splits into two) can be a separate (also atomic) operation.

Concurrent code to allocate memory

```
void *MallocWord( int n)
/* return a pointer to a block of memory size n words */
{
    Fnode *p, *q = NULL; /* return Null if no mem left */

    TaskSuspendAll();
    p = [ smallest node in freeList with
          arraysize >= n, if one such exists ];
    if (p != NULL) [ remove p from Freelist ];
    TaskResumeAll();

    If ([ space left for another block in p mem array ]) {
        q = [ new Freenode structure of correct size ];
        [ adjust arraysize field of p ];
        [ insert q into Freelist - atomic (see next slide) ];
    }
    if (p != NULL) {      /*check whether block was allocated */
        return p->mem; /*if so, return pointer to first word of free memory */
    } else {
        return NULL;      /* if not, return NULL */
    }
}
```

This implementation assumes that MallocWord() is never used in an interrupt.

What change would have to be made if it were?

Free list node insertion

[insert q into Freelist - atomic];

/ the insertion is very fast so switch off interrupts for critical section */*

```
enterCRITICAL_SECTION();  
q->next = Freelist;  
FreeList = q;  
exitCRITICAL_SECTION();
```

Lecture 3: Summary

- ◆ We can separate functionality, and timing (scheduling) issues in an RTOS.
 - ❖ Define tasks so that they will function correctly regardless of scheduling
 - ❖ Choose scheduling (see later) to ensure hard deadlines are met
- ◆ The **execution trace** interleaves operations from different tasks arbitrarily (but we assume that all tasks able to progress will do so eventually).
- ◆ Functions which can be safely called from different tasks are called **re-entrant**.
 - ❖ They must not rely on private use of hardware or static variables except inside critical sections
 - ❖ They must not call other non-re-entrant functions
 - ❖ C Library functions (e.g. printf) are not generally re-entrant
- ◆ Data sharing between tasks leads to problems unless all **atomic operations** are protected by critical section entry & exit
- ◆ **Shared data structures** must be left in a consistent state outside of critical sections
 - ❖ Use critical sections to encapsulate write operations on a data structure
 - ❖ Use critical sections to ensure read of different parts of a data structure is consistent

Review Questions 3

- 3.1** In slide 1.55 explain why both the two reads AND the two writes must be atomic by giving in each case an execution trace that leads to an error otherwise.
- 3.2** In a priority-scheduled RTOS the highest-priority READY task will always run. Suppose in the problem from slide 1.55 you may assume that MonitorTask() is higher priority than ControlTask(). How does this change the necessary critical section code in either task? Explain your reasoning.
- 3.3** Shared-data problems are important and a common source of bugs in RTOS applications. Problem Sheet 1 contains further examples.

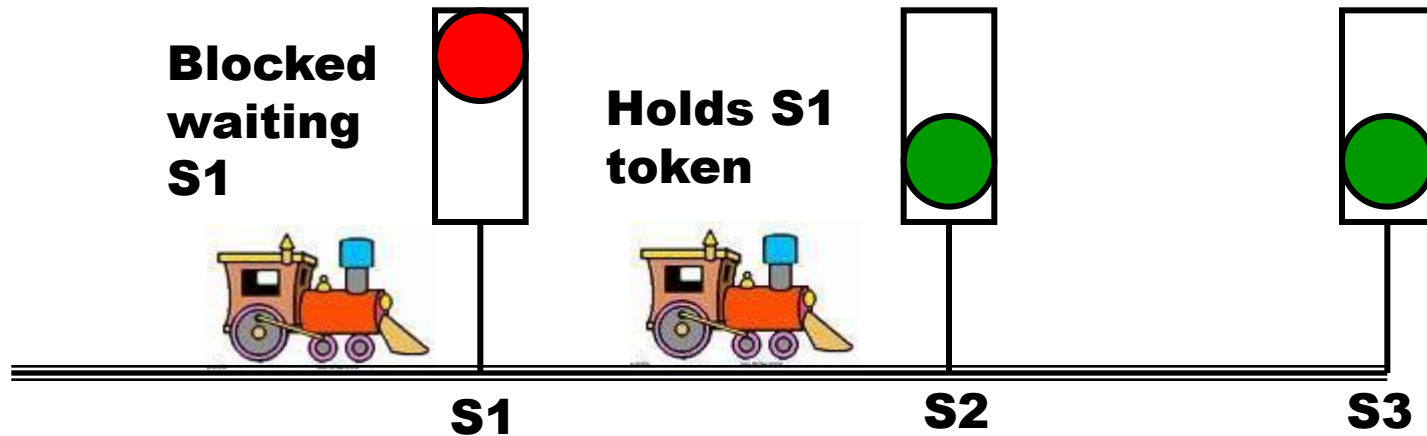
Lecture 4: Semaphores & resource access

"We semaphore from ship to ship, but they're sinking, too." Mignon McLaughlin

- ◆ The previous lecture showed how shared data structures in memory need private access from tasks to maintain consistency
 - ❖ Implemented via critical sections which enforce atomicity of operations
- ◆ This is a brute-force way to enforce exclusive access. Switching off task preemption (or, more drastically, interrupts) for long periods of time is not feasible, since it blocks ALL other system tasks
- ◆ **We need a more selective way of blocking just the tasks that try to use the shared resource**
 - ❖ This applies to many different types of resource: hardware, data structure, software.
- ◆ **The solution is to use semaphores/Mutex**
 - ❖ - Semaphore: một cơ chế (key, cờ, biến, ...) giúp quản lý các nguồn chia sẻ và đảm bảo access không bị tắc nghẽn. Nó được dùng để đồng bộ giữa các tasks và giữa các tasks và ngắt
 - Mutex (Mutual exclusion) là một cờ/key/object loại trừ lẫn nhau. Nó hoạt động như một người giữ cổng cho một phần mã, tài nguyên cho phép một luồng vào và chặn truy cập đến tất cả các luồng khác. Điều này đảm bảo rằng mã, tài nguyên được kiểm soát sẽ chỉ được truy cập bởi một luồng duy nhất tại một thời điểm.

-
- ◆ Semaphores are provided by almost all RTOS as part of the API (application program interface) together with the basic task creation & delay functions.
 - ◆ This lecture will examine:
 - ❖ Why are semaphores useful?
 - ❖ How do they work?
 - ❖ What variants of semaphore can be found in different RTOS – what are the advantages of different variants
 - ❖ What are the typical problems using semaphores.
 - ◆ When compared with use of critical sections, semaphores are expensive:
 - ❖ The semaphore operations typically take longer to execute than critical section entry & exit
 - ❖ The semaphores themselves require (a small amount) of RAM and increase the size of the RTOS kernel itself
 - ❖ RTOS will usually allow semaphore code to be removed from the RTOS when it is not needed, saving code space.

Semaphore introduction



- ◆ The name is taken from analogy with railway signal semaphores, used to ensure that AT MOST ONE TRAIN can run on a given section of track.
 - ❖ Here the shared resource is the section of railway track
- ◆ Binary semaphores have a single token which can be **acquired** by a task. The task holding the token **releases** it back to the semaphore, after which the cycle can repeat with another task.

OTP/Token: Là khóa/mã được sử dụng để lock/unlock truy cập

Trong giao dịch điện tử nó là chữ ký số và được mã hóa thành những con số trên thiết bị chuyên biệt. Mã Token tạo ra là dạng mã OTP nghĩa là mã sử dụng được một lần và tạo ngẫu nhiên cho mỗi giao dịch.

What Is Semaphore?

In concurrent programming, semaphore is typically an integer variable that is initialized to the number of resources present in the system. The value of semaphore can be modified only by two functions: wait () and signal () apart from initialization.

Semaphores which are restricted to the values 0 and 1 (lock/unlock, available/unavailable) are referred to as binary semaphore and are used to implement locks. On the other hand, Semaphores which allow an arbitrary resource count are referred to as counting semaphore.

What Are Some Of The Disadvantages Of Semaphore?

Semaphore programming is a complex method and therefore chances of not achieving mutual exclusion are high.

Semaphore is more prone to errors

The operating system has to keep track of all calls to wait and signal semaphore.

Chances of deadlock are high in semaphore incase the wait and signal operations require to be executed in the correct order.

What Are The Some Of The Advantages Of Semaphore

Does not allow multiple processes to enter critical section.

It allows more than one thread to access the critical section.

They allow efficient management of resources.

There is no wastage of process time and resources in semaphore due to busy waiting.

What You Need To Know About Semaphore

- Semaphore is a signaling mechanism and a thread waiting on a semaphore can be signaled by another thread.
- Semaphore is for processes.
- Semaphore is atomic but not singular in nature.
- A binary semaphore can be used as a mutex along with providing feature of signaling amongst threads.
- Semaphore value can be changed by any process acquiring or releasing the resource.
- Semaphore is an integer variable.
- If locked, a semaphore can be acted upon by different threads.
- A semaphore uses two atomic operations, wait and signal for process synchronization.
- Only one process can acquire binary semaphore at a time but multiple processes can simultaneously acquire semaphore in case of counting semaphore.
- Semaphore works in kernel space.
- The concept of ownership is absent in semaphore.
- Semaphore can be categorized into counting semaphore and binary semaphore.
- If all resources are being used, the process requesting for resource performs wait () operation and block itself till semaphore count become greater than one.

What Is Mutex?

In concurrent programming, Mutex is an object in a program that serves as a lock, used to negotiate mutual exclusion among threads. Mutex is a special case of the Semaphore; it is a mutual exclusion object that synchronizes access to a resource. A mutex object only allows one thread into a controlled section, forcing other threads which attempt to gain access to that section to wait until the first thread has exited from that section.

When a program is started, a mutex is created with a unique name. After this stage, any thread that needs the resource must lock the mutex from other threads while it is using the resource. The mutex is set to unlock when the data is no longer needed or the routine is finished.

What Are Some Of The Disadvantages Of Mutex?

It is difficult to lock or unlock mutex from a different context than the one that acquired it.

Only one thread is supposed to be allowed in the critical section at a time.

In case of busy waiting state the CPU time is wasted.

If a thread obtains a lock and in the process it is preempted, then, the other thread may not be able to move.

What Are Some Of The Advantages of Mutex?

There are no race conditions and data always remain consistent due to the fact that, in mutex, only one thread is in critical section at any given time.

The thread with mutex has ownership over the resource.

Mutex is typically atomic and singular in nature.

What You Need To Know About Mutex

- The Mutex is a locking mechanism that makes sure only one thread can acquire the mutex at a time and enter the critical section.
- Mutex is for thread.
- Mutex is typically atomic and singular in nature.
- A mutex can never be used as a semaphore.
- Mutex object lock is released only by the process that has acquired the lock on it.
- Mutex is an Object.
- Mutex if locked has to be unlocked by the same thread.
- Mutex object is locked or unlocked by the process requesting or releasing the resource.
- Only one thread can acquire a mutex at a time.
- Mutex works in userspace.
- The thread with mutex has ownership over the resource.
- Mutex does not have further categorization.
- If a mutex object is already locked, the process requesting for resources waits and queued by the system till lock is released.

What is a Semaphore ?

Consider a situation where there are two person who wants to share a bike. At one time only one person can use the bike. The one who has the bike key will get the chance to use it. And when this person gives the key to 2nd person, then only 2nd person can use the bike.

Semaphore is just like this Key and the bike is the shared resource. Whenever a task wants access to the shared resource, it must acquire the semaphore first. The task should release the semaphore after it is done with the shared resource. Till this time all other tasks have to wait if they need access to shared resource as semaphore is not available. Even if the task trying to acquire the semaphore is of higher priority than the task acquiring the semaphore, it will be in wait state until semaphore is released by the lower priority task.

Use of Semaphore

1. Managing Shared Resource
2. Task Synchronization

Apart from managing shared resource, task synchronization can also be performed with the help of a semaphore. In this case semaphore will be like a flag not key.

1. Unilateral Rendezvous

This is one way synchronization which uses a semaphore as a flag to signal another task.

2. Bilateral Rendezvous

This is two way synchronization performed using two semaphores. A bilateral rendezvous is similar to a unilateral rendezvous, except both tasks must synchronize with one another before proceeding.

Types of semaphore

1. Binary Semaphore

Binary semaphore is used when there is only one shared resource.

2. Counting Semaphore

To handle more then one shared resource of same type, counting semaphore is used.

3. Mutual Exclusion Semaphore or Mutex

To avoid extended priority inversion, mutexes can be used. You can check Mutex Working [here](#).

Operations on Semaphore

Basically, there are 3 operations related to the semaphore:

1. Create
2. Acquire
3. Release

Semaphore functions

API Details

Defination	<code>vSemaphoreCreateBinary(xSemaphoreHandle xSemaphore)</code>
Input Arguments	<code>xSemaphoreHandle</code> : It is a handle to the created semaphore. It can be used futher to use other semaphore APIs.
Return Value	none
Description	It creates binary semaphore. It writes non NULL value to the argument <code>xSemaphoreHandle</code> if semaphore is created.
Usage	<code>xSemaphoreHandle Sem_A = NULL;</code> <code>vSemaphoreCreateBinary(Sem_A);</code>

Defination	<code>xSemaphoreTake(xSemaphoreHandle xSemaphore, portTickType xBlockTime)</code>
Input Arguments	<code>xSemaphoreHandle</code> : It is a handle to the semaphore being obtained. <code>portTickType</code> : The time in ticks to wait for the semaphore to become available.
Return Value	TRUE if semaphore is obtained else FLASE.
Description	It is used to request to acquire the semaphore. The macro <code>portTICK_RATE_MS</code> can be used to convert this the <code>portTickType</code> to a real time. A block time of zero can be used to poll the semaphore.
Usage	<code>xSemaphoreTake(Sem_A,50);</code>

Defination	<code>xSemaphoreGive(xSemaphoreHandle xSemaphore)</code>
Input Arguments	<code>xSemaphoreHandle</code> : It is a handle to the semaphore being released.
Return Value	TRUE if semaphore is released else FLASE.
Description	It is used to release the semaphore. Don't use this API to release semaphore from ISR.
Usage	<code>xSemaphoreGive(Sem_A);</code>

```

#include "FreeRTOSConfig.h";          /* Scheduler include files. */
#include "FreeRTOS.h"
#include "task.h"
#include "croutine.h"
#include "semphr.h"
#include "uart.h"                      // Explore Embedded UART library

static void My_LPT(void* pvParameters); static void My_MPT(void* pvParameters); static void My_HPT(void* pvParameters);
xTaskHandle LPT_Handle;  xTaskHandle MPT_Handle;  xTaskHandle HPT_Handle;

xSemaphoreHandle Sem_A = NULL;

#define LED_My_LPT   0x02u
#define LED_My_MPT   0x04u
#define LED_My_HPT   0x08u
#define LED_PORT LPC_GPIO2->FIOPIN

int main(void)
{
    SystemInit();          /* Initialize the controller */
    UART_Init(38400);       /* Initialize the Uart module */
    LPC_GPIO2->FIODIR = 0xffffffffu;
    vSemaphoreCreateBinary(Sem_A); /* Create binary semaphore */
    if(Sem_A != NULL)
    {
        UART_Printf("\n\r\n\r\nSemaphore successfully created, Creating low priority task");
        xTaskCreate( My_LPT, ( signed char * )"LowTask", configMINIMAL_STACK_SIZE, NULL, 1, &LPT_Handle );
        vTaskStartScheduler();
    } else
        UART_Printf("\n\rFailed to create Semaphore");
    while(1);
    return 0;
}

```

```

static void My_LPT(void* pvParameters)
{
    unsigned char LowPrio;

    LED_PORT = LED_My_LPT;          /* Led to indicate the execution of My_LPT*/

    LowPrio = uxTaskPriorityGet(LPT_Handle);
    UART_Printf("\n\rLPT:%d,Acquiring semaphore",LowPrio);

    xSemaphoreTake(Sem_A,portMAX_DELAY);

    UART_Printf("\n\rLPT: Creating HPT");
    xTaskCreate( My_HPT, ( signed char * )"HighTask", configMINIMAL_STACK_SIZE, NULL, 3, &HPT_Handle );

    LED_PORT = LED_My_LPT;          /* Led to indicate the execution of My_LPT*/
    LowPrio = uxTaskPriorityGet(LPT_Handle);
    UART_Printf("\n\rLPT:%d Creating MPT",LowPrio);
    xTaskCreate( My_MPT, ( signed char * )"MidTask", configMINIMAL_STACK_SIZE, NULL, 2, &MPT_Handle );

    LED_PORT = LED_My_LPT;          /* Led to indicate the execution of My_LPT*/
    LowPrio = uxTaskPriorityGet(LPT_Handle);
    UART_Printf("\n\rLPT:%d Releasing Semaphore",LowPrio);
    xSemaphoreGive(Sem_A);

    LED_PORT = LED_My_LPT;          /* Led to indicate the execution of My_LPT*/
    LowPrio = uxTaskPriorityGet(LPT_Handle);
    UART_Printf("\n\rFinally Exiting LPT:%d",LowPrio);
    vTaskDelete(LPT_Handle);
}

```

```

static void My_MPT(void* pvParameters)
{
    uint8_t MidPrio;

    LED_PORT = LED_My_MPT;      /* Led to indicate the execution of My_MPT*/

    MidPrio = uxTaskPriorityGet(MPT_Handle);

    UART_Printf("\n\rIn MPT:%d",MidPrio);

    vTaskDelete(MPT_Handle);
}

static void My_HPT(void* pvParameters)
{
    uint8_t HighPrio;

    LED_PORT = LED_My_HPT;      /* Led to indicate the execution of My_HPT*/
    HighPrio = uxTaskPriorityGet(HPT_Handle);
    UART_Printf("\n\rIn HPT:%d, trying to Acquire the semaphore",HighPrio);

    HighPrio = uxTaskPriorityGet(HPT_Handle);
    xSemaphoreTake(Sem_A,portMAX_DELAY);
    LED_PORT = LED_My_HPT;      /* Led to indicate the execution of My_HPT*/
    UART_Printf("\n\rIn HPT:%d, Acquired the semaphore",HighPrio);

    HighPrio = uxTaskPriorityGet(HPT_Handle);
    UART_Printf("\n\rIn HPT:%d, releasing the semaphore",HighPrio);
    xSemaphoreGive(Sem_A);

    UART_Printf("\n\rExiting the HPT");
    vTaskDelete(HPT_Handle);
}

```

Button_LCD_UART Example (continued)

```
//STM32F103ZET6 FreeRTOS Test
#include "stm32f10x.h"
//#include "stm32f10x_it.h"
#include "mytasks.h"
//task priorities
#define mainLED_TASK_PRIORITY          ( tskIDLE_PRIORITY )
#define mainButton_TASK_PRIORITY      ( tskIDLE_PRIORITY )
#define mainButtonLEDs_TASK_PRIORITY  ( tskIDLE_PRIORITY +1)
#define mainLCD_TASK_PRIORITY         ( tskIDLE_PRIORITY )
#define mainUSART_TASK_PRIORITY       ( tskIDLE_PRIORITY )
#define mainLCD_TASK_STACK_SIZE configMINIMAL_STACK_SIZE+50
#define mainUSART_TASK_STACK_SIZE configMINIMAL_STACK_SIZE+50
int main(void)
{
    //init hardware
    LEDsInit();
    ButtonsInit();
    LCD_Init();
    Usart1Init();

    xTaskCreate( vLEDFlashTask, ( signed char * ) "LED", configMINIMAL_STACK_SIZE, NULL, mainLED_TASK_PRIORITY, NULL );
    xTaskCreate( vButtonCheckTask, ( signed char * ) "Button", configMINIMAL_STACK_SIZE, NULL, mainButton_TASK_PRIORITY, NULL );
    xTaskCreate( vButtonLEDsTask, ( signed char * ) "ButtonLED", configMINIMAL_STACK_SIZE, NULL, mainButtonLEDs_TASK_PRIORITY, NULL );
    xTaskCreate( vLCDTask, ( signed char * ) "LCD", mainLCD_TASK_STACK_SIZE, NULL, mainLCD_TASK_PRIORITY, NULL );
    xTaskCreate( vUSARTTask, ( signed char * ) "USART", mainUSART_TASK_STACK_SIZE, NULL, mainUSART_TASK_PRIORITY, NULL );

    //start scheduler
    vTaskStartScheduler();
    //you should never get here
    while(1)
    {
    }
}
```

```

//mytasks.c
#include "mytasks.h"
#include <math.h>; #include <stdio.h>; #include <stdlib.h>; #include <string.h>

const char * const pcUsartTaskStartMsg = "USART task started.\r\n";
const char * const pcLCDTaskStartMsg = " LCD task started.";

static xSemaphoreHandle xButtonWakeupSemaphore = NULL;
static xSemaphoreHandle xButtonTamperSemaphore = NULL;
static xSemaphoreHandle xButtonUser1Semaphore = NULL;
static xSemaphoreHandle xButtonUser2Semaphore = NULL;
xQueueHandle RxQueue, TxQueue;
char stringbuffer[39];

void vLEDFlashTask( void *pvParameters )
{
    portTickType xLastWakeTime;
    const portTickType xFrequency = 1000;
    xLastWakeTime=xTaskGetTickCount();
    for( ;; )
    {
        LEDToggle(5);
        vTaskDelayUntil(&xLastWakeTime,xFrequency);
    }
}

```

```

void vButtonCheckTask( void *pvParameters )
{
    //for debounce
    static uint8_t count;
    portTickType xLastWakeTime;
    const portTickType xFrequency = 20;
    xLastWakeTime=xTaskGetTickCount();
    //create semaphores for each button
    vSemaphoreCreateBinary(xButtonWakeupSemaphore);
    vSemaphoreCreateBinary(xButtonTamperSemaphore);
    vSemaphoreCreateBinary(xButtonUser1Semaphore);
    vSemaphoreCreateBinary(xButtonUser2Semaphore);
    //check if semaphores were created successfully
    if((xButtonWakeupSemaphore!=NULL)&&(xButtonTamperSemaphore!=NULL)
    &&(xButtonUser1Semaphore!=NULL)&&(xButtonUser2Semaphore!=NULL))
    {
        //successfully created
        //resets initial semaphores to 0
        xSemaphoreTake(xButtonWakeupSemaphore, (portTickType)0);
        xSemaphoreTake(xButtonTamperSemaphore, (portTickType)0);
        xSemaphoreTake(xButtonUser1Semaphore, (portTickType)0);
        xSemaphoreTake(xButtonUser2Semaphore, (portTickType)0);
    } else {
        //send error of failure
    }
    for (;;)
    {
        if (ButtonRead(BWAKEUPPORT, BWAKEUP)==pdTRUE)
        {
            count++;
            if(count==DEBOUNCECOUNTS)
            {
                xSemaphoreGive(xButtonWakeupSemaphore);
                count = 0;
            }
        }
    }
}

```

```

if (ButtonRead(BTAMPERPORT, BTAMPER)==pdTRUE)
{
    count++;
    if(count==DEBOUNCECOUNTS)
    {
        xSemaphoreGive(xButtonTamperSemaphore);
        count = 0;
    }
}
if (ButtonRead(BUSER1PORT, BUSER1)==pdTRUE)
{
    count++;
    if(count==DEBOUNCECOUNTS)
    {
        xSemaphoreGive(xButtonUser1Semaphore);
        count = 0;
    }
}
if (ButtonRead(BUSER2PORT, BUSER2)==pdTRUE)
{
    count++;
    if(count==DEBOUNCECOUNTS)
    {
        xSemaphoreGive(xButtonUser2Semaphore);
        count = 0;
    }
}
vTaskDelayUntil(&xLastWakeTime,xFrequency);
}
}

```


Mutex functions

Mutex is a special type of binary semaphore used for controlling access to the shared resource. It is used to avoid extended priority inversion using priority inheritance technique.

Priority inheritance can be implemented in two ways : changing the priority of the task trying to access the mutex:

1. to the priority equal to the priority of the task acquiring the mutex **(adopted in FreeRTOS)**

or

2. to the higher priority than the priority of the task acquiring the mutex

so that the task trying to access the mutex will immediately get the mutex when other task releases the mutex.

API Details

Create Mutex

{{#Widget:LibTable}}

Defination	vSemaphoreCreateMutex(xSemaphoreHandle xSemaphore)
Input Arguments	xSemaphoreHandle : It is a handle to the created mutex. It can be used futher to use other semaphore APIs.
Return Value	none
Description	It creates the mutex. It writes non NULL value to the argument xSemaphoreHandle if mutex is created.
Usage	xSemaphoreHandle Sem_A = NULL; vSemaphoreCreateMutex(Sem_A);

All other APIs required to use the mutex are same as semaphore. You can find them [here](#).

```

#include "FreeRTOSConfig.h"    /* Scheduler include files. */
#include "FreeRTOS.h"
#include "task.h"
#include "croutine.h"
#include "semphr.h"
#include "uart.h"              // Explore Embedded UART library

static void My_LPT(void* pvParameters); static void My_MPT(void* pvParameters); static void My_HPT(void* pvParameters);
xTaskHandle LPT_Handle;  xTaskHandle MPT_Handle;  xTaskHandle HPT_Handle;
xSemaphoreHandle xSemaphore = NULL;

#define LED_My_LPT  0x02u      //Low/Medium/High Priority Task
#define LED_My_MPT  0x04u
#define LED_My_HPT  0x08u
#define LED_PORT LPC_GPIO2->FIOPIN

int main(void)
{
    SystemInit();              /* Initialize the controller */
    UART_Init(38400);          /* Initialize the Uart module */
    LPC_GPIO2->FIODIR = 0xffffffffu;
    xSemaphore = xSemaphoreCreateMutex();      /* Create Mutex */
    if(xSemaphore != NULL)
    {
        UART_Printf("\n\r\nSemaphore successfully created, Creating low priority task");
        xTaskCreate( My_LPT, ( signed char * )"LowTask", configMINIMAL_STACK_SIZE, NULL, 1, &LPT_Handle );
        vTaskStartScheduler();      //Run My_LPT (Low Priority Task)
    } else
        UART_Printf("\n\rFailed to create Semaphore");
    while(1);    //you should never get here?
    return 0;
}

```

```

static void My_LPT(void* pvParameters)
{
    unsigned char LowPrio;

    LED_PORT = LED_My_LPT;    /* Led to indicate the execution of My_LPT*/

    LowPrio = uxTaskPriorityGet(LPT_Handle);
    UART_Printf("\n\rLPT:%d,Acquiring semaphore",LowPrio);

    xSemaphoreTake(xSemaphore,portMAX_DELAY);

    UART_Printf("\n\rLPT: Creating HPT");
    xTaskCreate( My_HPT, ( signed char * )"HighTask", configMINIMAL_STACK_SIZE, NULL, 3, &HPT_Handle );

    LED_PORT = LED_My_LPT;    /* Led to indicate the execution of My_LPT*/
    LowPrio = uxTaskPriorityGet(LPT_Handle);
    UART_Printf("\n\rLPT:%d Creating MPT",LowPrio);
    xTaskCreate( My_MPT, ( signed char * )"MidTask", configMINIMAL_STACK_SIZE, NULL, 2, &MPT_Handle );

    LED_PORT = LED_My_LPT;    /* Led to indicate the execution of My_LPT*/
    LowPrio = uxTaskPriorityGet(LPT_Handle);
    UART_Printf("\n\rLPT:%d Releasing Semaphore",LowPrio);
    xSemaphoreGive(xSemaphore);

    LED_PORT = LED_My_LPT;    /* Led to indicate the execution of My_LPT*/
    LowPrio = uxTaskPriorityGet(LPT_Handle);
    UART_Printf("\n\rFinally Exiting LPT:%d",LowPrio);
    vTaskDelete(LPT_Handle);

}

```

```
static void My_MPT(void* pvParameters)
{
    uint8_t MidPrio;

    LED_PORT = LED_My_MPT;    /* Led to indicate the execution of My_MPT*/
    MidPrio = uxTaskPriorityGet(MPT_Handle);

    UART_Printf("\n\rIn MPT:%d",MidPrio);

    vTaskDelete(MPT_Handle);
}
```

```
static void My_HPT(void* pvParameters)
{
    uint8_t HighPrio;

    LED_PORT = LED_My_HPT;    /* Led to indicate the execution of My_HPT*/
    HighPrio = uxTaskPriorityGet(HPT_Handle);
    UART_Printf("\n\rIn HPT:%d, trying to Acquire the semaphore",HighPrio);

    HighPrio = uxTaskPriorityGet(HPT_Handle);
    xSemaphoreTake(xSemaphore,portMAX_DELAY);
    LED_PORT = LED_My_HPT;    /* Led to indicate the execution of My_HPT*/
    UART_Printf("\n\rIn HPT:%d, Acquired the semaphore",HighPrio);

    HighPrio = uxTaskPriorityGet(HPT_Handle);
    UART_Printf("\n\rIn HPT:%d, releasing the semaphore",HighPrio);
    xSemaphoreGive(xSemaphore);

    UART_Printf("\n\rExiting the HPT");
    vTaskDelete(HPT_Handle);
}
```


Semaphore Implementation

- ◆ We need only keep track of whether the semaphore has the token (state=1) or not (state=0).
- ◆ When a task waits on the semaphore either it is given the token immediately (state→0) or it blocks until the token is released by another task.
- ◆ Tasks using the semaphore must call a semaphore wait function first followed by a semaphore release or signal function.

SemaWait(Semaphore s)

```
{
  if (s->state == 1)
    s->state = 0;
} else {
  [add current task to s->waiters]
  [suspend current task]
}
/* if suspended another task will run */
```

Critical sections

SemaSignal(Semaphore s)

```
{
  if ( [ s->waiters is empty ] ) {
    s->state = 1; /* give token back */
  } else {
    [wake up highest priority task in s->waiters]
    /* task we have woken up is implicitly given token */
  }
  /* the task woken up may preempt current task and */
  /* run immediately if higher priority than signal task */
}
```

Semaphore Usage

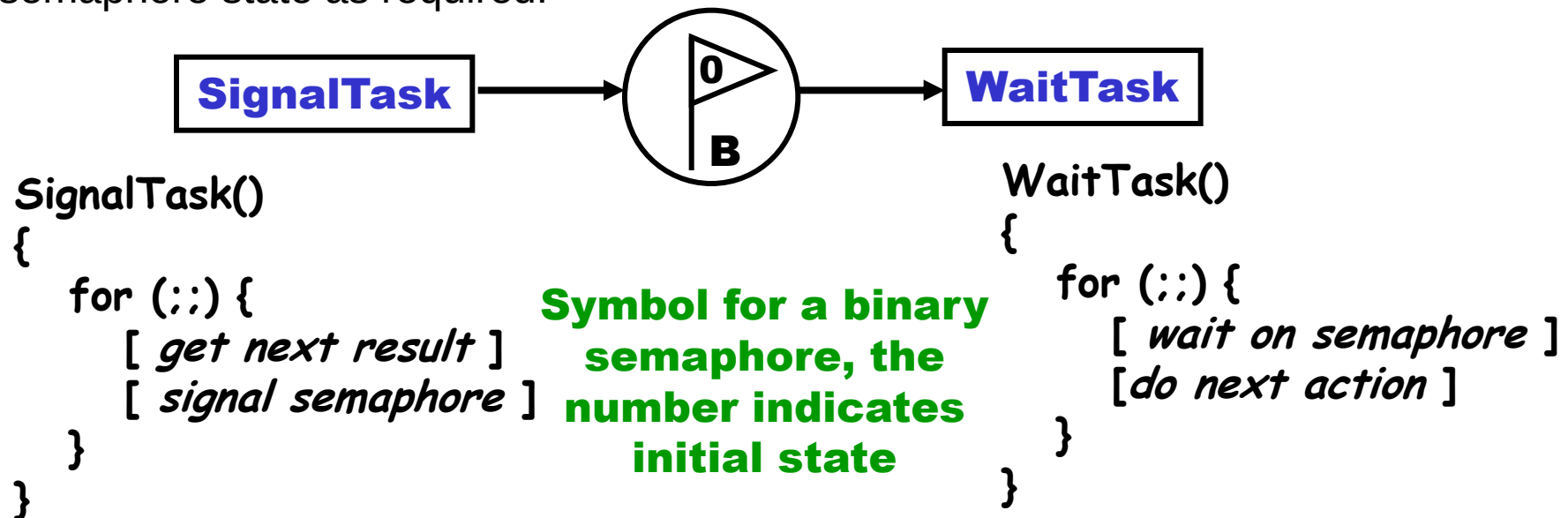
- ◆ There is no agreement on the names for the two basic semaphore operations
 - ❖ The common names are shown in the table – all make sense.
- ◆ Typical FreeRTOS code shown below – **with a TIMEOUT to detect when tasks never get semaphore**

Command to acquire token	Command to release token
Wait	Signal
Acquire	Release
Take	Give
Pend	Post
Lock	Unlock

```
#include "semaphr.h"
xSemaphoreHandle s;
#define TIMEOUT 100; /* max no of ticks to wait for sema */
Task1()
{
    /* create the semaphore in just ONE task */
    vSemaphoreCreateBinary( s); /* in FreeRTOS semaphore creation is managed via a macro */
    for (;;) {
        Semaphore Wait includes optional timeout
        if (xSemaphoreTake(s, TIMEOUT)==pdFALSE) [ handle timeout error ];
        /* use shared resource */
        if (xSemaphoreGive(s) == pdFALSE) [ handle multiple signal error – should never happen ];
    }
}
```

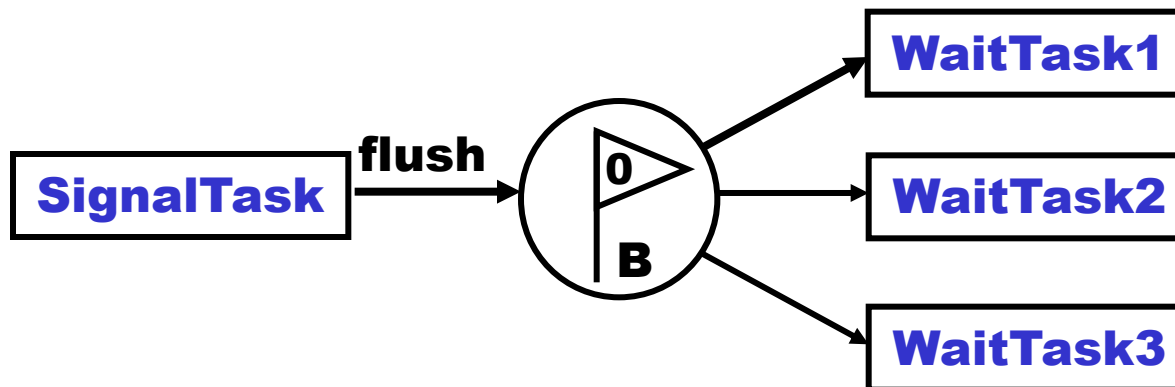
Signal & Wait Paradigm

- ◆ The simplest use for a binary semaphore is unusual, because it does not protect a resource.
- ◆ The semaphore acts as an RTOS primitive to synchronise a waiting task WaitTask with a signalling task SignalTask. The semaphore use is illustrated in the diagram
- ◆ For this application **the semaphore must be initialised without a token (state=0)**. If the RTOS does not allow creation like this an initial call to SemaWait() immediately after creation (which will not block) will have the effect of changing semaphore state as required.



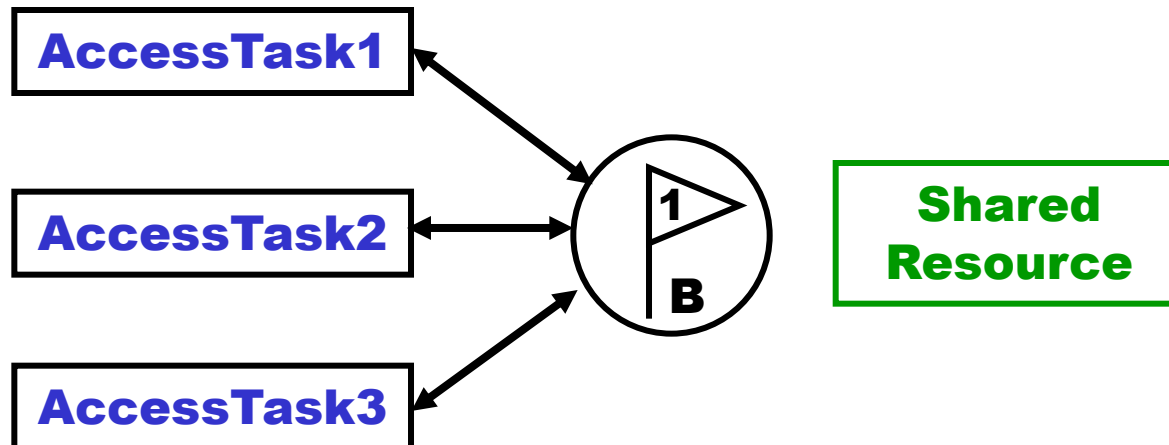
One to Many Synchronisation Paradigm

- ◆ Sometimes multiple tasks must be synchronised with a single task.
- ◆ Some RTOS provide a **SemaFlush()** command which wakes up all the semaphore's list of waiters.
- ◆ Otherwise a flush command can be simulated by reading the number of waiting tasks and signalling that number of times to the semaphore.
 - ❖ FreeRTOS does not provide the facilities to do this in the API
 - ❖ We will cover implementation under FreeRTOS in Part 2.



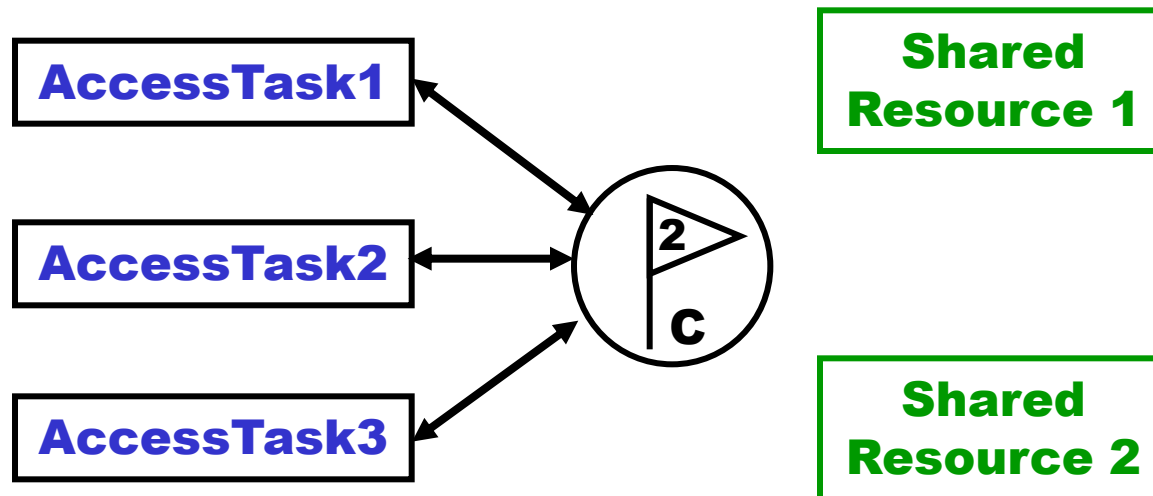
Mutual Exclusive Access Paradigm

- ◆ Here the double-arrow indicates semaphore wait followed by semaphore signal, with resource access allowed in the section of code between these two calls
- ◆ Note the initial value is 1



Multiple Resource Exclusive Access Paradigm

- ◆ A **counting semaphore** can be used to control access to N identical resources.
 - ❖ The initial value is set to the number of resources (here 2)
 - ❖ at any time at most 2 tasks have access to a token and so can use a resource



Recursive (nested) semaphore use

- ◆ When semaphores control an exclusive resource the same issue arises as for critical sections (slide 1.39).
- ◆ In this code the semaphore operations are executed in order 1,2,3,4.
 - ❖ Operation 2 will cause the task to block indefinitely.
 - ❖ Operation 3 will cause the semaphore to be released when it should not be.
- ◆ Some semaphore APIs allow nested use of semaphore Take & Give operations by a task. The inner operations (2 & 3) have no affect on semaphore state so that the semaphore token is taken for the whole period of the outermost section.
 - ❖ This allows freer coding style with semaphores
 - ❖ Usually found on specialised "mutex" semaphores

```
void f1(void)
{
1: SemaphoreTake( s);
  f2();
  /* want this still to hold s */
4: SemaphoreGive(s);
  /* no longer holding s after
   outermost exit */
}
```

```
void f2(void)
{
2: SemaphoreTake( s);
  /* uses shared resource */
3: SemaphoreGive( s);
}
```

**This code will not work
with normal semaphores**

```

void *MallocWord( int n)
{
    Fnode *q = NULL; /* return Null if no mem left */
    Fnode *p ;
    SemaphoreTake(FreeListSema);
    p = [ smallest node in freeList with arraysize >= n, if one such exists ]
    [ remove p from Freelist ]
    SemaphoreGive(FreeListSema);
    If ( [ space left for another block in p mem array ] ) {
        q = [ new Freenode structure of correct size ]
        [ adjust arraysize field of p ]
        FreeBlock(q->mem); /*reuse Freeblock() code to insert q into FreeList*/
    }
    return p->mem;
}

```

Case Study: Memory Allocation

```

Void FreeBlock( void *p)
{
    Fnode *mcb = [ pointer to Fnode structure containing block p ];
    SemaphoreTake(FreeListSema);
    mcb->next = FreeList;
    FreeList = mcb;
    SemaphoreGive(FreeListSema);
}

```

**This is correct, but
is it really a good
solution?**

When to create semaphores

- ◆ A semaphore used by, say, two tasks, must be created **before** either task first uses it.
 - ❖ In general not possible to know that one task executes before the other
- ◆ Solutions that rely on relative priority of the two tasks are bad
 - ❖ subsequent performance tuning may reverse this and it is stupid to require a condition on priorities when it is not really necessary.
- ◆ Best solution – create a semaphore in a startup task which is guaranteed to run before the tasks which use the semaphore.
- ◆ This is special case of a more general rule:
 - ❖ **DON'T RELY ON RELATIVE ORDER OF OPERATIONS IN DIFFERENT TASKS UNLESS THIS IS KNOWN FOR CERTAIN**
 - ❖ **USE A SINGLE TASK FOR SEQUENCING WHERE NEEDED**
 - E.g. the startup task can create semaphores before it enables multi-tasking

Semaphore Problems

- ◆ Semaphores provide a neat solution to many mutual exclusion problems. However, they create their own troubles. Here is a list of the most common semaphore-use bugs:
 - ❖ **Forgetting to take the semaphore.** If just one task uses the shared resource forgetting to take the semaphore there will be a potential shared-data bug.
 - ❖ **Forgetting to release the semaphore.** This is not so difficult to find, since it will probably cause the whole system to lock up waiting for the semaphore. However it is easy to do.
 - ❖ **Taking the wrong semaphore.** It is very easy to use the wrong semaphore in semaphore calls and so get unpredictable (not always obviously wrong) results.
 - ❖ **Holding a semaphore for too long.** Other tasks waiting on the semaphore will be blocked for as long as it is held and therefore may miss real-time deadlines
 - ❖ **Nested semaphore calls.** Typically a function which needs the semaphore calls another which also needs the same semaphore. The exclusive access is needed for the task, not function, so what is required is to keep the hold throughout both functions, but the nested function will block when it tries to claim the semaphore!

Summary of Semaphore Features

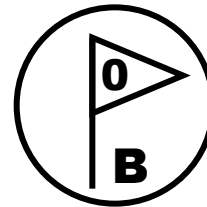
- ◆ Different RTOS will provide different selections of the following possible enhancements of a **basic binary semaphore**

Counting	Allow more than one semaphore token, implemented by a state variable which can be initialised to any positive integer. Binary semaphores are often implemented as a special case of counting semaphores (count = 1), since the code is identical.
Timeout	Return from wait operation with error indication if blocked for more than a given time – application task can then recover from the error. (Zero timeout conventionally used to disable this feature).
Flush	Release all waiting tasks through a single operation (semaphore state after flush is not defined – check RTOS documentation to be sure)
Mutex	A variable set of features useful when controlling mutually exclusive access to a resource. This may include ability to control task priorities so as to eliminate priority inversion (see later), and recursive use (slide 1.78).

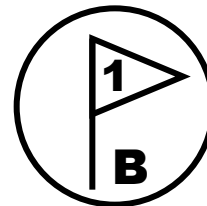
Lecture 4: Summary

- ◆ Semaphores can be used to:
 - ❖ Enforce exclusive access to resource – "**Mutual Exclusion**"
 - ❖ Synchronise tasks
- ◆ Semaphores must be created by the application before they are used
 - ❖ Make sure creation is guaranteed to be before use regardless of scheduling
- ◆ Binary semaphores are most basic function. Additional features that may be added (any combination) are:
 - ❖ Counting
 - ❖ Timeout
 - ❖ Flush
 - ❖ Mutex (discussed in lecture on scheduling)

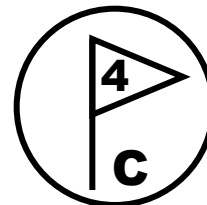
Semaphore **Wait/Signal** is indicated by an arrow **from/to** the semaphore



Binary semaphore initialised to 0



Binary semaphore initialised to 1



Counting semaphore initialised to 4

Lecture 4: Review Questions

- ◆ See web – problem sheet 2

Lecture 5: Inter-Task & Resource Sharing

Với Inter-task Communication:

1. Signal Events – Đồng bộ các task
2. Message queue – Trao đổi tin nhắn giữa các task trong hoạt động giống như FIFO
3. Mail queue – Trao đổi dữ liệu giữa các task sử dụng hàng đợi của khối bộ nhớ

Với Resource Sharing

1. Semaphores – Truy xuất tài nguyên liên tục từ các task khác nhau
2. Mutex – Đồng bộ hóa truy cập tài nguyên sử dụng Mutual Exclusion

Semaphores: Dùng để đồng bộ hóa quyền truy cập vào các tài nguyên dùng chung.

Event Flags: Dùng để đồng bộ hóa các hoạt động cần có sự phối hợp của nhiều task.

Mailboxes, Pipes, Message queues: Dùng để quản lý các thông điệp gửi đi – đến giữa các task.

Signal Events

Signal event được dùng để đồng bộ các task, ví dụ như bắt task phải thực thi tại một sự kiện nào đó được định sẵn

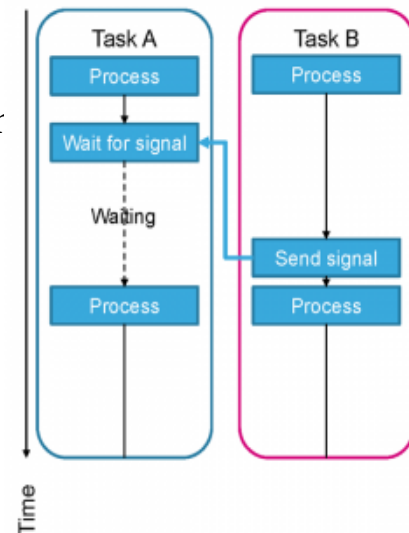
Ví dụ: Một cái máy giặt có 2 task là Task A điều khiển động cơ, Task B đọc mức nước từ cảm biến nước đầu vào

- Task A cần phải chờ nước đầy trước khi khởi động động cơ. Việc này có thể thực hiện được bằng cách sử dụng signal event
- Task A phải chờ signal event từ Task B trước khi khởi động động cơ
- Khi phát hiện nước đã đạt tới mức yêu cầu thì Task B sẽ gửi tín hiệu tới Task A

Với trường hợp này thì task sẽ đợi tín hiệu trước khi thực thi, nó sẽ nằm trong trạng thái là WAITING cho đến khi signal được set. Ngoài ra ta có thể set 1 hoặc nhiều signal trong bất kỳ các task nào khác.

Mỗi task có thể được gán tối đa là 32 signal event

Ưu điểm của nó là thực hiện nhanh, sử dụng ít RAM hơn so với semaphore và message queue nhưng có nhược điểm lại chỉ được dùng khi một task nhận được sigr



Message Queue



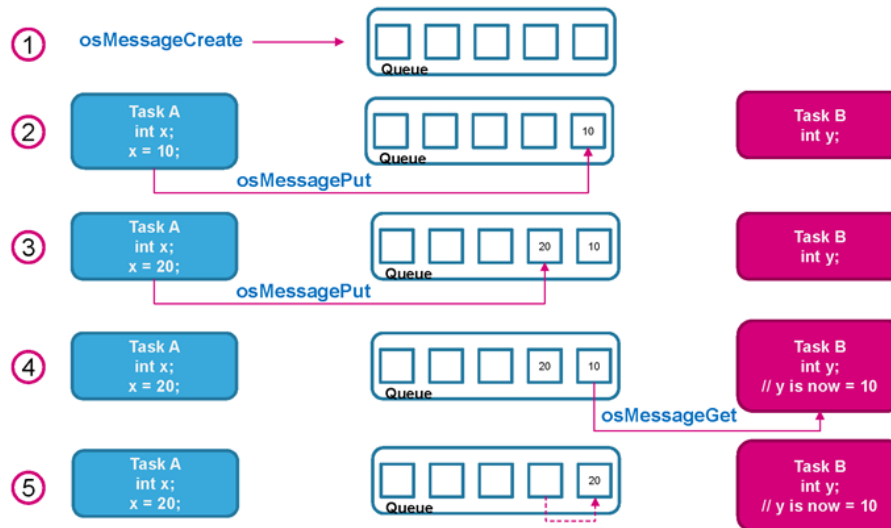
Message queue là cơ chế cho phép các task có thể kết nối với nhau, nó là một FIFO buffer được định nghĩa bởi độ dài (số phần tử mà buffer có thể lưu trữ) và kích thước dữ liệu (kích thước của các thành phần trong buffer). Một ứng dụng tiêu biểu là buffer cho Serial I/O, buffer cho lệnh được gửi tới task

Task có thể ghi vào hàng đợi (queue)

- Task sẽ bị khóa (block) khi gửi dữ liệu tới một message queue đầy
- Task sẽ hết bị khóa (unblock) khi bộ nhớ trong message queue còn trống
- Trường hợp nhiều task mà bị block thì task với mức ưu tiên cao nhất sẽ được unblock trước

Task có thể đọc từ hàng đợi (queue)

- Task sẽ bị block nếu message queue trống
- Task sẽ được unblock nếu có dữ liệu trong message queue.
- Tương tự ghi thì task được unblock dựa trên mức độ ưu tiên

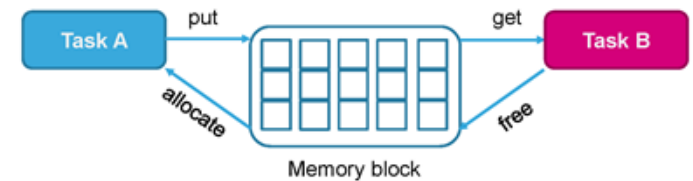


Mail Queue

Giống như message queue nhưng dữ liệu sẽ được truyền dưới dạng khối(memory block) thay vì dạng đơn. Mỗi memory block thì cần phải cấp phát trước khi đưa dữ liệu vào và giải phóng sau khi đưa dữ liệu ra

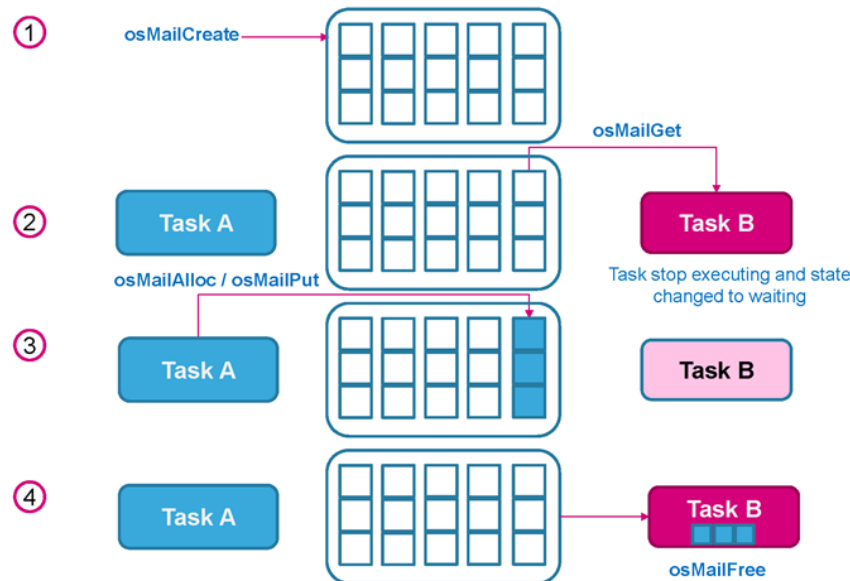
Gửi dữ liệu với mail queue

- Cấp phát bộ nhớ từ mail queue cho dữ liệu được đặt trong mail queue
- Lưu dữ liệu cần gửi vào bộ nhớ đã được cấp phát
- Đưa dữ liệu vào mail queue



Nhận dữ liệu trong mail queue bởi task khác

- Lấy dữ liệu từ mail queue, sẽ có một hàm để trả lại cấu trúc/ đối tượng
- Lấy con trỏ chứa dữ liệu
- Giải phóng bộ nhớ sau khi sử dụng dữ liệu



Semaphore

Được sử dụng để đồng bộ task với các sự kiện khác trong hệ thống. Có 2 loại

1. Binary semaphore

- Trường hợp đặc biệt của counting semaphore
- Có duy nhất 1 token
- Chỉ có 1 hoạt động đồng bộ

2. Counting semaphore

- Có nhiều token
- Có nhiều hoạt động đồng bộ
- Counting semaphore được dùng để:

+ Counting event

* Một event handler sẽ 'give' semaphore khi có event xảy ra (tăng giá trị đếm semaphore)

* Một task handler sẽ 'take' semaphore khi nó thực thi sự kiện (giảm giá trị đếm semaphore)

* Count value là khác nhau giữa số sự kiện xảy ra và số sự kiện được thực thi

* Trong trường hợp counting event thì semaphore được khởi tạo giá trị đếm bằng 0

+ Resource management

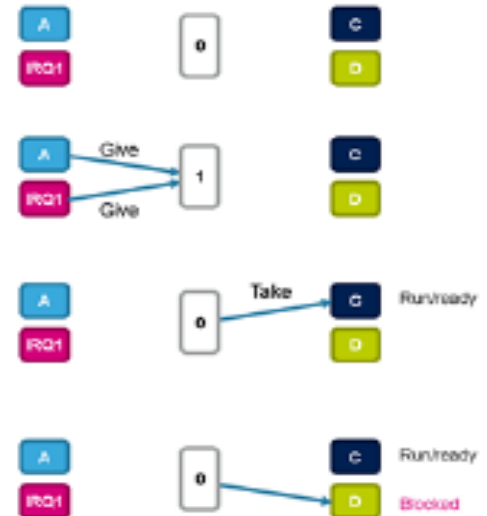
* Count value sẽ chỉ ra số resource sẵn có

* Để điều khiển và kiểm soát được resource của task dựa trên count value của semaphore (giá trị giảm), nếu count value giảm xuống bằng 0 nghĩa là không có resource nào free.

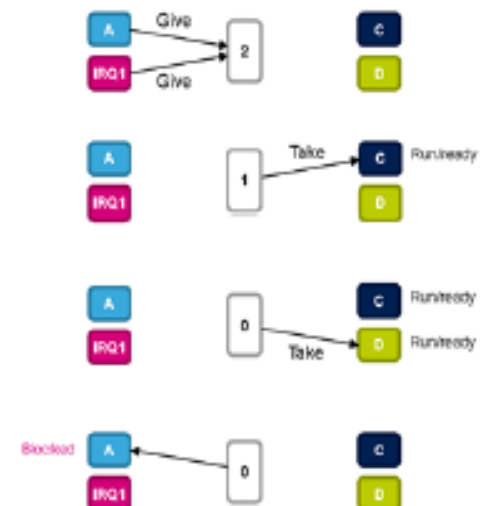
* Khi một task finish với resource thì nó sẽ give semaphore trở lại để tăng count value của semaphore.

* Trong trường hợp resource management thì count value sẽ bằng với giá trị max của count value khi semaphore được tạo.

• Binary semaphore



• Counting semaphore (value = 2)



Mutex

Sử dụng cho việc loại trừ (mutual exclusion), hoạt động như là một token để bảo vệ tài nguyên được chia sẻ. Một task nếu muốn truy cập vào tài nguyên chia sẻ

- Cần yêu cầu (đợi) mutex trước khi truy cập vào tài nguyên chia sẻ
- Đưa ra token khi kết thúc với tài nguyên.

Tại mỗi một thời điểm thì chỉ có 1 task có được mutex. Những task khác muốn cùng mutex thì phải block cho đến khi task cũ thả mutex ra.

Về cơ bản thì Mutex giống như binary semaphore nhưng được sử dụng cho việc loại trừ chứ không phải đồng bộ. Ngoài ra thì nó bao gồm cơ chế thừa kế mức độ ưu tiên (Priority inheritance mechanism) để giảm thiểu vấn đề đảo ngược ưu tiên, cơ chế này có thể hiểu đơn giản qua ví dụ sau:

- Task A (low priority) yêu cầu mutex
- Task B (high priority) muốn yêu cầu cùng mutex trên.
- Mức độ ưu tiên của Task A sẽ được đưa tạm về Task B để cho phép Task A được thực thi
- Task A sẽ thả mutex ra, mức độ ưu tiên sẽ được khôi phục lại và cho phép Task B tiếp tục thực thi.



.....

Lecture 5: Data Transfer & Message Queues

“Dogs come when they're called; cats take a message and get back to you later.”

Mary Bly

- ◆ Unlike semaphores, **message queues** can carry data (messages) from one task to another.
- ◆ Like semaphores, tasks can block on message queues and therefore they can be used for task synchronisation in much the same way as semaphores
 - ❖ Message queues are however NOT used for exclusive access to a shared resource
- ◆ This lecture will focus on different types of data transfer between tasks and message queues as a solution to this problem.
- ◆ The implementation of message queues is a good deal more complex than that of semaphores, and will be considered at high level only here. Concrete implementations will be discussed in Part 2.

MESSAGE QUEUE

A queue is a FIFO (First In First Out) type buffer where data is written to the end (tail) of the queue and removed from the front (head) of the queue. It is also possible to write to the front of a queue.

A queue can either hold the data or pointer to the data. In FreeRTOS, the data items are directly copied to the queue. Each data item is of fixed size. The size of data item and maximum number of data items are fixed when queue is created.

Queue can also be used as semaphore, mutex, event flag, etc. FreeRTOS does the same. It reduces memory usage in case of using multiple RTOS services e.g. semaphore and queue in same application.

Operations on queue

1. Create

2. Read

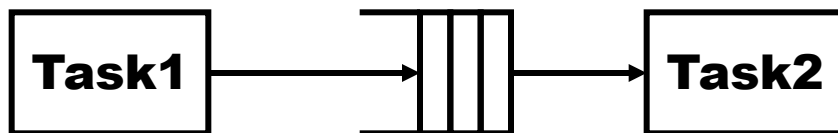
In read operation, the data item is returned. If queue is empty the requesting task will wait for specified time. If multiple tasks are waiting, then data item will be returned to the either highest priority task or the one which made the request first depending upon RTOS implementation. For the second case waiting list is maintained with each queue. In FreeRTOS, queue is implemented in FIRST way.

3. Write(Pend)

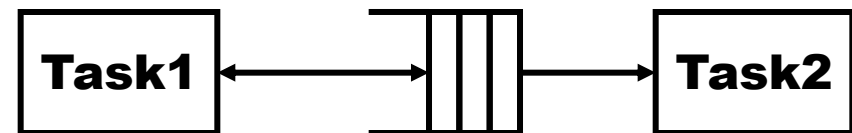
In write operation, the data item is directly copied to queue. If queue is full the requesting task will wait for specified time. For multiple tasks in wait state, process is same as read operation.

What is a Message Queue?

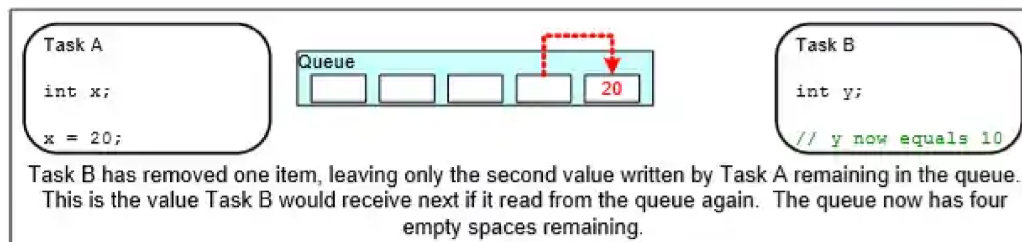
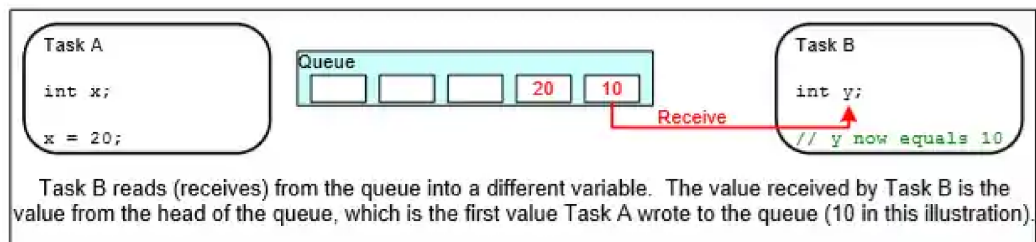
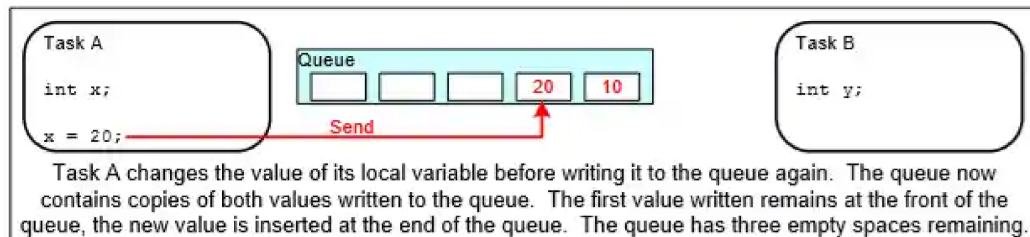
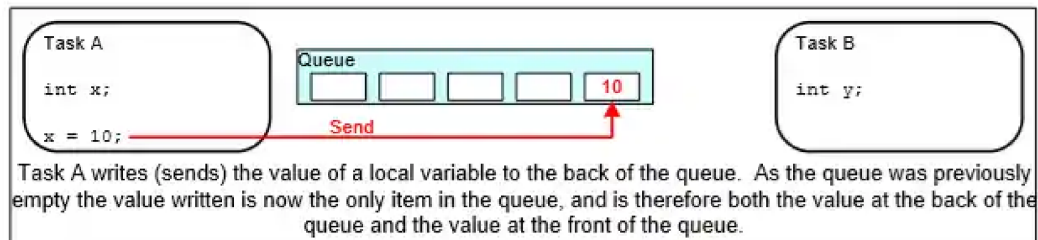
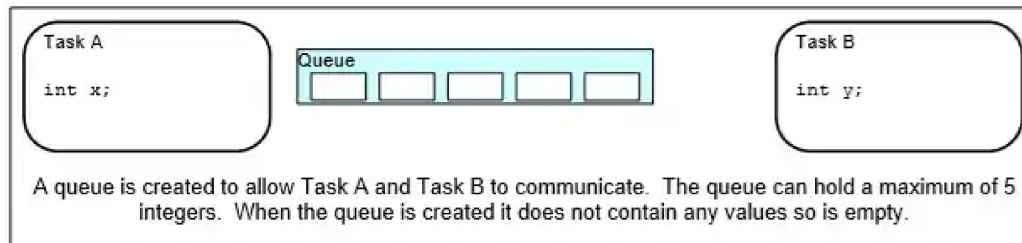
- ◆ A **Message Queue** is a dynamically created RTOS object which allows messages to be sent between tasks.
 - ❖ The queue has a first-in-first-out (FIFO) buffer which can contain any number of messages from 0 up to a fixed limit.
 - ❖ **QueueReceive()** – extract and return with the message at the front of the queue.
 - If queue is empty block until a message is written
 - ❖ **QueueSend()** – write a message to the queue
 - **Blocking send** – if queue is full block writing process until space is available for write completion (with optional timeout).
 - **Non-blocking send** – return immediately with an error indication if the write could not complete due to lack of space in queue.



**Non-blocking send
Message queue**



**Blocking send
Message queue**



API Details

Creating a queue

{{#Widget:LibTable}}

Defination	xQueueHandle xQueueCreate (unsigned portBASE_TYPE uxQueueLength, unsigned portBASE_TYPE uxItemSize)
Input Arguments	1. unsigned portBASE_TYPE : The maximum number of items that the queue can hold at a time. 2. unsigned portBASE_TYPE : The size in bytes of data item to be stored in the queue.
Return Value	xQueueHandle : NULL when queue is not created Non NULL when queue is created.
Description	It creates queue.
Usage	xQueueHandle MyQueueHandleId; MyQueueHandleId = xQueueCreate(3,20);

Writting data to the queue

{{#Widget:LibTable}}

Defination	portBASE_TYPE xQueueSend (xQueueHandle xQueue, const void * pvItemToQueue, portTickType xTicksToWait)
Description	It is equivalent to xQueueSendToBack().
Usage	char TxBuffer[MaxElementsPerQueue]="Hello world"; xQueueSend(MyQueueHandleId,TxBuffer,100)

Writing data at the front of queue

{{#Widget:LibTable}}

Defination	portBASE_TYPE xQueueSendToFront (xQueueHandle xQueue, const void * pvItemToQueue, portTickType xTicksToWait)
Input Arguments	1. xQueueHandle : handle to the queue in which data is to be posted 2. const void * : pointer to the item to be posted 3. portTickType : the max amount of time for which task will wait for a space to become available on the queue if queue is full
Return Value	returns TRUE on successfully posting the item else FALSE
Description	It posts item at the back of the queue.
Usage	char TxBuffer[MaxElementsPerQueue]="Hello world"; xQueueSendToFront(MyQueueHandleId,TxBuffer,100)

Writting data at the back of queue

{{#Widget:LibTable}}

Defination	portBASE_TYPE xQueueSendToBack (xQueueHandle xQueue, const void * pvItemToQueue, portTickType xTicksToWait)
Input Arguments	1. xQueueHandle : handle to the queue in which data is to be posted 2. const void * : pointer to the item to be posted 3. portTickType : the max amount of time for which task will wait for a space to become available on the queue if queue is full
Return Value	returns TRUE on successfully posting the item else FALSE
Description	It posts item at the front of the queue.
Usage	char TxBuffer[MaxElementsPerQueue]="Hello world"; xQueueSendToBack(MyQueueHandleId,TxBuffer,100)

Receiving data from the queue

{{#Widget:LibTable}}

Defination	portBASE_TYPE xQueueReceive (xQueueHandle xQueue, void *pvBuffer, portTickType xTicksToWait)
Input Arguments	1. xQueueHandle : handle to the queue in which data write is desired. 2. void * : pointer to the buffer where received item will be copied 3. portTickType : the max amount of time for which task will wait for a item to become available on the queue if queue is empty
Return Value	returns TRUE on successfully posting the item else FALSE
Description	It reads data from the front of the queue.
Usage	char RxBuffer[MaxElementsPerQueue]; xQueueReceive(MyQueueHandleId,RxBuffer,100);

USE OF QUEUE WITHOUT DELAYS

```
#include "FreeRTOSConfig.h"; #include "FreeRTOS.h"; #include "task.h"
#include "croutine.h"; #include "queue.h"; #include "uart.h"
```

// Explore Embedded UART library

```
#define MaxQueueSize 3
#define MaxElementsPerQueue 20
```

```
static void MyTask1(void* pvParameters); static void MyTask2(void* pvParameters);
xTaskHandle TaskHandle_1; xTaskHandle TaskHandle_2;
```

```
xQueueHandle MyQueueHandleId;
```

```
#define LED_Task1 0x02u
#define LED_Task2 0x04u
#define LED_PORT LPC_GPIO2->FIOPIN
```

```
int main(void)
{
```

```
    SystemInit();          /* Initialize the controller */
    UART_Init(38400);       /* Initialize the Uart module */
    LPC_GPIO2->FIODIR = 0xffffffffu;
```

```
    MyQueueHandleId = xQueueCreate(MaxQueueSize,MaxElementsPerQueue);    /* Create a queue */
```

```
    if(MyQueueHandleId != 0)
    {
```

```
        UART_Printf("\n\rQueue Created");
        xTaskCreate( MyTask1, ( signed char * )"Task1", configMINIMAL_STACK_SIZE, NULL, 3, &TaskHandle_1 );
        xTaskCreate( MyTask2, ( signed char * )"Task2", configMINIMAL_STACK_SIZE, NULL, 2, &TaskHandle_2 );
        vTaskStartScheduler();      /* start the scheduler */
    }
```

```
    else
```

```
        UART_Printf("\n\rQueue not Created");
```

```
    while(1);
    return 0;
```

```
}
```

```

static void MyTask1(void* pvParameters)
{
    char RxBuffer[MaxElementsPerQueue];

    LED_PORT = LED_Task1;      /* Led to indicate the execution of Task1*/
    UART_Printf("\n\rTask1, Reading the data from queue");

    if(pdTRUE == xQueueReceive(MyQueueHandleId,RxBuffer,100))
    {
        LED_PORT = LED_Task1;      /* Led to indicate the execution of Task1*/
        UART_Printf("\n\rBack in task1, Received data is:%s",RxBuffer);
    } else
    {
        LED_PORT = LED_Task1;      /* Led to indicate the execution of Task1*/
        UART_Printf("\n\rBack in task1, No Data received:");
    }

    vTaskDelete(TaskHandle_1);
}

```

```

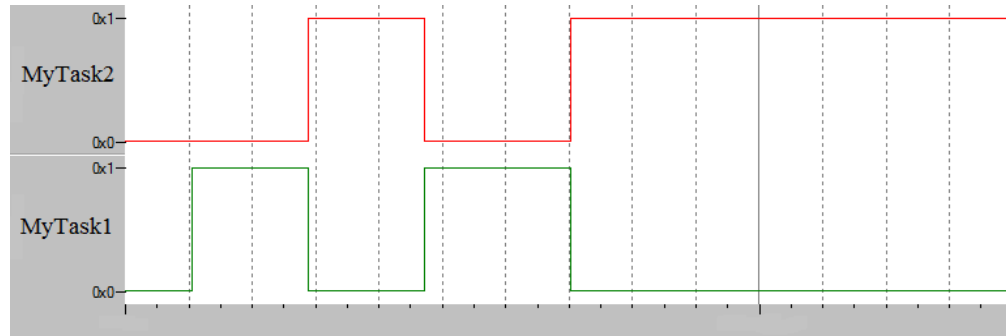
static void MyTask2(void* pvParameters)
{
    char TxBuffer[MaxElementsPerQueue]="Hello world";

    LED_PORT = LED_Task2;      /* Led to indicate the execution of Task2*/
    UART_Printf("\n\rTask2, Filling the data onto queue");

    if(pdTRUE == xQueueSend(MyQueueHandleId,TxBuffer,100))
    {
        LED_PORT = LED_Task2;      /* Led to indicate the execution of Task2*/
        UART_Printf("\n\rSuccessfully sent the data");
    } else
    {
        LED_PORT = LED_Task2;      /* Led to indicate the execution of Task2*/
        UART_Printf("\n\rSending Failed");
    }

    UART_Printf("\n\rExiting task2");
    vTaskDelete(TaskHandle_2);
}

```



USE OF QUEUE WITH DELAYS

```
#include "FreeRTOSConfig.h"; #include "FreeRTOS.h"; #include "task.h"
#include "croutine.h"; #include "queue.h"; #include "uart.h"           // Explore Embedded UART library
#define MaxQueueSize 3
#define MaxElementsPerQueue 20
static void MyTask1(void* pvParameters); static void MyTask2(void* pvParameters);
xTaskHandle TaskHandle_1; xTaskHandle TaskHandle_2;
xQueueHandle MyQueueHandleId;
#define LED_Task1 0x02u
#define LED_Task2 0x04u
#define LED_PORT LPC_GPIO2->FIOPIN
int main(void)
{
    SystemInit();                /* Initialize the controller */
    UART_Init(38400);            /* Initialize the Uart module */
    LPC_GPIO2->FIODIR = 0xffffffffu;
    MyQueueHandleId = xQueueCreate(MaxQueueSize,MaxElementsPerQueue);    /* Create a queue */
    if(MyQueueHandleId != 0)
    {
        UART_Printf("\n\rQueue Created");
        xTaskCreate( MyTask1, ( signed char * )"Task1", configMINIMAL_STACK_SIZE, NULL, 3, &TaskHandle_1 );
        xTaskCreate( MyTask2, ( signed char * )"Task2", configMINIMAL_STACK_SIZE, NULL, 2, &TaskHandle_2 );
        vTaskStartScheduler();    /* start the scheduler */
    }
    else
        UART_Printf("\n\rQueue not Created");
    while(1);
    return 0;
}
```

```
static void MyTask1(void* pvParameters)
```

```
{
```

```
char RxBuffer[MaxElementsPerQueue];
```

```
LED_PORT = LED_Task1;      /* Led to indicate the execution of Task1*/
```

```
UART_Printf("\n\rTask1, Waiting for some time");
```

```
vTaskDelay(200);
```

```
LED_PORT = LED_Task1;      /* Led to indicate the execution of Task1*/
```

```
UART_Printf("\n\rTask1, Reading the data from queue");
```

```
if(pdTRUE == xQueueReceive(MyQueueHandleId,RxBuffer,100))
```

```
{
```

```
    LED_PORT = LED_Task1;      /* Led to indicate the execution of Task1*/
```

```
    UART_Printf("\n\rBack in task1, Received data is:%s",RxBuffer);
```

```
} else
```

```
{
```

```
    LED_PORT = LED_Task1;      /* Led to indicate the execution of Task1*/
```

```
    UART_Printf("\n\rBack in task1, No Data received:");
```

```
}
```

```
vTaskDelete(TaskHandle_1);
```

```
}
```

```
static void MyTask2(void* pvParameters)
```

```
{
```

```
char TxBuffer[MaxElementsPerQueue]="Hello world";
```

```
LED_PORT = LED_Task2;      /* Led to indicate the execution of Task2*/
```

```
UART_Printf("\n\rTask2, Filling the data onto queue");
```

```
if(pdTRUE == xQueueSend(MyQueueHandleId,TxBuffer,100))
```

```
{
```

```
    LED_PORT = LED_Task2;      /* Led to indicate the execution of Task2*/
```

```
    UART_Printf("\n\rSuccessfully sent the data");
```

```
} else
```

```
{
```

```
    LED_PORT = LED_Task2;      /* Led to indicate the execution of Task2*/
```

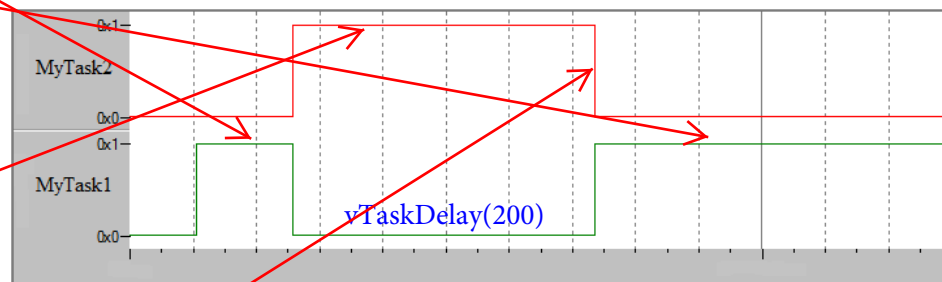
```
    UART_Printf("\n\rSending Failed");
```

```
}
```

```
UART_Printf("\n\rExiting task2");
```

```
vTaskDelete(TaskHandle_2);
```

```
}
```



Button_LCD_UART Example (continued)

```
//STM32F103ZET6 FreeRTOS Test
#include "stm32f10x.h"
//#include "stm32f10x_it.h"
#include "mytasks.h"
//task priorities
#define mainLED_TASK_PRIORITY          ( tskIDLE_PRIORITY )
#define mainButton_TASK_PRIORITY      ( tskIDLE_PRIORITY )
#define mainButtonLEDs_TASK_PRIORITY  ( tskIDLE_PRIORITY +1)
#define mainLCD_TASK_PRIORITY         ( tskIDLE_PRIORITY )
#define mainUSART_TASK_PRIORITY       ( tskIDLE_PRIORITY )
#define mainLCD_TASK_STACK_SIZE configMINIMAL_STACK_SIZE+50
#define mainUSART_TASK_STACK_SIZE configMINIMAL_STACK_SIZE+50
int main(void)
{
    //init hardware
    LEDsInit();
    ButtonsInit();
    LCD_Init();
    Usart1Init();

    xTaskCreate( vLEDFlashTask, ( signed char * ) "LED", configMINIMAL_STACK_SIZE, NULL, mainLED_TASK_PRIORITY, NULL );
    xTaskCreate( vButtonCheckTask, ( signed char * ) "Button", configMINIMAL_STACK_SIZE, NULL, mainButton_TASK_PRIORITY, NULL );
    xTaskCreate( vButtonLEDsTask, ( signed char * ) "ButtonLED", configMINIMAL_STACK_SIZE, NULL, mainButtonLEDs_TASK_PRIORITY, NULL );
    xTaskCreate( vLCDTask, ( signed char * ) "LCD", mainLCD_TASK_STACK_SIZE, NULL, mainLCD_TASK_PRIORITY, NULL );
    xTaskCreate( vUSARTTask, ( signed char * ) "USART", mainUSART_TASK_STACK_SIZE, NULL, mainUSART_TASK_PRIORITY, NULL );

    //start scheduler
    vTaskStartScheduler();
    //you should never get here
    while(1)
    {
    }
}
```

Button_LCD_UART Example (continued)

```
/*mytasks.c*/
```

```
#include "mytasks.h";#include <math.h>;#include <stdio.h>;#include <stdlib.h>;#include <string.h>
```

```
const char * const pcUsartTaskStartMsg = "USART task started.\r\n";
```

```
const char * const pcLCDTaskStartMsg = " LCD task started.";
```

```
static xSemaphoreHandle xButtonWakeupSemaphore = NULL;static xSemaphoreHandle xButtonTamperSemaphore = NULL;
```

```
static xSemaphoreHandle xButtonUser1Semaphore = NULL;static xSemaphoreHandle xButtonUser2Semaphore = NULL;
```

```
xQueueHandle RxQueue, TxQueue;
```

```
char stringbuffer[39];
```

```
void vUSARTTask( void *pvParameters ){
```

```
    portTickType xLastWakeTime;
```

```
    const portTickType xFrequency = 50;
```

```
    xLastWakeTime=xTaskGetTickCount();
```

```
    char ch;
```

```
    // Create a queue capable of containing 128 characters.
```

```
    RxQueue = xQueueCreate( configCOM0_RX_BUFFER_LENGTH, sizeof( portCHAR ) );
```

```
    TxQueue = xQueueCreate( configCOM0_TX_BUFFER_LENGTH, sizeof( portCHAR ) );
```

```
    if(( TxQueue == 0 )||( RxQueue == 0 ))
```

```
    {
```

```
        // Failed to create the queue.
```

```
        LEDOn(1);    LEDOn(3);    LEDOn(5);
```

```
    }
```

```
    USART1PutString(pcUsartTaskStartMsg,strlen( pcUsartTaskStartMsg ));
```

```
    for( ;; )
```

```
    {
```

```
        //Echo back
```

```
        if (Usart1GetChar(&ch))
```

```
        {
```

```
            Usart1PutChar(ch);
```

```
        }
```

```
        vTaskDelayUntil(&xLastWakeTime,xFrequency);
```

```
    }
```

```
}
```

```
void vLEDFlashTask( void *pvParameters )
```

```
{
```

```
    portTickType xLastWakeTime;
```

```
    const portTickType xFrequency = 1000;
```

```
    xLastWakeTime=xTaskGetTickCount();
```

```
    for( ;; )
```

```
    {
```

```
        LEDToggle(5);
```

```
        vTaskDelayUntil(&xLastWakeTime,xFrequency);
```

```
    }
```

```
}
```

```
uint32_t Usart1GetChar(char *ch){
```

```
    if(xQueueReceive( RxQueue, ch, 0 ) == pdPASS)
```

```
    {
```

```
        return pdTRUE;
```

```
    }
```

```
    return pdFALSE;
```

```
}
```

```
uint32_t Usart1PutChar(char ch){
```

```
    if(xQueueSend( TxQueue, &ch, 10 ) == pdPASS )
```

```
    {
```

```
        USART_ITConfig(USART1, USART_IT_TXE, ENABLE);
```

```
        return pdTRUE;
```

```
    }else{
```

```
        return pdFAIL;
```

```
    }
```

```
}
```

Button_LCD_UART Example (continued)

The rest load is left for the interrupt handler, which responds to interrupt requests and sends bytes from TxQueue or receives and places them to RxQueue. If we want to send the data to the Queue from an ISR, we have to use the interrupt safe version of these functions.

```
void USART1_IRQHandler(void)
```

```
{
    long xHigherPriorityTaskWoken = pdFALSE;
    uint8_t ch;
    //if Receive interrupt
    if (USART_GetITStatus(USART1, USART_IT_RXNE) != RESET)
    {
        ch=(uint8_t)USART_ReceiveData(USART1);
        xQueueSendToBackFromISR( RxQueue, &ch, &xHigherPriorityTaskWoken );
    }
    if (USART_GetITStatus(USART1, USART_IT_TXE) != RESET)
    {
        if( xQueueReceiveFromISR( TxQueue, &ch, xHigherPriorityTaskWoken ) )
        {
            USART_SendData(USART1, ch);
        }else{
            //disable Transmit Data Register empty interrupt
            USART_ITConfig(USART1, USART_IT_TXE, DISABLE);
        }
    }
    portEND_SWITCHING_ISR( xHigherPriorityTaskWoken );
}
```

/* The xHigherPriorityTaskWoken parameter must be initialized to pdFALSE as it will get set to pdTRUE inside the interrupt safe API function if a context switch is required. */

/* Pass the xHigherPriorityTaskWoken value into portEND_SWITCHING_ISR(). If xHigherPriorityTaskWoken was set to pdTRUE inside xSemaphoreGiveFromISR() then calling portEND_SWITCHING_ISR() will request a context switch. If xHigherPriorityTaskWoken is still pdFALSE then calling portEND_SWITCHING_ISR() will have no effect */

It is necessary to know that special queue handling functions have to be used inside interrupt handlers, such as xQueueReceiveFromISR and xQueueSentoBackFromISR.

xQueueSendToFrontFromISR will send the data to the front of the Queue. All the data, which is already available in the queue, will shift back and next time if we read the Queue, we will get this particular data.

Also note that there is no waiting time here. So if the Queue is full, the function will simply timeout, as in the ISR, we can't afford to wait for the space to become available in the Queue.

Button_LCD_UART Example (continued)

```
/* * usart.c */
#include "usart.h"; #include "mytasks.h"
#define serPUT_STRING_CHAR_DELAY      ( 5 / portTICK_RATE_MS )
void Usart1Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure; USART_InitTypeDef USART_InitStructure; USART_ClockInitTypeDef USART_ClockInitStructure;
    //enable bus clocks
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA | RCC_APB2Periph_AFIO, ENABLE);
    //Set USART1 Tx (PA.09) as AF push-pull
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;      GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; GPIO_Init(GPIOA, &GPIO_InitStructure);
    //Set USART1 Rx (PA.10) as input floating
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10; GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    USART_ClockStructInit(&USART_ClockInitStructure);    USART_ClockInit(USART1, &USART_ClockInitStructure);
    USART_InitStructure.USART_BaudRate = 9600; USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    USART_InitStructure.USART_StopBits = USART_StopBits_1;    USART_InitStructure.USART_Parity = USART_Parity_No ;
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
    //Write USART1 parameters
    USART_Init(USART1, &USART_InitStructure);
    //Enable USART1
    USART_Cmd(USART1, ENABLE);    USART_DMACmd( USART1, ( USART_DMAREq_Tx | USART_DMAREq_Rx ), ENABLE );

    //configure NVIC
    NVIC_InitTypeDef NVIC_InitStructure;
    //select NVIC channel to configure
    NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;
    //set priority to lowest
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x0F;
    //set subpriority to lowest
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0x0F;
    //enable IRQ channel
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    //update NVIC registers
    NVIC_Init(&NVIC_InitStructure);
    //disable Transmit Data Register empty interrupt
    USART_ITConfig(USART1, USART_IT_TXE, DISABLE);
    //enable Receive Data register not empty interrupt
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
}
```

Button_LCD_UART Example (continued)

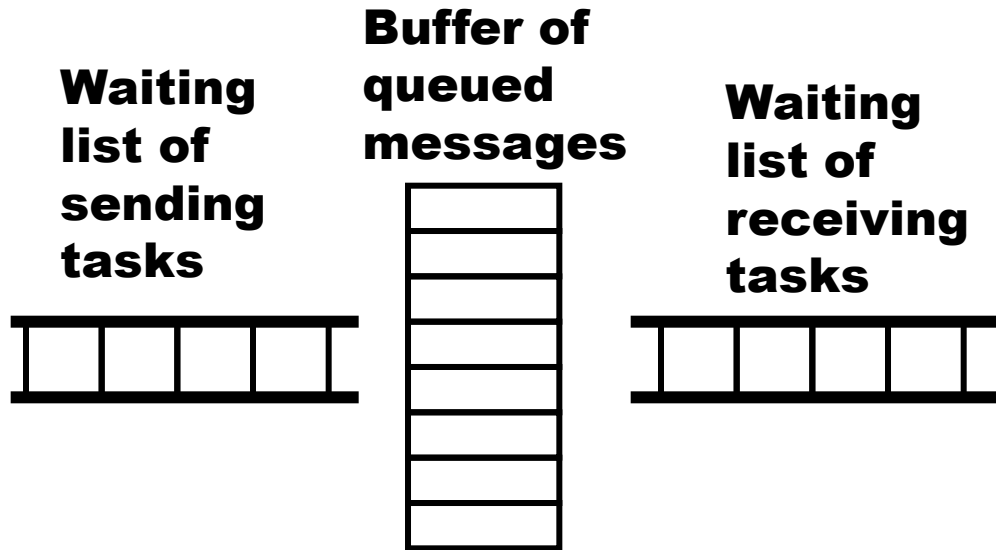
```
uint32_t Usart1PutChar(char ch)
{
    if( xQueueSend( TxQueue, &ch, 10 ) == pdPASS )
    {
        USART_ITConfig(USART1, USART_IT_TXE, ENABLE);
        return pdTRUE;
    }else{
        return pdFAIL;
    }
}

void USART1PutString( const char * const pcString, unsigned long ulStringLength)
{
    unsigned long ul;
    for( ul = 0; ul < ulStringLength; ul++ )

        {
            if( xQueueSend( TxQueue, &( pcString[ ul ] ), serPUT_STRING_CHAR_DELAY ) != pdPASS )
            {
                /* Cannot fit any more in the queue. Try turning the Tx on to clear some space. */
                USART_ITConfig( USART1, USART_IT_TXE, ENABLE );
                vTaskDelay( serPUT_STRING_CHAR_DELAY );
                /* Go back and try again. */
                continue;
            }
        }
    USART_ITConfig( USART1, USART_IT_TXE, ENABLE );
}

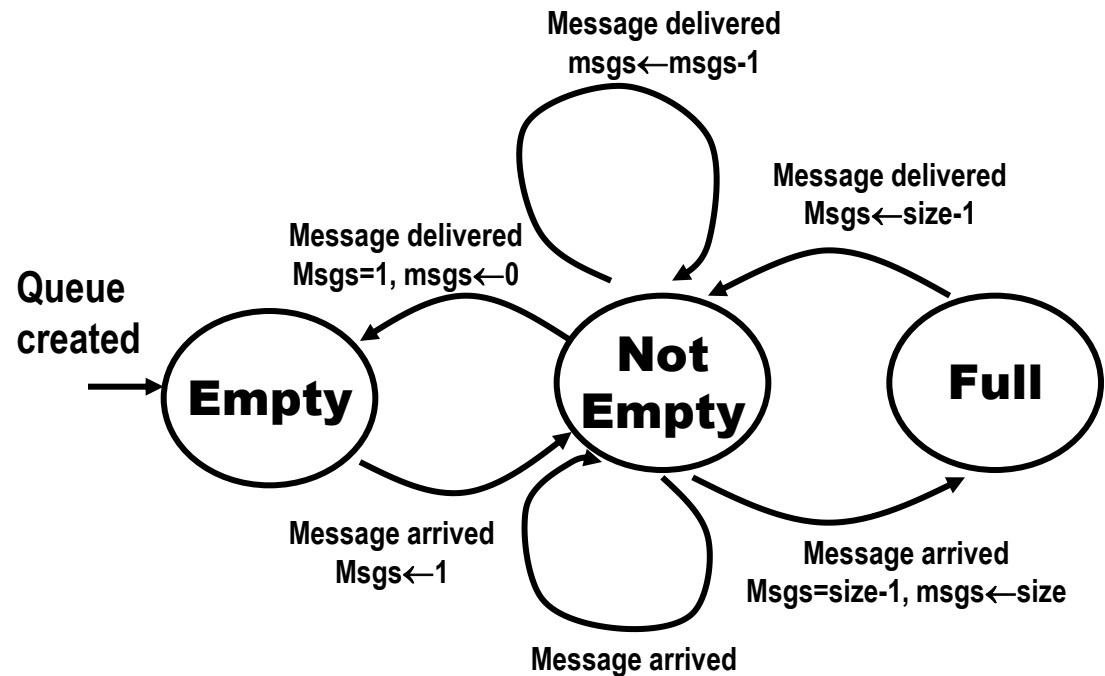
uint32_t Usart1GetChar(char *ch){
    if(xQueueReceive( RxQueue, ch, 0 ) == pdPASS)
    {
        return pdTRUE;
    }
    return pdFALSE;
}
```

Implementation



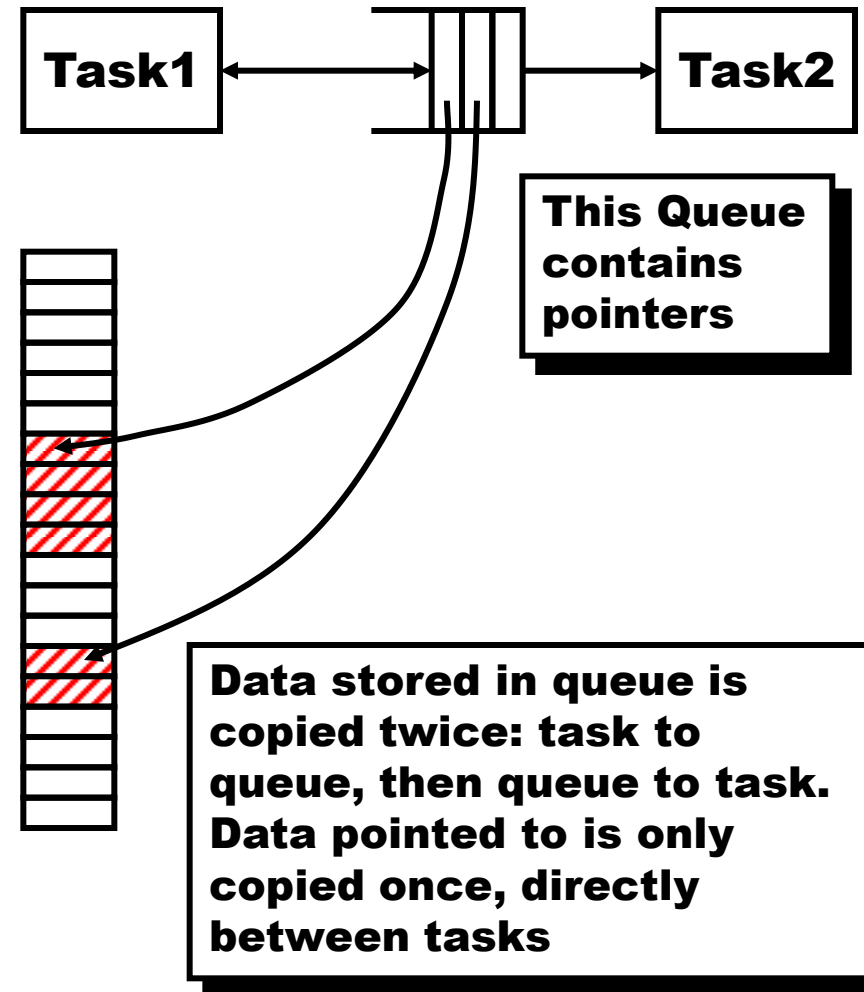
Typical usage: sending tasks never wait, there is only one receiving task in message wait/process loop

State	Receive Waiting List	Send Waiting List	Msgs Size = max length of queue
Empty	Any number	0	0
Not Empty	0	0	$1 \leq \text{msgs} < \text{size}$
Full	0	Any number	size



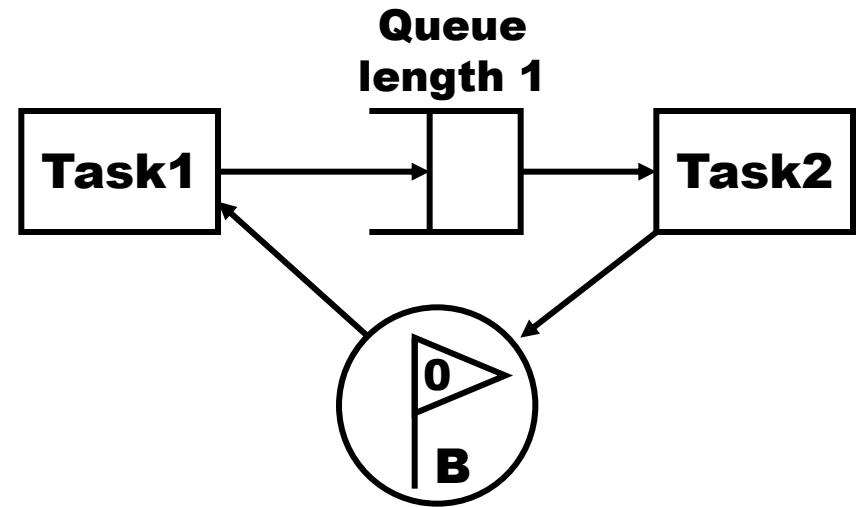
Message-queue data: to copy or not to copy

- ◆ Message queues may contain *complete messages*, or *pointers to data* elsewhere.
 - ❖ Some RTOS only allow pointers
- ◆ **Advantages of pointers**
 - ❖ Queue storage is fixed size
 - ❖ Message data can be arbitrary size
 - ❖ Data copying is minimised
- ◆ **Disadvantage of pointers**
 - ❖ Data pointed to must not be overwritten by the source task until it has been read from the queue and processed by the destination task. This may be a long time after the data was sent.
 - Care needed not to overwrite data held in buffers to soon



Interlocked one-way communication

- ◆ Special case of a message queue length 1 is sometimes called a **Mailbox**
- ◆ *Task1* sends a message to the queue and then waits on the semaphore.
- ◆ *Task 2* will receive the message, process it, and only then signal the semaphore.
- ◆ Because the two tasks stay in lock-step *Task1* knows it can over-write buffer storage used for the sent message any time after the semaphore wait ends.



Blocking send to a message queue length 1 will also provide interlocked communication, without need for a semaphore.

Can you see why if the queue contains pointers to message data the semaphore version might be preferred?

Interlocked one-way communication (2)

- ◆ Pseudocode for this problem, using a queue of pointers to message data
- ◆ It does not matter whether queue send is blocking or non-blocking since semaphore ensures it will never fail
- ◆ Note message is stored in buffer in Task1, no need for another buffer in Task2

Startup()

```
{  
    [ Create queue q1, size = 1 pointer,  
        length = 1 ]  
    [ Create binary semaphore s1, init 0 ]  
    [ Create tasks Task1 & Task2 ]  
    [ start scheduler ]  
}
```

Task1()

```
{  
    [ allocate buffer to store message ]  
    for (;;) {  
        [ write next message into buffer ]  
        [ send pointer to buffer to queue q1 ]  
        [ wait on sema s1 ]  
    }  
}
```

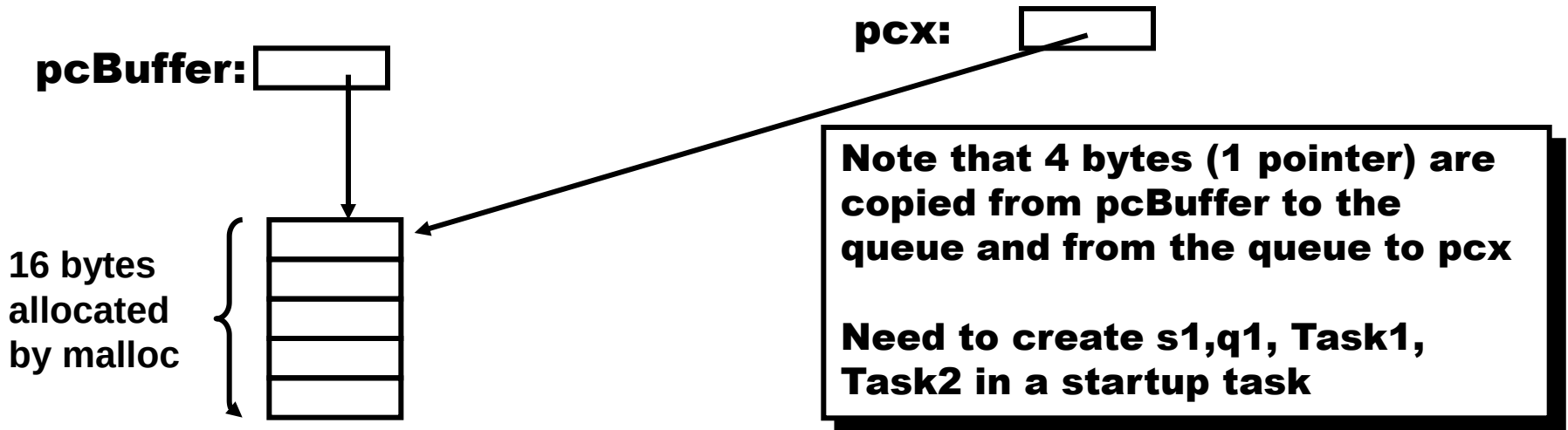
Task2()

```
{  
    for (;;) {  
        [ Receive pointer to  
            data from q1 ]  
        [ process data ]  
        [ signal to sema s1 ];  
    }  
}
```

Interlocked one-way communication: (3) details under FreeRTOS

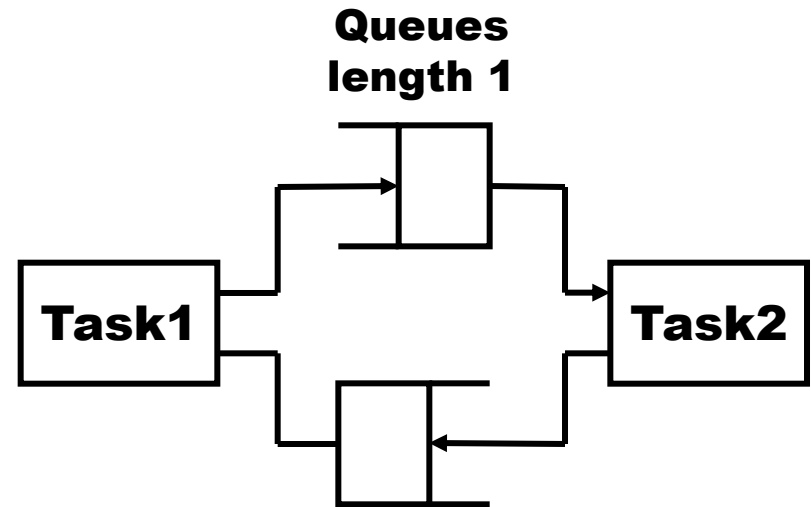
```
void Task1(void *p)
{
    /* buffer for data */
    char *pcBuffer = malloc(16);
    for (;;) {
        [ put new data into pcBuffer ]
        xQueueSend(q1, &pcBuffer, 0);
        xSemaphoreTake(s1, 0);
    }
}
```

```
void Task2( void *p)
{
    char *pcx; /* pointer to data */
    for (;;) {
        xQueueReceive(q1, &pcx, 0)
        [ process data using pcx ]
        xSemaphoreGive(s1);
    }
}
```



Interlocked two-way communication

- ◆ Often a client/server model of communication is required where each message sent has a reply.
- ◆ Two message queues of length 1 will implement this and ensure that the two tasks stay in lock-step



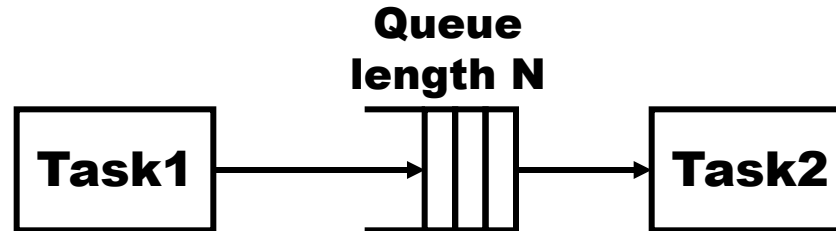
Interlocked two-way communication

- ◆ Each task will block waiting for the other so both tasks run
- ◆ Each task can only process messages when the other is blocked, so this interlocked communication will be slower than non-interlocked
- ◆ By making either q1 or q2 of length 2 it is possible to speed up the system, by sending one message in advance of the reply received so that one item is always buffered in the queue.
- ◆ **Why must Task1 also be changed to enable this?**

```
Task1()
{
    for (;;) {
        1:[ generate next message ]
        2:[ send message to queue q1 ]
        7:[ receive reply from queue s2 ]
    }
}
```

```
Task2()
{
    for (;;) {
        3:[ receive message from q1 ]
        4:[ process message ]
        5:[ generate reply ]
        6:[ write reply to q2 ];
    }
}
```

Non-interlocked one-way communication



- ◆ Non-interlocked communication is implemented by a queue of length > 1 .
- ◆ *Task1* can send up to N messages before *Task2* processes any of them.
 - ❖ After that *Task1* either blocks, or, as here, has a queue send error
 - ❖ Using blocking send would make the two tasks **weakly coupled**.
- ◆ Queue operates as a FIFO buffer
- ◆ If queue send is blocking, and N is large, this is similar to the operation of a **pipe**. However pipes are always character-oriented, and have OS primitives that allow multiple characters to be sent and received.
- ◆ Suppose *Task1* generates 1 message every 10ms, and *Task2* processes messages at an average rate of 200 per second.
 - ❖ How large must the queue be to ensure no errors in *Task1* if *Task2* stops processing messages for 100ms once every 2 seconds?

Queue Features

◆ Pointers only, or fixed size messages?

- ❖ FreeRTOS allows any fixed-size message.
 - Queue must specify message length on creation
 - Send & Receive from queue uses pointers to message storage – the specified number of bytes is copied to/from the queue
 - If queue itself contains pointers, Send & receive functions must contain pointers to pointers!

◆ Message order

- ❖ First-in First-out (FIFO) – the normal method
- ❖ Last-in first-out (LIFO). Effectively, by sending a message LIFO you are making it the first message to be read & therefore this is good for high priority messages. Note however that later LIFO messages will displace earlier ones at the head of the queue.

◆ Blocking or non-blocking Send?

- ❖ If non-blocking Send the queue has one list of blocked tasks waiting to **receive** a message
- ❖ If blocking send the queue also has a list of tasks blocked waiting to send.

◆ Timeouts allowed?

- ❖ Any blocking operation may have optional timeout

◆ Tasks woken on FIFO or priority basis?

- ❖ Priority scheduling means that normally if a list of blocked tasks waits for an event, the task woken when the event arrives is the HPT.
 - This can result in high priority tasks hogging all traffic and starving lower priority tasks.
- ❖ Alternative (less common) gives messages to tasks on a strict first-come first-served basis.

◆ Queue Broadcast/Flush?

- ❖ Wake up all tasks waiting to receive a message with a single transmitted message.

Lecture 5: Summary

- ◆ **Message queues** implement communication & synchronisation (execution ordering) between tasks
- ◆ **Basic functionality**
 - ❖ Single-ended – tasks block on reading if queue is empty
 - ❖ Double-ended – tasks also block on writing if queue is full
 - ❖ Can allow as option a non-blocking read (or write) API function
- ◆ **Storage implementation options**
 - ❖ Messages may be fixed-length or variable-length.
 - ❖ Storage for messages may be allocated by kernel or RTOS
 - ❖ Messages may be copied or shared (queue contains pointers)
- ◆ **Queue message order**
 - ❖ Strictly speaking queue is FIFO (first-in-first-out)
 - ❖ Queues can allow LIFO (stack) or priority-based order
- ◆ **Task wakeup**
 - ❖ Can be implemented FIFO (longest waiting gets message) or by priority (highest priority gets message).
- ◆ **Queue flush**
 - ❖ Deletes all outstanding messages in queue
 - ❖ Surprisingly useful

Lecture 5: Review Questions

- ◆ See web – problem sheet 2

Lecture 6: Synchronisation

"If you don't get all the clocks synchronized when the leap second occurs -- you could have potentially interesting effects. The Internet could stop working, cell phones could go out."

Geoff Chester

◆ Barrier synchronisation

- ❖ Definition

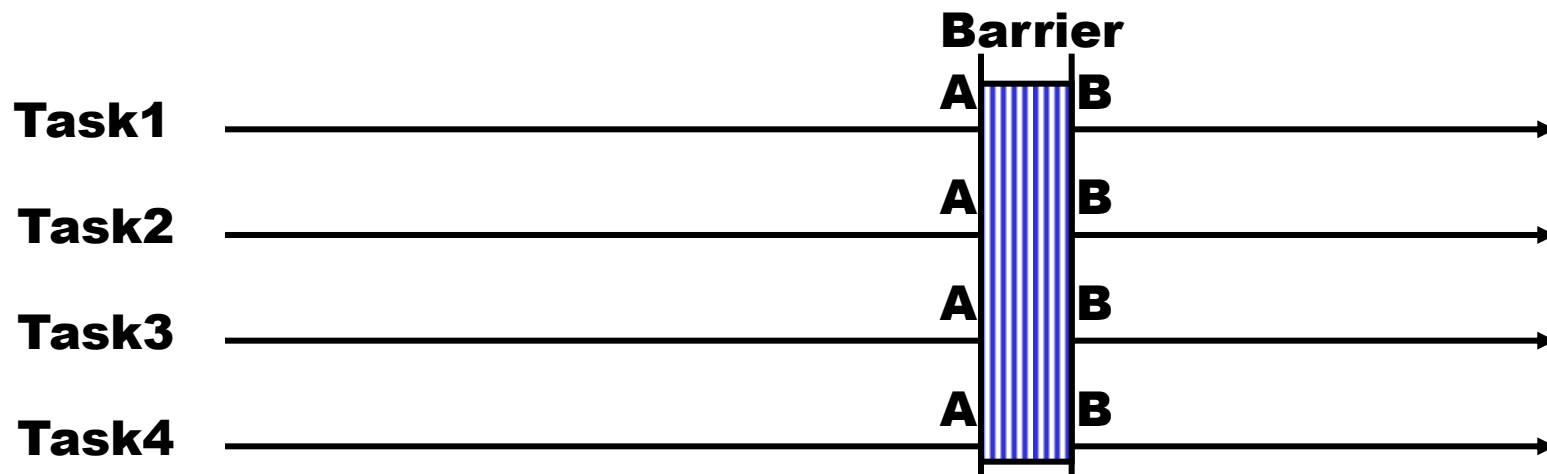
- ❖ Solutions

◆ Synchronisation Objects

- ❖ Event Registers & Event Flags

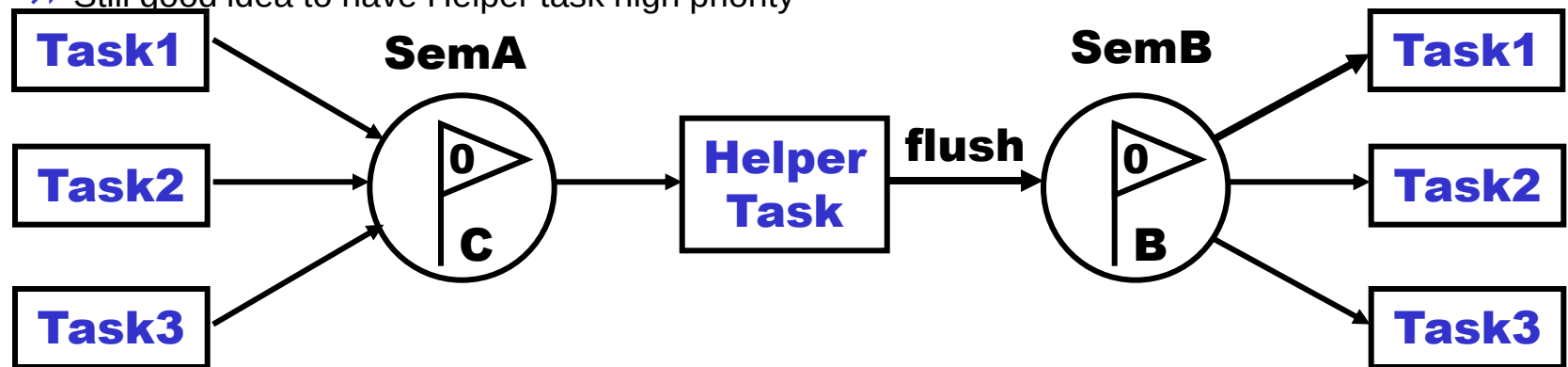
Barrier Synchronisation

- ◆ This is the typical **activity synchronisation** problem where any number of tasks need to be temporally aligned so that they all execute specific sections of code starting at the same time.
 - ❖ Found where tasks need to cooperate in the solution of a problem
 - ❖ In this case none can start until it is known that all are ready to start
- ◆ 1. When each task arrives at point A in the barrier, it must post its arrival to the other tasks, and wait
- ◆ 2. When all tasks have arrived at A, all tasks are allowed to proceed from point B in the barrier.
- ◆ The next few slides look at solutions



Solution 1: Helper Task & Semaphores

- ◆ The first solution conceptually uses a **helper task** to count the number of tasks that have arrived.
 - ❖ Each task signals to a counting semaphore **SemA** on arrival, and then waits on another binary semaphore **SemB**. The helper task loops waiting on the semaphore SemA, incrementing a private count variable every time the semaphore is signalled.
 - ❖ When the helper task has counted the correct number of SemA signals it exits the loop and signals to **SemB**.
 - All tasks must be woken, so if a flush operation on SemB is available it should be used.
 - Otherwise a loop is necessary to wake up all the tasks through repeated signals.
 - ❖ The helper task must be higher priority than that the signalling tasks if SemA is binary – otherwise counts may be lost.
 - Better solution, shown here, is for SemA to be counting semaphore.
 - Still good idea to have Helper task high priority



- ◆ Rather than have a separate helper task, the **highest priority** of the synchronising tasks (TaskH below) can serve as helper while it is waiting at the barrier. If SemA is a **counting semaphore** it does not matter if other tasks arrive first – the semaphore signals will be remembered.
- ◆ The two semaphores must be created somewhere before the barrier code executes.

```
#define NSYNCH 3 /* number of tasks to  
synchronise */
```

```
TaskH()  
/* highest priority task to synchronise */  
{  
    .....  
    /* barrier point A */  
    for (count=0; count < NSYNCH-1; count++) {  
        [ wait on SemA ]  
    }  
    for count=0; count < NSYNCH-1; count++) {  
        [signal to SemB ]  
    }  
    /* barrier point B */  
    .....  
}
```

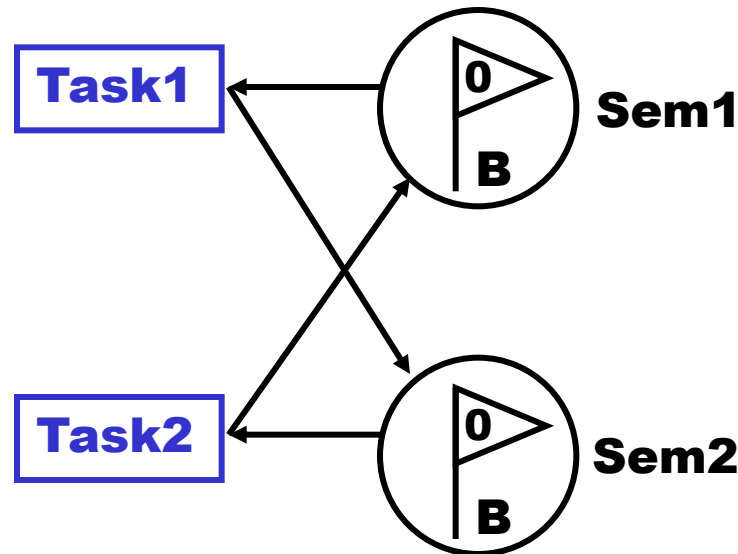
```
TaskX()  
/*all other tasks to synchronise*/  
{  
    .....  
    /* barrier point A */  
    [ Signal to SemA ]  
    [ Wait on semB ]  
    /* barrier point B */  
}
```

Alternative solutions

- ◆ It seems rather contrived to use a helper task counting arrivals
 - ❖ Ideally a semaphore could be used which allows a **negative** count value $-N$. Only after $N+1$ posts will a task waiting on this semaphore be woken.
 - Some RTOS allow this – the implementation is identical to a normal counting semaphore
 - ❖ This eliminates the loop, but two semaphores are still needed.
- ◆ Alternatively use an RTOS construct which allows tasks to wait on the conjunction (AND) of multiple condition variables can be used. Each task to synchronise will set one of the constituent condition variables and then wait on them all.
 - ❖ See later this lecture

Rendezvous synchronisation

- ◆ A special case of barrier synchronisation is where two tasks need to be synchronised. Straightforward solution uses two binary semaphores – each task signals one semaphore and then waits on the other one.
 - ❖ Write code for this using the FreeRTOS semaphore API.



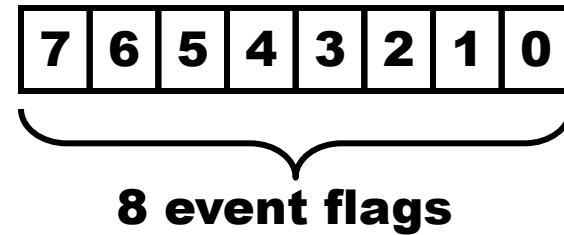
Event Flag Registers

- ◆ An Event Flag Register is a special set of boolean variables managed by the RTOS
 - ❖ typically 8 or 32 occupying one machine word of storage
 - ❖ Think of these variables as representing a set of boolean conditions of the system.
- ◆ Each variable can be set or reset independently by an API function
 - ❖ Any number of tasks can update the same or different variables
 - ❖ As system condition changes so variables can be updated by any task with knowledge of the change.
- ◆ Tasks can wait on conditions (boolean expressions) of the variables.
 - ❖ OR of arbitrary variables
 - ❖ AND of arbitrary variables
- ◆ Since different variables can be controlled by different tasks one common use is many-to-one synchronisation

Event Flag Example

```
/*Event Flag object Creation */  
/*Create 8 bits of event flags */  
/*Each bit 7:0 represents one flag */
```

```
FlagsA = EventFlagCreate();
```



```
/* Changing value of flags */
```

```
/* 2nd parameter is always a bit mask which specifies which flags are set or reset */
```

```
/* Each bit of 3rd parameter specifies whether corresponding flag is 1 or 0 */
```

```
EventFlagChange(FlagsA, 0xFF, 0x00); /* set all flags to 0 */
```

```
EventFlagChange(FlagsA, 0x11, 0xFF); /* set flags 4 & 0 to 1 */
```

```
/* Waiting on flag conditions */
```

```
/* 2nd parameter is mask which specifies which flags are considered */
```

```
/* 3rd parameter specifies value required of each flag for wait to end */
```

```
EventFlagWaitAll(FlagsA, 0xF0, 0xF0); /* wait until flags 7,6,5,4 are all set */
```

```
EventFlagWaitAny(FlagsA, 0xF0, 0x30); /* wait until any of 7,6 is reset or 5,4 is set */
```

Cf Event Flags & Semaphore

- ◆ Sets of event flags allow wait on multiple variables
- ◆ Any number of tasks can wait on a given boolean proposition, when this is true all tasks will be woken up together
 - ❖ A future task waiting on the same proposition will not block (if the system has not changed).
 - ❖ This is different from a semaphore where even if it is flushed this will not allow future tasks to proceed.
- ◆ Event flags allow simple implementation of the barrier synchronisation problem in two different ways
 - ❖ Mutex variable & one event flag
 - ❖ Multiple event flags
- ◆ The disadvantage of using event flags for synchronisation is that their value must be reset before they are reused for another synchronisation operation
 - ❖ This is different from semaphores which "auto-reset" on waking a task

Solution 2: Mutex Variable & Event Flag

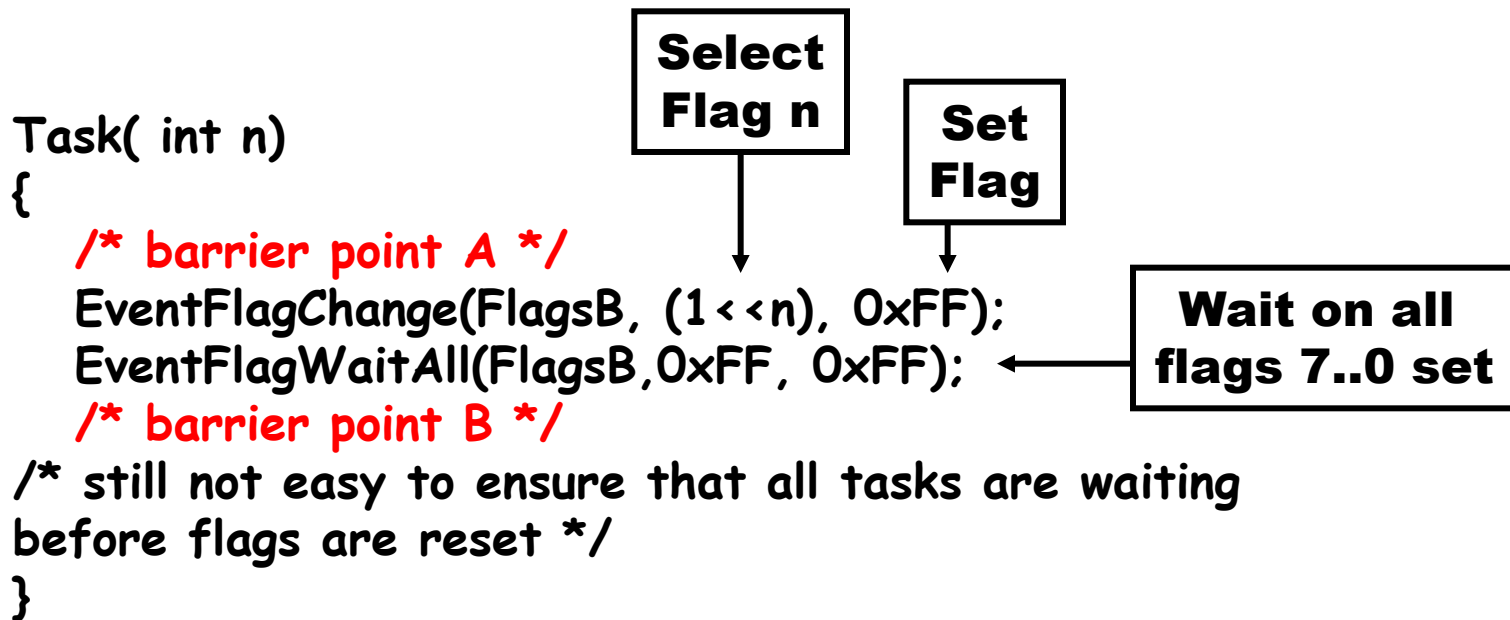
- ◆ The first semaphore can be eliminated if we use a small critical section of code
 - ❖ of course, this could be implemented using a semaphore to enforce mutual exclusion
- ◆ The function **Barrier()** below is called by all tasks to synchronise when they reach the barrier. It performs all the necessary synchronisation.
- ◆ FlagsB must be created before this code is executed
 - ❖ BarrierInit() resets everything for the next synch.
 - ❖ Not easy to ensure all tasks are waiting before BarrierInit() is called

```
void Barrier()
{
    /* Barrier point A */
    ENTER_CRITICAL_SECTION();
    Synch = Synch+1;
    if (synch == NSYNCH) {
        EventFlagChange(FlagsB,1,1);
    }
    EXIT_CRITICAL_SECTION();
    EventFlagWaitAll(FlagsB,1,1);
    /* Barrier point B */
}
```

```
void BarrierInit()
{
    Synch=0;
    EventFlagChange(FlagsB,1,0);
}
```

Solution 3: Multiple event flags

- ◆ Each task to be synchronised sets one event flag on reaching the barrier & then waits on all flags set



/* Assume 8 tasks created with n = 7..0 */

Solution 4: Multiple event flags

Can avoid reset by using **differential** change in flag state.
Invert value of flag each time barrier is crossed.
Must keep track of old flag values in variable, or read it via API function.

```
Task( int n)
{
    char flagValue=0x00; /* either 0x00 or 0xFF */
    for (;;) {
        /* barrier point A */
        flagValue = flagValue ^ 0xFF; /* invert value of flag
        EventFlagChange(FlagsB, (1<<n), flagValue); /*mark this task at A*/
        EventFlagWaitAll(FlagsB,0xFF, flagValue);/*wait till all tasks at A*/
        /* barrier point B */
    }
}
/* Assume 8 tasks created with n = 7..0*/
```

Lecture 6 Summary

- ◆ Barrier synchronisation is often necessary when multiple tasks cooperate on solution of a problem
 - ❖ Can be solved using semaphores
 - May need to use counting semaphores
 - ❖ Better solutions use event flags
 - One flag and a mutex variable
 - One flag per task to be synchronised
- ◆ Event flag registers allow waiting on arbitrary AND or OR of boolean condition variables "flags"
- ◆ Event flags are important because there are many synchronisation problems which cannot easily be solved by semaphores.

Property	Semaphore	Mutex
Basic	Semaphore is a signalling mechanism and a thread waiting on a semaphore can be signaled by another thread.	Mutex is a locking mechanism that makes sure only one thread can acquire the mutex at a time and enter the critical section
Existence	Semaphore is an integer variable. Semaphore is usually for processes	Mutex is an object. Mutex is usually for thread
Function	Semaphore allow multiple program threads to access a finite instance of resources.	Mutex allow multiple program thread to access a single resource but not simultaneously.
Ownership	Semaphore value can be changed by any process acquiring or releasing the resource.	Mutex object lock is released only by the process that has acquired the lock on it.
Nature	Semaphore is atomic but not singular in nature.	Mutex is typically atomic and singular in nature.
Categorize	Semaphore can be categorized into counting semaphore and binary semaphore.	Mutex is not categorized further.
Operation	Semaphore value is modified using wait() and signal() operation.	Mutex object is locked or unlocked by the process requesting or releasing the resource.
Resources Occupied	If all resources are being used, the process requesting for resource performs wait() operation and block itself till semaphore count become greater than one.	If a mutex object is already locked, the process requesting for resources waits and queued by the system till lock is released
Release Of The Mechanism	Semaphore value can be changed by any process acquiring or releasing the resource.	Mutex object lock is released only by the process that has acquired the lock on it.
Thread & Process	Only one process can acquire binary semaphore at a time but multiple processes can simultaneously acquire semaphore in case of counting semaphore.	Only one thread can acquire a mutex at a time.
Working	Semaphore works in kernel space.	Mutex works in userspace.
Ownership Concept	The concept of ownership is absent in semaphore.	The thread with mutex has ownership over the resource.

Conclusion:

Semaphore is a better option in case there are multiple instances of resources available. In the case of single shared resource mutex is a better choice.

Lecture 7: Scheduling Theory

**Deadlines are things that we pass through on
the way to finishing.**

Peter Gabriel

- ◆ In RTOS applications tasks have deadlines. This lecture will look at what these are, and how design can ensure that they are met.
- ◆ This lecture has two parts
 - ❖ 1. How are tasks scheduled?
 - ❖ 2. Deadline guarantees through Rate Monotonic Analysis

Scheduling

- ◆ At any time during execution of an RTOS there is a set of READY tasks.
- ◆ Only one task can be running at a given time
 - ❖ The scheduler must decide which of the READY tasks runs
- ◆ In implementations, there are two separate decisions to be made about scheduling:
 - ❖ **When** does scheduling happen (with possible switch in running task)?
 - ❖ **Which** task is selected to run?
- ◆ Most RTOS have a **scheduler** internal function which is called whenever scheduling is allowed to happen and which selects the next task to run and (if it is not already running) performs a **task switch**
 - ❖ We look at the mechanics of the task switch in Part 2

When to schedule?

- ◆ All RTOS must schedule when the current running task calls a blocking API function.
 - ❖ Otherwise the system would halt!
- ◆ Other than this, we will examine three commonly used choices for when to schedule:
 - ❖ **Non-preemptive (coroutine) scheduling**
 - All scheduling is explicitly allowed by the running task
 - At other times preemption is not allowed
 - ❖ **Preemptive scheduling**
 - The RTOS scheduler is called whenever something happens which may change the scheduling decision:
 - ◆ Task priority changing
 - ◆ Task changing state
 - ❖ **Preemptive scheduling with time-slicing**
 - The scheduler is called from system clock tick even when no task has changed state

Co-routine scheduling

- ◆ Co-routines are tasks which must co-operate in their own scheduling.
 - ❖ An API function **TaskYield()** allows any task to call the scheduler
 - Will either return with no operation or if another task needs to run will result in task switch
 - ❖ Blocking API functions call the scheduler when they block
 - ❖ Otherwise scheduling cannot happen
- ◆ **Advantages**
 - ❖ Scheduling is always via function call. Although separate stacks are still needed and context must be saved on stck, but there is less context to save.
 - The task-switch looks to the compiler like a function call/return. Registers which the compiler expects to be changed by a function call therefore do not need to be saved
 - Less context to save ➔ faster context switch, smaller stacks
 - ❖ Critical sections are no longer necessary! No code can be interrupted by another task unless it explicitly calls the scheduler or a blocking API function.
 - This makes code more efficient, and easier to write.
 - ❖ RTOS Kernel scheduling code is usually simpler

Co-routine scheduling (cont'd)

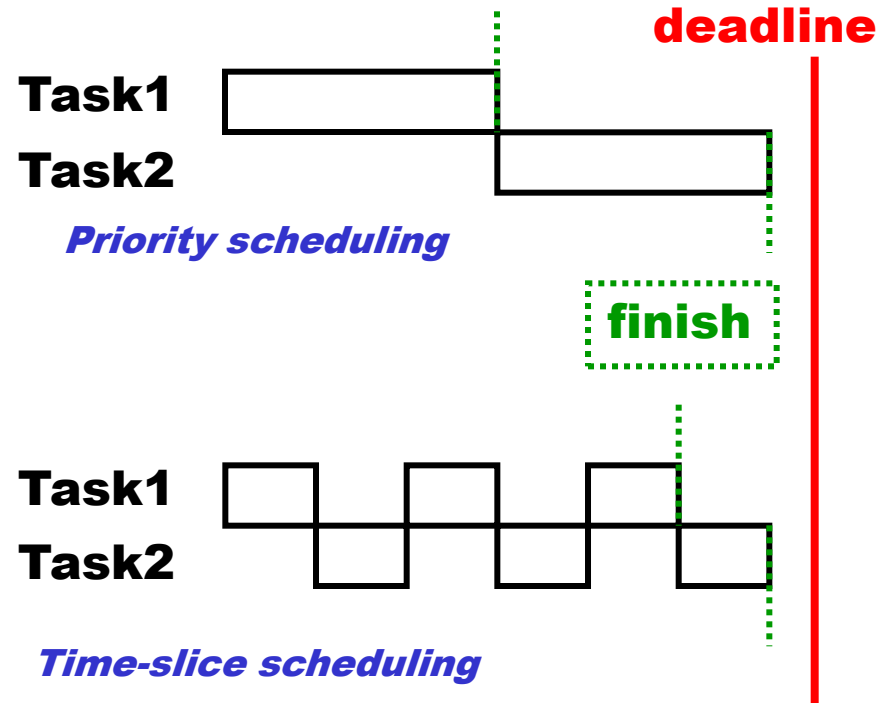
- ◆ The advantages on the previous slide mean that co-routine based systems are very compact and efficient.
- ◆ There are however big disadvantages which mean that most RTOS (and nearly all large RTOS) use preemptive scheduling.
- ◆ **Disadvantages**
 - ❖ All application code must be written correctly – **TaskYield()** must be called within any loop that may last a long time.
 - A single "rogue" task will freeze the entire system
 - It is possible (but complex) to require compilers automatically to insert TaskYield() within every loop.
 - ❖ Task-level response time is not easily guaranteed, since it depends on maximum length of time between **TaskYield()** calls.
 - ❖ Calls to **TaskYield()** in inner loops can slow down execution
- ◆ **Note that co-routines and tasks can co-exist within one system** – so getting the advantages of both at the cost of some complexity
 - ❖ FreeRTOS allows mixed co-routines and tasks
 - ❖ Co-routines run within a single task

Preemptive scheduling

- ◆ **Preemption** means task switches as the result of an interrupt
 - ❖ From hardware device interrupt, where the ISR signals an OS object and unblocks a task
 - ❖ From timer interrupt which drives the system clock tick and may wake up tasks waiting on delay.
- ◆ The implementation cost is that task switch must be implemented both from a task (blocking API function) and direct from interrupt.
 - ❖ Since all state must sometimes be saved, all state must always be saved to make task switch uniform.
 - ❖ **Task-level task switch** is usually implemented using a software interrupt followed by the same mechanism as for **interrupt-level switch**
- ◆ **Advantage:** can ensure that at all times the highest priority task which is READY will be running.

Time-slice (round-robin) Scheduling

- ◆ This is a small addition to preemptive scheduling. Sometimes it is useful to have tasks which share execution time. This can be implemented in a time-slicing system by giving tasks the same priority. The scheduler will allocate to each task a time-slice before switching to the next task.
- ◆ Surprisingly, this feature is **not usually what is needed** in a real-time system. Given two tasks of equal priority (same deadline):
 - ❖ All that matters is do they both meet the deadline
 - ❖ This is no more likely if they time-slice
 - In fact slightly less likely since the switching consumes CPU time
 - ❖ If one task finishes first it will release system resources which may help other tasks.



**Time-slice scheduling
is fairer but overall
worse for finish times**

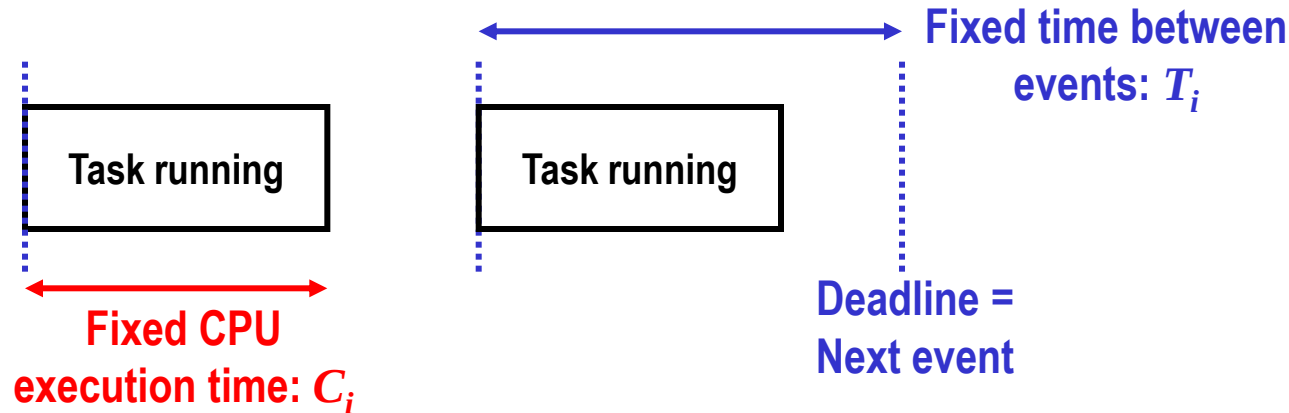
Which task to schedule?

- ◆ Nearly all RTOS use **priority-based scheduling**. This allows tasks with tight deadlines to run faster, as is usually required.
 - ❖ Assign to each Task i a priority π_i
 - ❖ Scheduler (called whenever task blocks or unblocks):
 - If the current RUNNING task is READY and has highest priority make no change
 - Otherwise schedule the READY task with highest priority
- ◆ In an RTOS no task (except for the lowest priority task) should run with 100% CPU utilisation, since this will block lower priority tasks
 - ❖ In an RTOS tasks unblock in response to some event, implement some real-time operation, and then block again
 - ❖ Priority scheduling works well

Deadlines

- ◆ Real-time systems are classified according to what happens if deadlines are missed
- ◆ **Hard Real-time**
 - ❖ Missing deadlines is disastrous
 - ❖ E.g. aeroplane crashes
 - ❖ Hardware failure
 - ❖ Nuclear power plants (!)
- ◆ **Soft Real-time**
 - ❖ Missing deadlines degrades performance & is undesirable, but not catastrophic as long as infrequent
 - DSP systems for real-time voice processing
 - Airline real-time booking processing systems
- ◆ Either way, real-time systems have deadlines
 - ❖ Contrast this with application tasks on a PC where there is no fixed completion deadline, though speed is also desirable.

Job model for tasks in an RTOS

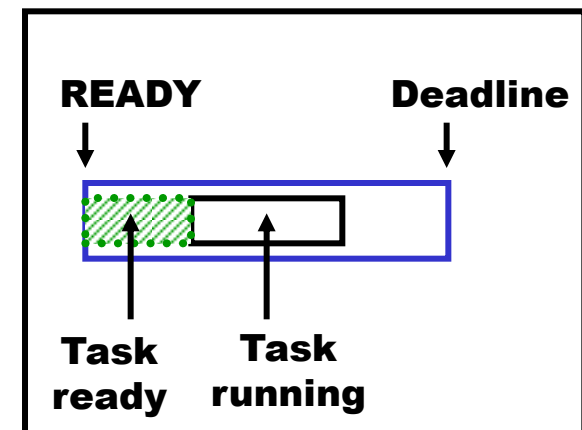
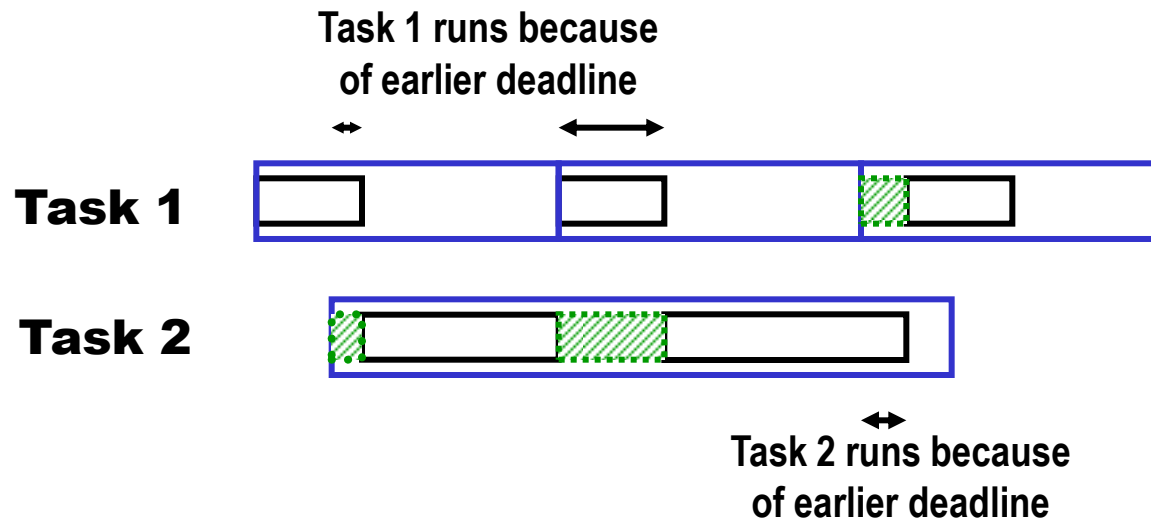


- ◆ Assume tasks wait on a periodic event and then have a fixed amount of CPU time to use before they block on the next event
- ◆ For correct operation the work associated with an event must be complete before the next happens
- ◆ Good model for many RTOS tasks
 - ❖ Ignores task synchronisation & resource access blocking
- ◆ Allows analysis of different scheduling strategies

```
Task()
{
    for ( ; ; ) {
        [ wait on next event ]
        [ process event ]
    }
}
```

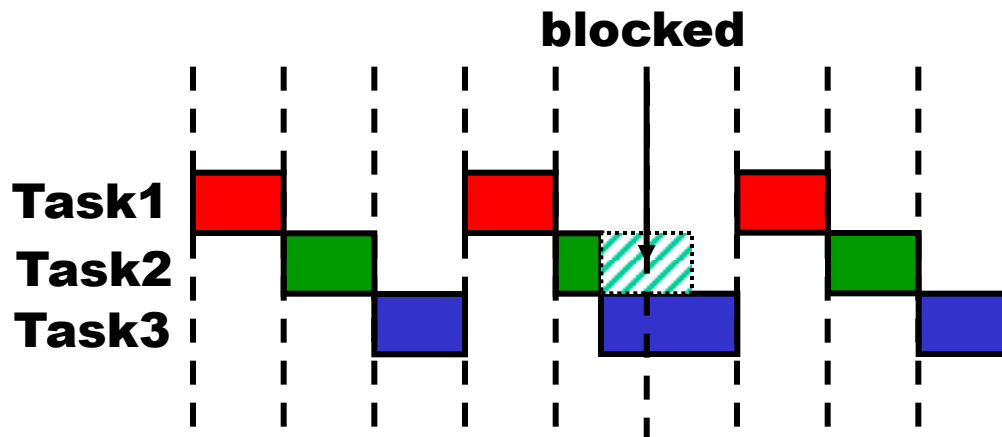
Earliest Deadline First (EDF) Scheduling

- ◆ Fixed **Priority scheduling** is not optimal for meeting deadlines.
- ◆ If deadlines are known in advance then in principle EDF scheduling will meet deadlines if any scheduling strategy can do this.
 - ❖ At any time schedule the READY task whose deadline will happen first
 - ❖ Not difficult to prove that this is optimal (if tasks block once per deadline)
- ◆ In practice not often used
 - ❖ Very time-consuming to implement as number of tasks increases
 - ❖ Difficult to get information on when future deadlines will happen



Round-robin scheduling

- ◆ Feature can be added to priority scheduling.
 - ❖ Allow tasks to have equal priority.
 - ❖ Run set of equal priority tasks in equal time-slices and strict rotation
 - When a task blocks before the end of its time-slice start the next one early
 - ❖ As pointed out earlier this strategy is not usually good for RTOS where **early completion** is more important than **fairness**.
 - ❖ Round-robin scheduling simple. It has the merit that READY tasks are guaranteed to be given a time-slice within a given time.



Why is it not good to switch round-robin to a new task every time-slice?

Rate Monotonic Analysis

- ◆ Suppose all n tasks in an RTOS follow the **job model** (slide 1.123)
 - ❖ Task i executes with CPU time $\leq \tau_i$
 - ❖ Task i waits on event with period $\geq T_i$
 - ❖ Task i has no other blocking (ignore synchronisation with other tasks)
- ◆ Schedule task i with fixed priority π_i so that faster deadlines have higher priority
 - ❖ $T_i \leq T_j \Rightarrow \pi_i \geq \pi_j$
- ◆ Then the system is guaranteed to meet all deadlines providing the total CPU Utilisation U is less than the **RMA limit** for n tasks $n \geq 1$
 - ❖ $U = \sum \tau_i / T_i \leq n \times \frac{1}{2^{n+1}}$
- ◆ Note that variable times are allowed as long as they obey the given inequalities

Example

- ◆ Consider 3 tasks as in the table
- ◆ The priorities must be assigned as shown, inversely with period.
- ◆ The total CPU utilisation U is 0.767
- ◆ The RMA limit is
$$\square(3) = 3(2^{1/3}-1) = 0.780$$
- ◆ Therefore the system meets the RMA limit and is guaranteed to meet all deadlines.

Task	T	C	U	Priority
1	30ms	5ms	0.167	3
2	10ms	3ms	0.333	1
3	15ms	4ms	0.267	2

RMA discussion

- ◆ Rate Monotonic Analysis provides a guarantee that systems obeying the RMA conditions WILL always meet deadlines.
 - ❖ It does not say that systems with higher utilisation will NOT meet deadlines.
- ◆ It is seldom possible to apply it exactly because most systems do have some inter-task synchronisation which slows down tasks
- ◆ In practice, it is a good guide as to what is a reasonable system
- ◆ The RMA utilisation limit $\frac{\sum C_i}{\sum T_i}$ decreases monotonically with increasing n and has asymptotic value of $\log_e 2 = 0.693$
 - ❖ As a rule of thumb RTOS with many tasks should never run with utilisation higher than this limit.
 - ❖ For safety, most realisable designs should run well below the RMA limit
- ◆ If the RMA limit is not met, for a given application, the solution is either to use a faster CPU, or, possibly, reduce the number of tasks.

Extended RMA

- ◆ We can include the effect of tasks blocking due to inter-task mutual-exclusion etc.

- ❖ Suppose the maximum time Task i can block before it completes is β_i
- ❖ Replace α_i by $(\alpha_i + \beta_i)$ in the RMA limit calculation

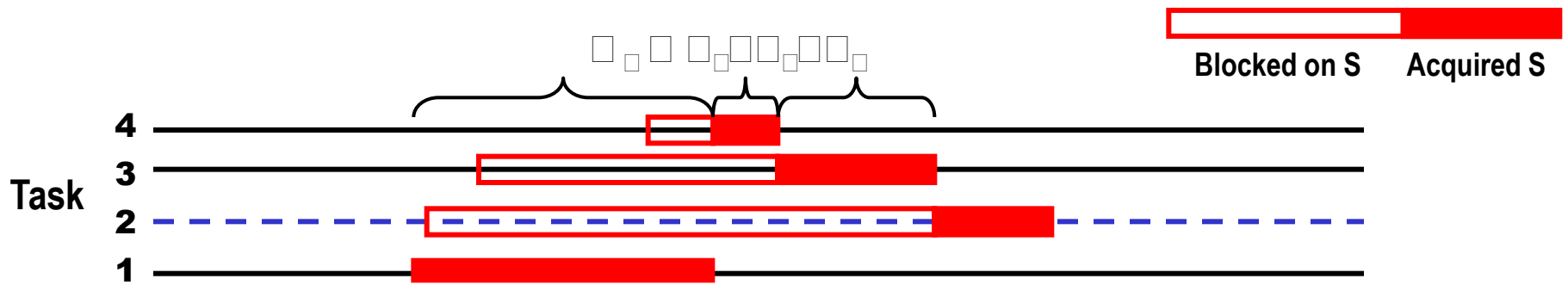
$$\sum ((\alpha_i + \beta_i) / T_i) \leq 1$$

- ❖ NB - blocking is not the same as waiting to run at a lower priority, which is included in RMA limit. Therefore β_i is the sum of all blocking on lower priority tasks, and blocking on higher priority tasks **only when they are themselves blocked**.

- ◆ This can be helpful where an upper bound can be put on blocking through mutual exclusion
- ◆ Where no such upper bound exists the system is unsafe and should not be used!

Estimating blocking B_i

- ◆ Suppose Task i blocks due to access to a shared resource governed by semaphore S , and has no other blocking while it executes. **Assume**
 - ❖ During each computation τ_i the task i claims S at most *once*.
 - ❖ The **maximum** time (critical section length) for which any task τ claims S is τ_c
 - NB we do not consider priority inversion here – which can increase this maximum time to longer than expected – see Lecture 9
 - ❖ No task can claim S more than once while a given task is waiting
- ◆ Worst case, when waiting on semaphore S , Task i may have to block while **one task of lower priority** claims the semaphore: $\tau_j = \max\{\tau_k \mid \tau_k < \tau_i\}$
 - ❖ Tasks of higher priority claiming semaphore **do not count as blocking**, since they would run anyway and having claimed semaphore cannot block



Further reading on scheduling

- ◆ A good accessible overview of hard real-time scheduling issues and deadline-based methods of scheduling from Peter Dibble
 - ❖ <http://www.embedded.com/showArticle.jhtml?articleID=9900112>

Lecture 7: Summary

- ◆ Scheduling can be through co-routines, or **preemptive**
 - ❖ Co-routines task-switch faster, and have some programming advantages, but suffer from unpredictable and longer task response time.
- ◆ Scheduling can be **priority-based**, or (much more complex) using **EDF**
 - ❖ EDF meets deadlines better but is usually too difficult to implement since deadlines cannot be easily be precisely determined in advance.
- ◆ **Round-robin scheduling** allocates time fairly to tasks of equal priority
 - ❖ Provides fairness at the expense of quicker completion
- ◆ Priority-based scheduling can be analysed for a **job model** of tasks using **Rate Monotonic Analysis (RMA)** which subject to certain conditions guarantees all deadlines will be met if the CPU utilisation ρ is less than the **RMA limit** $\rho(\rho)$.

Lectures 8 & 9: Liveness Problems in Real-Time Systems

Never discourage anyone...who continually makes progress, no matter how slow.

Plato

If debugging is the art of removing bugs, then programming must be the art of inserting them.

Unknown

- ◆ Real-time systems must have correct **timing**, with all deadlines met, as well as correct **function**.
- ◆ Rate Monotonic Analysis only guarantees timing when tasks do not block, or have limited time blocking
- ◆ This lecture looks at scheduling problems which can cause tasks to block indefinitely or for much longer than expected
 - ❖ Collectively these are called **liveness problems**
- ◆ We will consider
 - ❖ How to predict and avoid problems
 - ❖ How to correct problems

Anatomy of problems

- ◆ We will look at four common types of problem which mean tasks make no or slow progress:
 - ❖ Deadlock
 - ❖ Starvation
 - ❖ Livelock
 - ❖ Priority Inversion
- ◆ All have the characteristic that they are scheduling-dependent.
 - ❖ Given different a different scheduler they need not happen.
 - ❖ Contrast this with CPU utilisation $> 100\%$ which is a problem that cannot be cured by ANY schedule.
 - ❖ RMA does not help with these problems
 - RMA assumes tasks do not block waiting on resources

Deadlock

- ◆ A set of tasks D is deadlocked if
 - ❖ All tasks in D are blocked indefinitely
 - ❖ Regardless of future scheduling all tasks in D will stay blocked
 - No way out of deadlock
 - ❖ If the set of tasks is the whole system no further computation can be done.
- ◆ Deadlock is a global property of a system (or part of a system) it cannot easily be detected and avoided at a task level.
- ◆ Deadlock can occur very easily
 - ❖ programmers who do not consider it will almost certainly find that any medium-sized system deadlocks.
 - ❖ Therefore understanding it and avoiding it through analysis is essential
- ◆ Global mechanisms to detect and recover from deadlock exist, however it is usually essential to avoid deadlock in the first place

A simple example

- ◆ Consider two tasks T1 & T2 which are part of a concurrent system and shared resources A and B. The resources could be shared memory, hardware, etc.
 - ❖ Each resource is protected by a semaphore (Sa and Sb)
 - ❖ To perform part of the computation each task needs to use both A and B.
- ◆ Can you see what is wrong with the code?
 - ❖ Mutual exclusive use of A & B is guaranteed
 - ❖ Each task claims and then releases each semaphore once, as it should.
- ◆ The two tasks can deadlock!
 - ❖ A1,A2,B2(T2 blocks on Sa),B1(T1 blocks on Sb).....

```
T1()  
{
```

```
    A1: [ acquire Sa ]
```

```
    B1: [ acquire Sb ]
```

```
    C1: [ perform computation using A and B ]
```

```
    D1: [ release Sb ]
```

```
    E1: [ release Sa ]
```

```
}
```

```
T2()  
{
```

```
    A2: [ acquire Sb ]
```

```
    B2: [ acquire Sa ]
```

```
    C2: [ perform computation using A and B ]
```

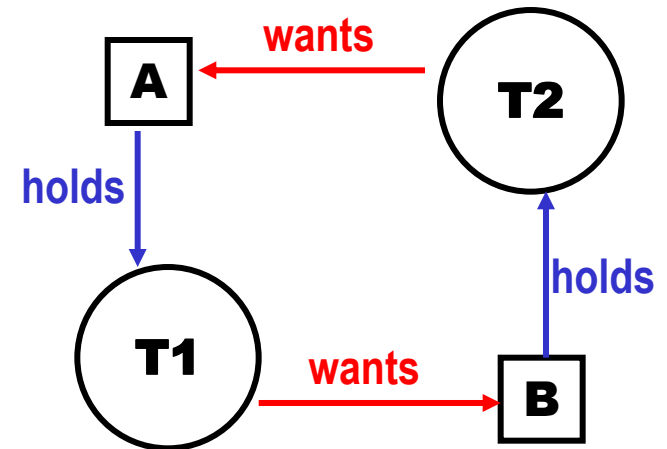
```
    D2: [ release Sa ]
```

```
    E2: [ release Sb ]
```

```
}
```

Discussion

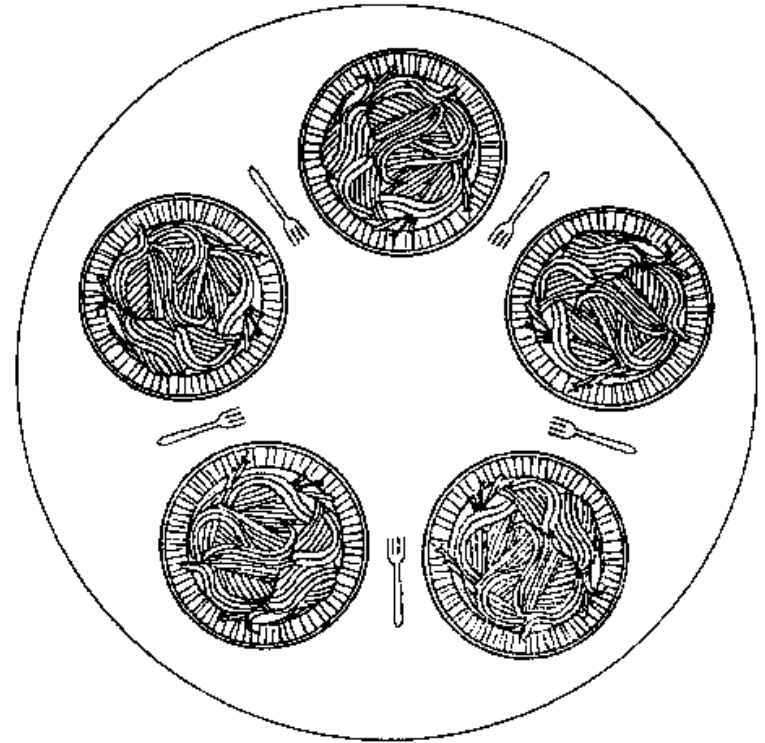
- ◆ This example reveals some interesting features of deadlock
 - ❖ **Attaining the deadlocked state depends on scheduling.**
 - In this case one of the tasks must acquire its first semaphore during the (short) time between when the second task acquires its first & second semaphore
 - ❖ **The deadlock, once achieved, is non-recoverable.**
 - We will consider later more complex systems which do allow recovery, providing the resource is **preemptible**.
 - ❖ **The deadlock is caused by a cyclic dependence between tasks, each of which wants a resource held by another task**
 - Cycle can include N tasks & N resources ($N \geq 2$)



Resource graph illustrates cyclic dependence

The Classic problem

- ◆ Deadlock can involve any number of tasks. The classic example is the so-called Philosopher's Dining Problem.
 - ❖ Philosophers eat at a circular table
 - ❖ Each Philosopher needs two forks to eat
 - ❖ The table is laid with each fork shared between two philosophers
 - ❖ Incorrect scheduling will lead to all philosophers having picked up one fork, waiting on the other one.



Conditions for Deadlock

◆ Non-preemptability

- ❖ Resource once allocated can't be released until task has finished.
- ❖ Don't confuse this with *task preemption*

◆ Exclusion

- ❖ Resource can't be held simultaneously by two tasks

◆ Hold-and-Wait

- ❖ Holder blocks awaiting the next resource

◆ Circular waiting

- ❖ tasks acquire resources in different orders

Strategies

- ◆ Deadlock is a key problem in concurrent system design. There are three types of strategy to deal with it:
 - ❖ Deadlock **detection & recovery**
 - ❖ Deadlock **avoidance**
 - ❖ Deadlock **prevention**
- ◆ One or other of these is essential for correct operation of any complex concurrent system

Deadlock Detection & Recovery

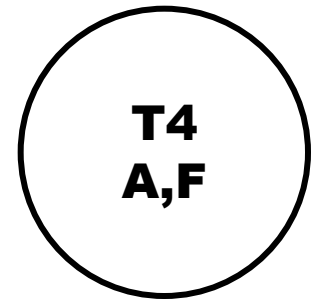
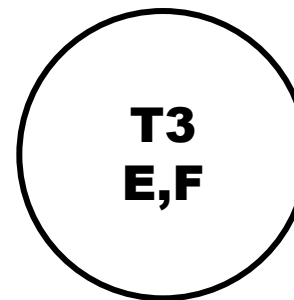
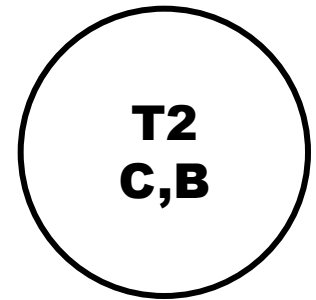
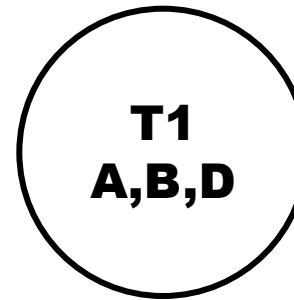
- ◆ A **stable deadlock** (as on previous slide) is where none of the deadlocked tasks expect a timeout or abort to break the deadlock
- ◆ **Deadlock detection** is a global algorithm used by the RTOS to detect deadlock situations and recover – externally from the tasks which are deadlocked.
 - ❖ Can be useful as a debugging aid, but in practice recovering from deadlock is difficult
- ◆ **Temporal deadlock** is a situation where one or more of the deadlocked tasks time out or aborts, thus freeing the requested resource and eliminating the deadlock.
 - ❖ Here the recovery mechanism is local to a single task
- ◆ Recovery from deadlock requires a reversal of previous resource allocation
 - ❖ Either **preempt the resource**, if this is possible without disturbing task execution
 - ❖ Or **roll back the task to a checkpoint** before the resource was allocated, and free the resource.

Deadlock Avoidance

- ◆ Deadlock requires a cyclic resource graph. The RTOS can track resource dependence and detect when a resource allocation request will lead to a cycle. The wait operation can then return with an error indication.
 - ❖ This is like global detection except that errors are discovered earlier and therefore recovery is easier.
 - ❖ Detecting cycles in the resource graph is however resource-intensive and requires a graph analysis algorithm to run every time a resource is requested or released.

Deadlock Prevention

- ◆ Best technique for small to medium-sized systems.
 - ❖ Simple & robust
 - ❖ No run-time cost
 - ❖ Disadvantage: relies on conservative resource use so may make systems slower than is possible.
- ◆ Establish a global allocation **order** on resources such that when resources A,B are used at the same time by a given task, either $A < B$ or $B < A$.
- ◆ Constrain all tasks so that resources are **acquired** (`SemaphoreTake()` etc) according to this order.
 - ❖ It does not matter in what order resources are released.



Choose order, e.g.:

A,B,C,D,E,F

→ T2 must acquire B before C etc

Using Timeouts

- ◆ There are cases where some known error condition (other than deadlock) can be detected using a timeout.
 - ❖ In this case the timeout is part of normal operation
- ◆ More often long delays in obtaining a resource are not expected
 - ❖ A timeout indicates an error condition
- ◆ Defensive programming
 - ❖ Use long timeouts – should never happen
 - ❖ Stop system with error on timeout.
- ◆ Don't rely on this to detect deadlock conditions
 - ❖ Deadlock may be possible but never happen due to scheduling
 - ❖ Such a system is unsafe and may deadlock in the future as the result of any small change
 - ❖ Make sure deadlock is prevented by design, as on previous slide.

Starvation

◆ Symptoms

- ❖ One (or more) tasks wait indefinitely on a shared resource held by other, normally running, tasks.

◆ Cause

- ❖ Two or more *other* tasks are using the resource in turn, without break, denying the starved task usage.
- ❖ Total *resource utilisation* (analogous to CPU utilisation) is 100%
- ❖ If higher priority tasks have 100% CPU utilisation so preventing execution of lower priority task this is special case of starvation.
- ◆ Starvation depends in general on details of scheduling, it is trickier to diagnose than deadlock
- ◆ In starvation, a task is prevented from execution by other tasks, which themselves are executing normally

Starving Philosophers

- ◆ To illustrate **starvation** consider a variant of the 3 Philosopher's problem
 - ❖ Philosophers "big Ned" & "big Tony" have priority over "Tiny Tim"
 - ❖ Philosophers share a single ladle
 - ladle = semaphore-protected resource
 - ❖ Each philosopher spends a fixed time eating with the ladle, and then releases the ladle for another fixed time, during which they talk.
- ◆ Two philosophers eat while the other is literally starved
 - ❖ Note that the **ladle is 100% utilised**. If the talking time were increased to 31s it would be free for Tim 1s every half minute and all Philosophers would be able to share it.

Philosopher()

```
{  
    [ pick up ladle ]  
    [ wait 30 seconds (eating) ]  
    [ replace ladle ]  
    [ wait 29 seconds (talking) ]  
}
```

**Big
Ned**

**Big
Tony**



**Tiny
Tim**

Deadlock vs Starvation

- ◆ Deadlock and starvation are easily confused.
- ◆ One way to see the difference is that although both **start** in a way that is dependent on the task scheduling, once they have started:
 - ❖ Deadlock can't be broken by any task schedule (because of the cyclic resource dependence)
 - ❖ Starvation can be broken by a fairer task schedule (making the tasks that are hogging the resource execute less)
 - ❖ Fixed priority scheduling, used in RTOS, tend to be unfair & can thus generate starvation easily
- ◆ The previous slide example is typical of starvation – a task waits on a resource which is used alternately by a number of higher priority tasks, and therefore never available.

Livelock - a system designer's nightmare

- ◆ The problem in **livelock** is that although all tasks are seemingly executing normally (ie not suspended indefinitely) no progress is made.
 - ❖ The best analogy is two people wanting to pass in a passage, who by bad luck each move to the right and left at the same time, so neither can pass.
 - ❖ In principle (and in concurrent systems) this can continue indefinitely.
- ◆ There are two variants of livelock, explained on the next slides.
 - ❖ Pseudo-deadlock
 - ❖ Real livelock

Pseudo-Livelock

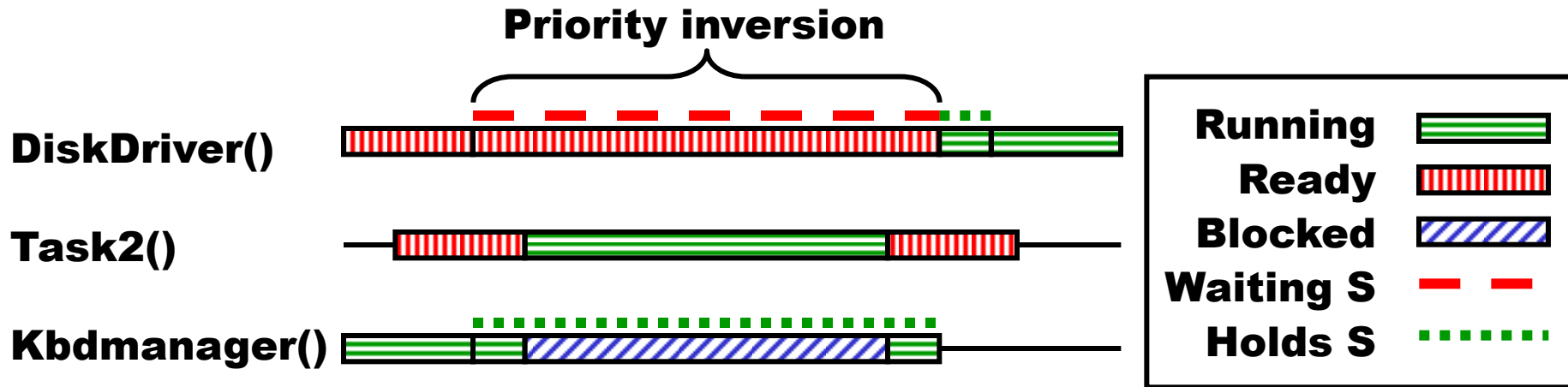
- ◆ For example, Dining Philosophers are all doing "**pickup left fork; busy/wait loop until right fork is free**" and pick up left forks at same time.
 - ❖ This is really a hidden form of the classic deadlock - the **busy/wait** polling loops make it seem that something is happening, when really the tasks are all waiting on resources.
 - ❖ In this case, as in deadlock, the pseudo-livelock can't be broken by any scheduling order.
- ◆ More interestingly, livelock may be dependent on scheduling, so that even after it occurs it could be broken by a different execution order.
 - ❖ See next slide

Real Livelock

- ◆ Consider the deadlocked philosophers with deadlock broken by a fixed timeout.
 - ❖ This is NOT a deadlocked system, since the shared resources are released after the timeout allowing the system to progress.
- ◆ Suppose that all pick up the first fork simultaneously, and all wait on the second fork at the same time. All will then timeout at the same time, and try again.
 - ❖ **The cycle will repeat indefinitely!**
 - ❖ Solution here is to add to each task a **different back-off time**
 - Task is unable to retry until back-off time expires
 - Typically use pseudo-random numbers to determine back-off time
- ◆ Ethernet protocol does exactly this to prevent livelock on ethernet networks due to repeated network collisions

Priority Inversion

- ◆ A high priority task **DiskDriver()** shares a resource with a low priority task **KbdManager()** using a semaphore S.
 - ❖ Assume no other READY tasks in the system
 - ❖ The resource is locked for a short time T by either task
 - ❖ When using S DiskDriver() must wait worst case for up to T while Keyboardmanager() finishes using S (priority inversion).
- ◆ Now suppose there is another task in the system **Task2()** which has priority just greater than **KbdManager()**. This can preempt **KbdManager()** while it holds S
- ◆ Effectively **DiskDriver()** is reduced to the priority of **KbdManager()** because **Task2()** runs in preference to it.
 - ❖ The period of priority inversion is now determined by **Task2()** & effectively **unbounded**.



Priority Inheritance Protocol (PIP)

- ◆ The solutions to this problem all use dynamic priorities. The idea is that **KbdManager()** should have its priority temporarily increased.
- ◆ Priority Inheritance Protocol (PIP)
 - ❖ Any task T which claims the semaphore S has priority dynamically increased to that of the highest priority task waiting on S
 - We say it inherits priority of this task.
 - ❖ Priority is increased to ensure this whenever a higher priority task waits on S
 - ❖ When T releases S it will drop back to its old priority
 - ❖ Priority inheritance is transitive.
- ◆ In PIP, a task can be blocked by a lower priority task in two ways
 - ❖ *Direct blocking* – when LPT has resourced needed by task
 - ❖ *Push-through blocking* - when task is prevented from executing due to LPT having inherited priority higher than task we call this
- ◆ PIP limits max blocking to at most one critical section for each semaphore, and also at most one critical section for each lower priority task
 - ❖ Schedulability (RMA etc) is determined by total blocking

Ceiling Priority Protocol (CPP)

- ◆ An different protocol keeps track for every resource R of the highest (static, not inherited) priority task that could use R – this is the static **ceiling priority** of the resource. The idea is then to give any task holding R its ceiling priority, whether or not any other task is waiting
- ◆ This speeds up critical sections of code and gets them out of the way more quickly.
- ◆ When compared with PIP it can therefore reduce **average** blocking in higher priority tasks, although it cannot reduce **worst-case** blocking

Priority Ceiling Protocol (PCP)

- ◆ Combines PIP with CPP
- ◆ The idea is to add some extra cases to where blocking occurs to PIP in order to reduce the overall time a task can be blocked by a lower priority task.
- ◆ This is a dynamic priority protocol which avoids priority inversion and **also avoids deadlock**.
- ◆ The **ceiling priority** for each resource is calculated as on the previous slide
- ◆ The **current priority ceiling** for a running system is defined to be the highest ceiling priority of any resource currently locked (i.e. with semaphore token given to a task).
 - ❖ Note that different tasks may hold each resource
- ◆ When a task T requests a resource R the PCP protocol states:
 1. If R is in use, T is blocked
 2. If R is free and the priority of T is higher than the current priority ceiling, R is allocated to T
 3. If the current priority ceiling is attained by one of the resources T currently holds, R is allocated to T, otherwise T is blocked.
 4. When T is blocked, the running task inherits T's priority if higher and executes at this priority until it releases every resource whose ceiling priority is higher than or equal to T's priority. The task then returns to its previous priority.

PCP example

- ◆ Task1 & Task2 have priority 1 & 2 respectively.
 - ❖ Both use resources A & B
 - ❖ They request these in opposite order so as to make a classic dining philosopher's deadlock possible
- ◆ Ceiling priority of A & B is 2.
 - ❖ Whichever Task first claims either A or B will raise current ceiling priority to 2.
 - ❖ The other task is therefore unable to claim A or B
 - ❖ The task which has claimed A or B will therefore always be able to claim the other resource
 - ❖ When both resources are released the current priority drops to 0 and then either resource may be claimed again.

```
Task1()
{
    [ claim A ]
    [claim B ]
    [release A]
    [release B]
}
```

```
Task2()
{
    [ claim B]
    [claim A]
    [release B]
    [release A]
}
```

PCP Analysis

- ◆ Under PCP, a task T can be blocked by a lower priority task for 3 different reasons:
 - ❖ *Direct blocking* - T is blocked because some other running task currently has the resource
 - This is unavoidable because of mutual exclusive access
 - ❖ *Push-through blocking* - T can be blocked when the blocking task has inherited a higher priority than T
 - Similar to what would happen under a priority inheritance protocol
 - This is necessary to prevent unbounded priority inversion
 - ❖ *Priority ceiling blocking* – if the priority ceiling of all resources currently locked by tasks *other than* T is higher than or equal to the priority of T , then T is blocked.
 - This prevents deadlock as proved in the next slide.

Blocking under PCP

- ◆ The total system blocking under PCP is more than under PIP, because of the extra condition (priority ceiling blocking) that can block tasks.
- ◆ However the maximum time a task can be blocked by *lower priority* task is now much less, and at most one critical section
 - ❖ Also deadlock is impossible
- ◆ This is the critical property that enables rate monotonic analysis to be done proving that all tasks will meet deadlines
 - ❖ The maximum blocking time for any task is now optimally bounded.

What Really happened to Pathfinder on Mars?

- ◆ A fascinating historical bug caused by priority inversion
- ◆ Bug caused catastrophic Pathfinder systems resets
 - ❖ Bug first occurred on Mars!
 - ❖ Diagnosed on Earth after careful simulation
 - ❖ Cured by bug fix uploaded from Earth
- ◆ Documents:
 - ❖ Informal account of the problem
http://research.microsoft.com/~mbj/Mars_Pathfinder/Mars_Pathfinder.html
 - ❖ And authoritative [follow-up](#) from Glen Reeves of JPL, who led the team finding the bug

Lectures 8 & 9: Summary

- ◆ Concurrent systems can suffer from:
 - ❖ **Deadlock**
 - ❖ **Starvation**
 - ❖ **Livelock – pseudo-livelock or real livelock**
 - ❖ **Priority inversion**
- ◆ Deadlock can be analysed by considering the **resource dependency graph** and dealt with through:
 - ❖ Detection & recovery
 - ❖ Avoidance
 - ❖ Prevention
- ◆ Protocols exist to stop priority inversion using dynamic task priorities:
 - ❖ **Priority Inheritance Protocol**
 - ❖ **Ceiling Priority Protocol**
 - ❖ **Priority Ceiling Protocol** (eliminates deadlock as well)